



HOJA DE PROBLEMAS 3: CAMINO DE DATOS Y CONTROL

Datos válidos para todos los ejercicios:

- Tiempo de acceso a la memoria (leer o escribir) = 1 ciclo
- Tiempo de acceso al banco de registros (leer 2 registros y escribir 1 registro) = 1 ciclo
- Tiempo para calcular una operación en la ALU = 1 ciclo
- Resto de tiempos = valor despreciable

1. **[Ejercicio de examen]** Se desea añadir una nueva instrucción al repertorio del procesador MIPS en su implementación multiciclo. Se ha bautizado la instrucción como **jj** y tiene la siguiente sintaxis en ensamblador, donde *dirección* es un inmediato de 26 bits codificado en la instrucción:

jj *dirección*

El funcionamiento de la instrucción es el siguiente:

- Tras leerse la instrucción de la memoria, se toman los 26 bits de *dirección* y se transforman en 32 bits con un procedimiento similar al de la instrucción **j**.
- La dirección de 32 bits obtenida se lee de la memoria.
- La palabra leída en dicha dirección de memoria corresponde a la dirección de memoria de la siguiente instrucción que hay que ejecutar (salto incondicional).

Se pide:

- a) Desde el punto de vista de la **arquitectura** (tal cual se vería en una tabla de resumen de instrucciones), escribir qué operación hace esta instrucción en notación de **pseudocódigo**.
- b) Justificar qué **formato binario** de instrucción sería el más adecuado para esta instrucción y descomponerla en campos. Habrá que indicar también **todos los operandos** de la instrucción, así como los **modos de direccionamiento** de cada uno de ellos.
- c) Explicar razonada y detalladamente **todos los cambios** que son necesarios en el camino de datos para poder ejecutar esta instrucción. Los requisitos que debe cumplir la instrucción son los siguientes:
 - El hecho de añadir esta nueva instrucción debe ser retrocompatible con todas las instrucciones estándar, que deben seguir siendo ejecutables como hasta ahora.
 - El número total de ciclos para ejecutar la nueva instrucción debe ser el mínimo posible. Para ello se podrán solapar en el mismo ciclo operaciones simultáneas que no interfieran entre sí.

- El número total de estados de la unidad de control debe ser el mínimo posible.
- El número total de cambios en el camino de datos debe ser el mínimo posible y también estos cambios han de ser lo más pequeños posible.

Este apartado tiene tres puntos de vista interrelacionados que habrá que explicar:

- Por un lado, habrá que **explicar** las modificaciones y **dibujarlas** sobre el camino de datos, tales como añadir cables o ampliar otros existentes, añadir multiplexores o ampliar otros existentes, añadir registros o ampliar otros existentes, añadir puertas lógicas o ampliar las existentes, etc.
- Por otro, habrá que **explicar, ciclo a ciclo**, qué operaciones se realizan (desde el punto de vista de la **organización**, es decir, indicando los componentes concretos del camino de datos involucrados). Para ello, se deberá usar la notación de **pseudocódigo** con el operador de asignación flecha $\leftarrow$ para indicar todas las operaciones que intervienen en dicho ciclo. Se pide hacer esto para **todos los ciclos** de la instrucción.
- Finalmente, habrá que ampliar el **diagrama de estados** de la unidad de control, así como la **tabla de verdad** de las salidas de la **unidad de control** para que incluyan los cambios realizados. Ambos se adjuntan con el enunciado.

Solución:

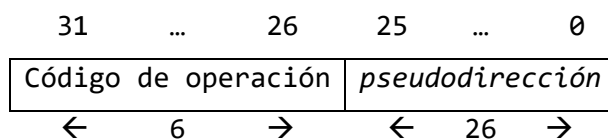
a) Esta instrucción realiza las operaciones:

$PC \leftarrow PC + 4$

$PC \leftarrow M[PC[31-28] || (dirección \ll 2)]$

IMPORTANTE: Esta es la notación de pseudocódigo de las operaciones realizadas por la instrucción **desde el punto de vista de la arquitectura**. Es lo que aparecería en una hoja de resumen de instrucciones y es independiente de la organización uniciclo o multiciclo del camino de datos. Solo pueden aparecer los registros visibles desde la arquitectura, como el **PC** o los del banco de registros, pero no **A**, **B**, **SalidaALU**, etc., porque esos corresponden exclusivamente a la organización multiciclo.

b) De los tres formatos de instrucción de MIPS, el que mejor encaja es el J, ya que especifica una dirección de memoria mediante una pseudodirección en binario puro con 26 bits. Por tanto, tendremos:



Esta instrucción tiene estos operandos:

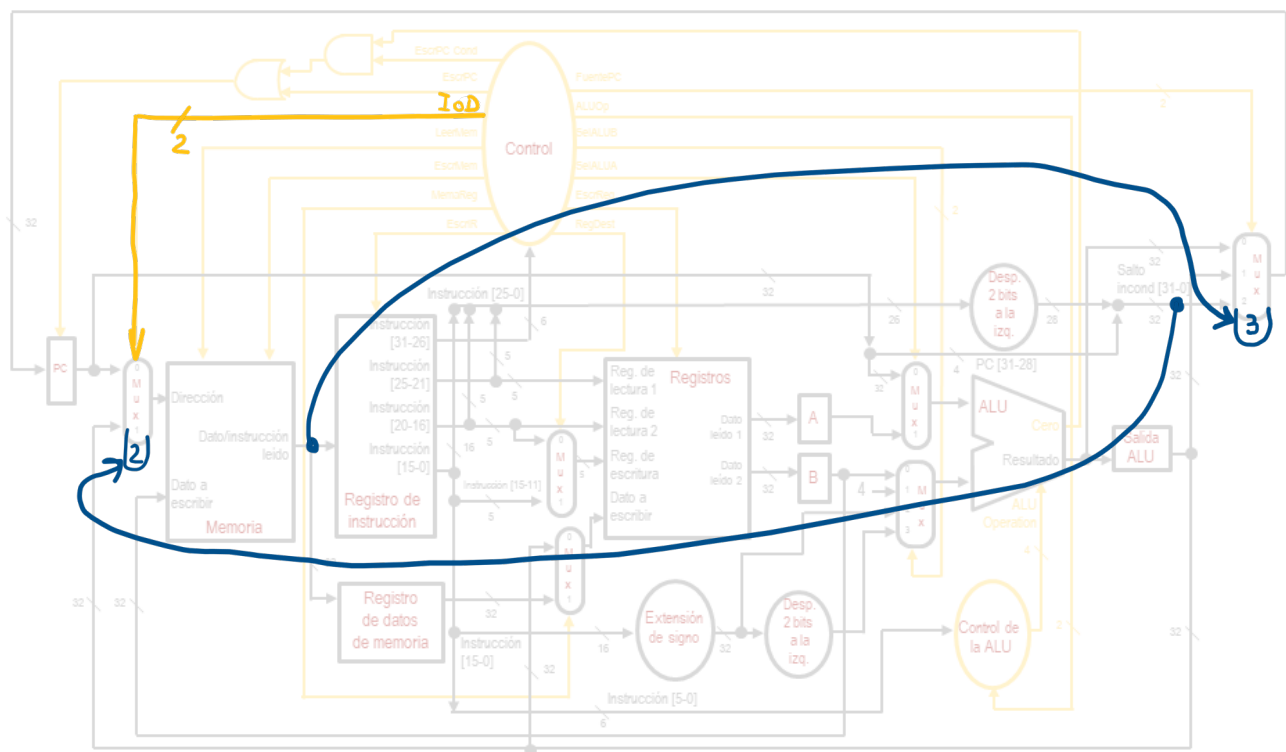
- La dirección de la instrucción destino del salto, que se especifica mediante la pseudodirección de 26 bits contenida en la instrucción y una lectura en memoria. La dirección de memoria que debemos leer se calcula mediante un direccionamiento pseudodirecto: se toma el valor de 26 bits, se multiplica por 4 (desplazamiento de 2 posiciones hacia la izquierda) y los 4 bits superiores se copian del valor del registro **PC** (cuando se haga esta operación el registro ya valdrá $PC + 4$). El resultado de esta operación es la dirección de memoria que leemos, con lo que habría también una indirección. Por tanto, el modo de direccionamiento de la dirección destino del salto no existe en MIPS, pero podríamos llamarlo como “pseudodirecto indirecto”.

- El registro **PC**, que primero se actualiza sumándole 4 y que luego recibe la dirección destino del salto, pero no se menciona en ningún lugar de la instrucción (modo de direccionamiento implícito).

c) Las modificaciones que se proponen son las siguientes:

- Llevamos un cable nuevo desde la entrada 2 del multiplexor controlado por la señal **FuentePC** hasta la entrada de direcciones del multiplexor controlado por **IoD**. Para ello, se amplía este multiplexor y la señal **IoD** pasa a ser de 2 bits (**IoD1 IoD0**). El nuevo cable se conecta a la entrada 2.
- Llevamos otro cable desde la salida de la memoria hasta el multiplexor controlado por la señal **FuentePC**, en concreto, a la entrada 3, hasta ahora sin usar.

El camino de datos modificado quedaría de la siguiente manera:



La ejecución ciclo a ciclo de la nueva instrucción sería la siguiente:

Ciclo 1 (IF)/Estado 0: Este ciclo está sin modificar. La búsqueda de la instrucción se realiza como en el resto de las instrucciones. Las señales de control son las habituales en el estado 0, salvo por el hecho de que ahora **IoD** es de 2 bits.

IR $\leftarrow$ **M**[**PC**]
PC $\leftarrow$ **PC** + 4

NOTA: Por convenio, los **estados** de la unidad de control se cuentan desde el **0** porque corresponde al valor de un registro interno de 4 bits. Sin embargo, contaremos los **ciclos** desde el **1**.

Señales de control:

$EscrPC = 1$ (al final del ciclo se actualiza el **PC**)
 $EscrPC\ Cond = 0$ (no es instrucción condicional)
 $IoD = 00$ (seleccionamos **PC** como dirección de memoria)
 $LeerMem = 1$ (se lee la memoria)
 $EscrMem = 0$ (si se lee la memoria, no se puede escribir en ella)
 $EscrIR = 1$ (se actualiza el registro **IR** con la instrucción leída)
 $FuentePC = 00$ (seleccionamos **PC** + 4 como nuevo valor de **PC**)
 $ALUOp = 00$ (sumar)
 $Se1ALUB = 01$ (seleccionamos 4 como segundo operando de la ALU)
 $Se1ALUA = 0$ (seleccionamos **PC** como primer operando de la ALU)
 $EscrReg = 0$ (protegemos el banco de registros contra escrituras indeseadas)
El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 2 (ID)/Estado 1: La decodificación de la instrucción se realiza como en el resto de las instrucciones.

$A \leftarrow Reg[IR[25-21]]$
 $B \leftarrow Reg[IR[20-16]]$
 $Salida\ ALU \leftarrow PC + extSigno16a32(IR[15-0]) \ll 2$

Los valores de los registros **A**, **B** y **Salida ALU** calculados en este ciclo son irrelevantes para esta instrucción, pero aun así se calculan por retrocompatibilidad con el resto de las instrucciones.

Señales de control:

$EscrPC = 0$ (protegemos el **PC** contra escrituras indeseadas)
 $EscrPC\ Cond = 0$ (no es instrucción condicional)
 $LeerMem = 0$ (no hay que leer la memoria)
 $EscrMem = 0$ (protegemos la memoria contra escrituras indeseadas)
 $EscrIR = 0$ (protegemos el registro **IR** contra escrituras indeseadas)
 $ALUOp = 00$ (sumar)
 $Se1ALUB = 11$ (seleccionamos el inmediato con extensión de signo y desplazamiento de 2 posiciones hacia la izquierda como segundo operando de la ALU)
 $Se1ALUA = 0$ (seleccionamos **PC** como primer operando de la ALU)
 $EscrReg = 0$ (protegemos el banco de registros contra escrituras indeseadas)
El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 3 (MEM)/Estado 10: Este tercer ciclo sería un nuevo estado. En él se realiza una lectura en memoria y la palabra leída es la dirección de la nueva instrucción que toca ejecutar. En concreto:

$PC \leftarrow M[PC[31-28] \ || \ (IR[25-0] \ll 2)]$

Señales de control:

EscrPC = 1 (al final del ciclo se actualiza el **PC**)

EscrPC Cond = 0 (no es instrucción condicional)

IoD = 10 (seleccionamos la dirección de 32 bits construida a partir de la pseudodirección de 26 bits como dirección de memoria)

LeerMem = 1 (se lee la memoria)

EscrMem = 0 (si se lee la memoria, no se puede escribir en ella)

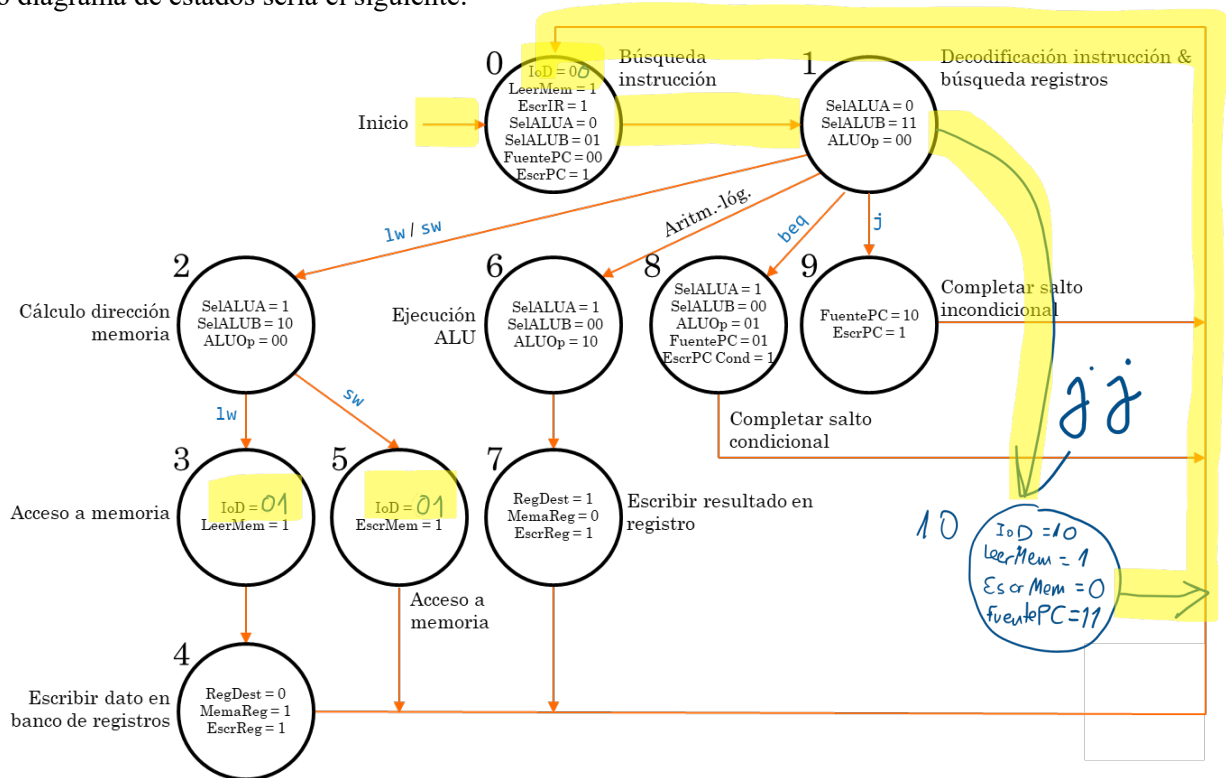
EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)

FuentePC = 11 (seleccionamos como nuevo valor de **PC** la palabra leída en memoria)

EscrReg = 0 (protegemos el banco de registros contra escrituras indeseadas)

El resto de las señales son irrelevantes para este ciclo (X).

El nuevo diagrama de estados sería el siguiente:



Hemos añadido un nuevo estado 10 después del 1. También hemos hecho que la señal **IoD** sea de 2 bits, lo que nos ha obligado a modificar también varios estados que no intervienen en esta instrucción, como el 3 y el 5.

La secuencia de estados de la nueva instrucción sería: 0 → 1 → 10 → 0.

La nueva tabla de verdad de las salidas de la unidad de control principal sería:

Estado actual	EsCrPC	EsCrPCCond	IoD1	IoD0	LeerMem	EsCrMem	EsCrIR	MemReg	FuentePC1	FuentePC0	ALUOp1	ALUOp0	Se1ALUB1	Se1ALUB0	Se1ALUA	EsCrReg	RegDest
0000	1	0	0	0	1	0	1	X	0	0	0	0	0	1	0	0	X
0001	0	0	X	X	0	0	0	X	X	X	0	0	1	1	0	0	X
0010	0	0	X	X	0	0	0	X	X	X	0	0	1	0	1	0	X
0011	0	0	0	1	1	0	0	X	X	X	X	X	X	X	X	0	X
0100	0	0	X	X	0	0	0	1	X	X	X	X	X	X	X	1	0
0101	0	0	0	1	0	1	0	X	X	X	X	X	X	X	X	0	X
0110	0	0	X	X	0	0	0	X	X	X	1	0	0	0	1	0	X
0111	0	0	X	X	0	0	0	0	X	X	X	X	X	X	X	1	1
1000	0	1	X	X	0	0	0	X	0	1	0	1	0	0	1	0	X
1001	1	0	X	X	0	0	0	X	1	0	X	X	X	X	X	0	X
1010	1	0	1	0	1	0	0	X	1	1	X	X	X	X	X	0	X

La antigua señal **IoD** ahora corresponde a **IoD0**. Hemos añadido la fila correspondiente al nuevo estado 10 (1010), así como la columna correspondiente a la nueva señal **IoD1**.

Para el nuevo estado, las señales **EsCrPC Cond**, **EsCrMem**, **EsCrIR** y **EsCrReg** se han mantenido a 0 para evitar que se modifique la memoria, el registro de instrucción o el banco de registros. La señal **EsCrPC Cond** podría ponerse como X, ya que tiene prioridad **EsCrPC** a la hora de ordenar la escritura en el registro **PC**, pero como no es una instrucción condicional, se ha optado por dejarla desactivada. El resto de las señales de control se han marcado a X al no intervenir en la instrucción y ser su valor irrelevante.

Microoperaciones realizadas por la instrucción, **desde el punto de vista de la organización multiciclo**:

```

1: IR ← M[PC]
   PC ← PC + 4
2: A ← Reg[IR[25-21]]
   B ← Reg[IR[20-16]]
   Salida ALU ← PC + extSigno16a32(IR[15-0])<<2
3: PC ← M[PC[31-28] || (IR[25-0]<<2)]

```

2. **[Ejercicio de examen]** Sea un procesador MIPS con un camino de datos multiciclo (se adjunta en una hoja aparte). Al repertorio seleccionado en clase, se desea añadir una variante de la instrucción existente de salto a registro **jr**, que llamaremos **jmr** (salto a memoria-registro). Un ejemplo de esta instrucción en ensamblador sería el siguiente:

jmr (\$s0)

La instrucción del ejemplo hace lo siguiente:

- El registro **\$s0** apunta a una palabra que se encuentra en la memoria. Se lee esta palabra en memoria.
- La palabra leída corresponde a la dirección de memoria de la siguiente instrucción del programa, es decir, se realiza un salto incondicional a dicha instrucción.

Se pide:

- a) Desde el punto de vista de la arquitectura (tal cual se vería en una tabla de resumen de instrucciones), escribir qué operación hace esta instrucción en notación de **pseudocódigo**.
- b) Justificar razonadamente qué **formato binario de instrucción** sería el más adecuado para esta instrucción y descomponerla en campos. No es necesario expresarla en hexadecimal.
- c) Indicar cuáles son todos los **operandos** de la instrucción, así como los **modos de direccionamiento** de cada uno de ellos.
- d) Explicar **razonada y detalladamente** todos los cambios que son necesarios en el camino de datos para poder ejecutar esta instrucción. Los requisitos que debe cumplir la instrucción son los siguientes:
 - El hecho de añadir esta nueva instrucción debe ser retrocompatible con todas las instrucciones estándar, que deben seguir siendo ejecutables como hasta ahora.
 - El número total de ciclos para ejecutar la nueva instrucción debe ser el mínimo posible. Para ello se podrán solapar en el mismo ciclo operaciones simultáneas que no interfieran entre sí.
 - El número total de estados de la unidad de control debe ser el mínimo posible.
 - El número total de cambios en el camino de datos debe ser el mínimo posible y también estos cambios han de ser lo más pequeños posible.
 - La prioridad de los criterios de minimización es la siguiente: número total de ciclos (más prioritario), número total de estados, número y tamaño de los cambios (menos prioritario).

Este apartado tiene tres puntos de vista interrelacionados que habrá que explicar:

- Por un lado, habrá que **explicar** las modificaciones y **dibujarlas** sobre el camino de datos, tales como añadir cables o ampliar otros existentes, añadir multiplexores o ampliar otros existentes, añadir registros o ampliar otros existentes, añadir puertas lógicas o ampliar las existentes, etc.
- Por otro, habrá que **explicar, ciclo a ciclo**, qué operaciones se realizan (desde el punto de vista de la organización, es decir, indicando los componentes concretos del camino de datos multiciclo involucrados). Para ello, se deberá usar la **notación de pseudocódigo** con el operador de asignación flecha $\leftarrow$ para indicar **todas las operaciones** que intervienen en dicho ciclo.

Además del pseudocódigo:

- Si el ciclo corresponde a un **estado nuevo** de la unidad de control, habrá que explicar e indicar el valor de **todas las señales de control relevantes**.
- Si el ciclo corresponde a un **estado existente que ahora es modificado**, habrá que explicar e indicar **solo** el valor de las señales de control **nuevas o modificadas**.
- Si el ciclo corresponde a un **estado existente que no se modifica**, **no** será necesario explicar ni indicar el valor de las señales de control.

- Finalmente, habrá que ampliar el **diagrama de estados** de la unidad de control, así como la **tabla de verdad** de las salidas de la **unidad de control** para que incluyan los cambios realizados.

Solución:

a) Esta instrucción realiza las siguientes operaciones **desde el punto de vista de la arquitectura**:

$PC \leftarrow PC + 4$
 $PC \leftarrow M[\$s0]$

Nótese que se modifica dos veces el valor del **PC**. La suma del 4 se hace en todas las instrucciones por defecto, aunque en este caso no sirva para nada.

b) En cuanto al formato de instrucción, solo necesitamos codificar 1 registro, con lo que podríamos elegir entre el I o el R, ya que en ese aspecto son equivalentes, pero, por simplicidad, elegimos el formato R.

Formato R:

← 31 – 26 →	← 25 – 21 →	← 20 – 16 →	← 15 – 11 →	← 10 – 6 →	← 5 – 0 →
Código de operación	Registro Base (\$s0)	XXXXXX	XXXXXX	XXXXXX	XXXXXXX
← 6 bits →	← 5 bits →	← 5 bits →	← 5 bits →	← 5 bits →	← 6 bits →

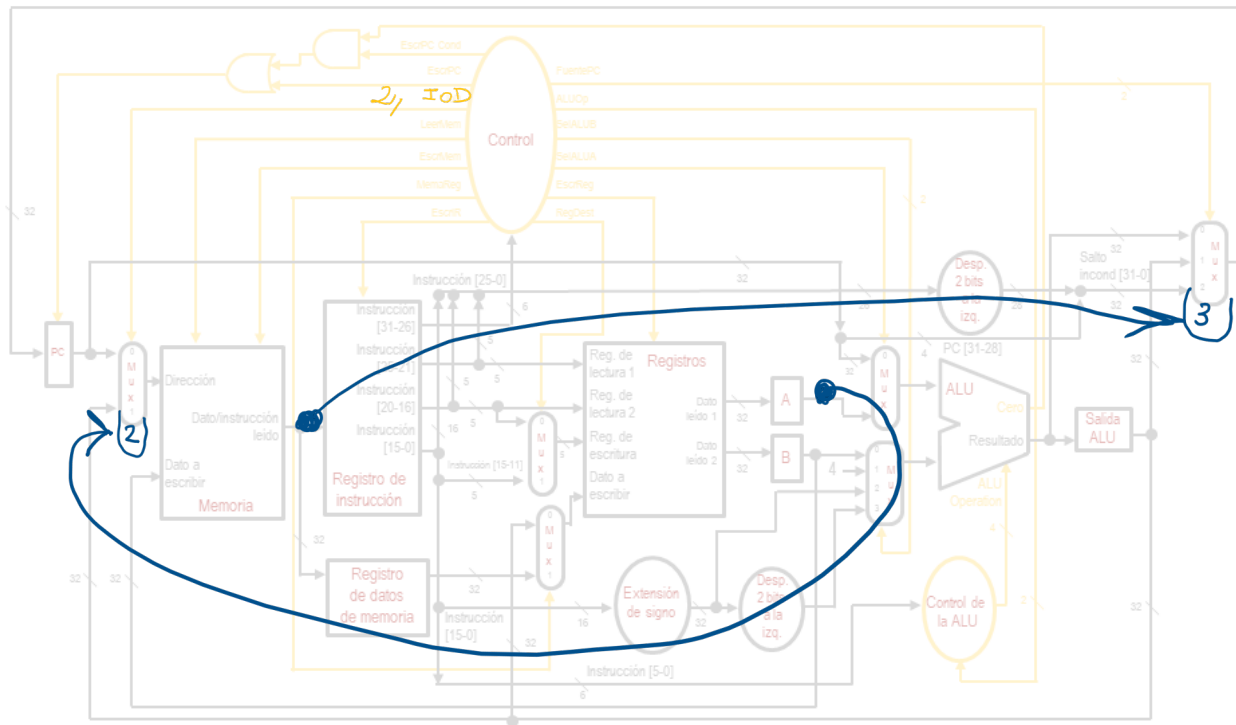
El código de operación sería único para esta instrucción, con lo que habría que encontrar una combinación libre en la hoja de resumen de instrucciones.

c) En cuanto a los operandos de la instrucción, son los siguientes:

- El valor leído en memoria, que se encuentra en la dirección a la que apunta el registro **\$s0**. Su modo de direccionamiento es indirecto a registro (sin desplazamiento), ya que su posición de memoria se especifica a través del valor de un registro. **Nótese que el registro \$s0 no es el operando, sino el medio para llegar al operando.**
- El contador de programa **PC**, que ve modificado su valor. Su modo de direccionamiento es implícito, ya que no se menciona por ninguna parte en la instrucción. El 4 que se le suma también es operando implícito, aunque normalmente no lo tenemos en cuenta.

d) Se proponen los siguientes cambios en el camino de datos:

- Conectamos con un cable la salida del registro **A** con el multiplexor controlado por la señal **IoD**, el cual pasa a tener 3 entradas. La propia señal **IoD** será ahora de 2 bits.
- Conectamos con un cable la salida de la memoria (Dato/Instrucción leído) con el multiplexor controlado por la señal **FuentePC**, el cual pasa a tener 4 entradas. La señal **FuentePC** sigue siendo de 2 bits.



La ejecución ciclo a ciclo sería la siguiente:

Ciclo 1 (IF)/Estado 0: La lectura de la instrucción se realiza como en el resto de las instrucciones. Las operaciones realizadas son:

$IR \leftarrow M[PC]$ // IR contiene la palabra de operación de la instrucción
 $PC \leftarrow PC + 4$

Las señales de control son las habituales. Debemos recordar que IoD ahora es de 2 bits y su valor es 00 .

Ciclo 2 (ID)/Estado 1: La decodificación de la instrucción se realiza como en el resto de las instrucciones. El registro A contiene el valor de $\$s0$, mientras que el valor de B es irrelevante. Las operaciones realizadas son:

$A \leftarrow \text{Reg}[IR[25-21]]$
 $B \leftarrow \text{Reg}[IR[20-16]]$
 $\text{Salida ALU} \leftarrow PC + \text{extSigno16a32}(IR[15-0]) \ll 2$

Las señales de control son las habituales.

Ciclo 3 (MEM)/Estado 10: En este ciclo, el valor de A se lleva a la entrada de dirección de la memoria y se activa la lectura. El valor leído en la memoria se escribe en el PC haciéndolo pasar por el multiplexor de FuentePC .

$PC \leftarrow M[A]$

Señales de control:

EscrPC = 1 (al final del ciclo se actualiza el PC)

EscrPC Cond = 0 (no es instrucción condicional)

IoD = 10 (seleccionamos A como dirección de memoria)

LeerMem = 1 (se lee la memoria)

EscrMem = 0 (si se lee la memoria, no se puede escribir en ella)

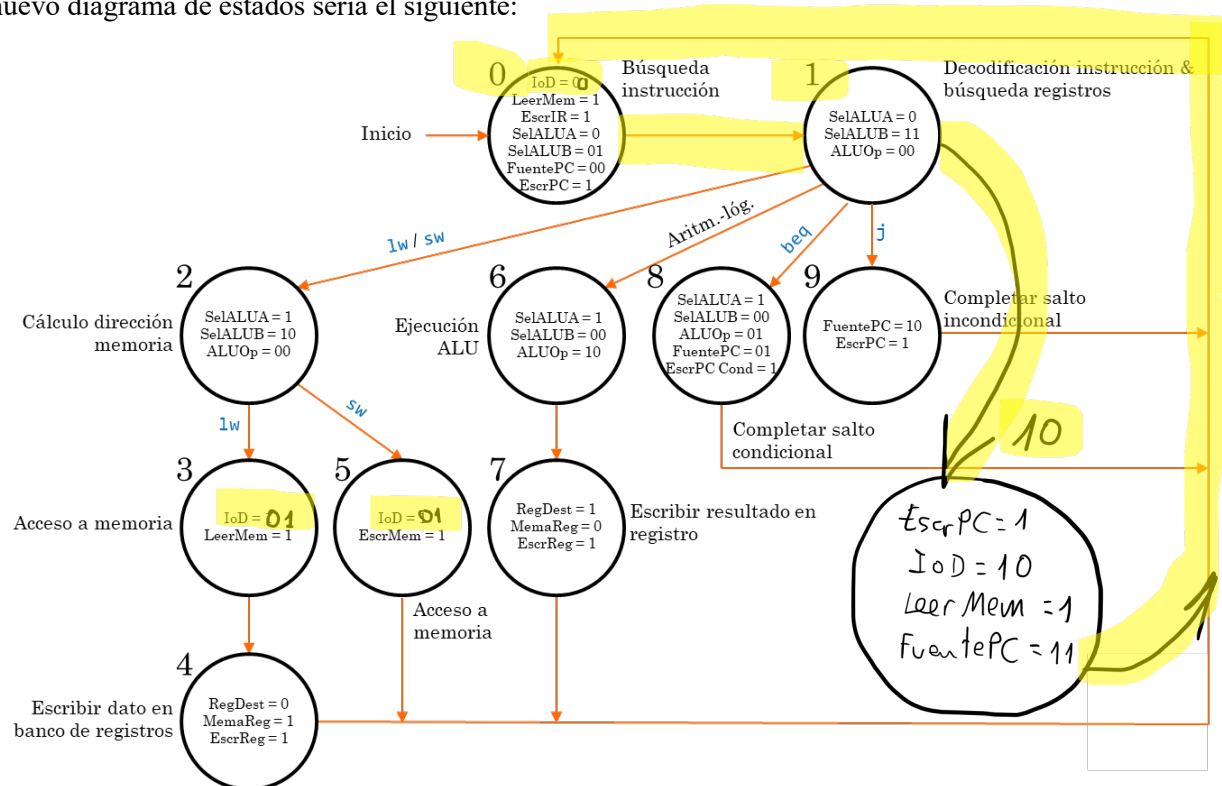
EscrIR = 0 (protegemos el registro IR contra escrituras indeseadas)

FuentePC = 11 (elegimos el valor que llega desde la memoria)

EscrReg = 0 (protegemos el banco de registros contra escrituras indeseadas)

El resto de las señales son irrelevantes para este ciclo (X).

El nuevo diagrama de estados sería el siguiente:



La secuencia de estados de la nueva instrucción sería: $0 \rightarrow 1 \rightarrow 10 \rightarrow 0$.

La nueva tabla de verdad de las salidas de la unidad de control principal sería:

Estado actual	EscrPC	EscrPCCond	Iod1	Iod0	LeerMem	EscrMem	EscrIR	MemReg	FuentePC1	FuentePC0	ALUOp1	ALUOp0	Se1ALUB1	Se1ALUB0	Se1ALUA	EscrReg	RegDest
0000	1	0	0	0	1	0	1	X	0	0	0	0	0	1	0	0	X
0001	0	0	X	X	0	0	0	X	X	X	0	0	1	1	0	0	X
0010	0	0	X	X	0	0	0	X	X	X	0	0	1	0	1	0	X
0011	0	0	0	1	1	0	0	X	X	X	X	X	X	X	X	0	X
0100	0	0	X	X	0	0	0	1	X	X	X	X	X	X	X	1	0
0101	0	0	0	1	0	1	0	X	X	X	X	X	X	X	X	0	X
0110	0	0	X	X	0	0	0	X	X	X	1	0	0	0	1	0	X
0111	0	0	X	X	0	0	0	0	X	X	X	X	X	X	X	1	1
1000	0	1	X	X	0	0	0	X	0	1	0	1	0	0	1	0	X
1001	1	0	X	X	0	0	0	X	1	0	X	X	X	X	X	0	X
1010	1	0	1	0	1	0	0	X	1	1	X	X	X	X	X	0	X

Microoperaciones realizadas por la instrucción, desde el punto de vista de la organización multiciclo:

- 1: $IR \leftarrow M[PC]$
 $PC \leftarrow PC + 4$
- 2: $A \leftarrow Reg[IR[25-21]]$
 $B \leftarrow Reg[IR[20-16]]$
 $Salida\ ALU \leftarrow PC + extSigno16a32(IR[15-0]) \ll 2$
- 3: $PC \leftarrow M[A]$

3. [Ejercicio de examen] Sea un procesador MIPS con un camino de datos multiciclo. Al repertorio seleccionado en clase, se desea añadirle la instrucción de salto a subrutina **jal**. Un ejemplo de esta instrucción en ensamblador sería el siguiente:

jal subrut

Esta instrucción hace lo siguiente: Guarda en el registro **\$ra** del banco la dirección de memoria de la instrucción que va inmediatamente después de la instrucción **jal** en el programa (es decir, el valor **PC+4**). La etiqueta **subrut** equivale a un número binario que es transformado en una dirección de 32 bits por el mismo procedimiento que la instrucción **j**. Esa dirección se guarda donde corresponda para que la siguiente instrucción en ejecutarse sea la alojada en dicha posición de memoria.

Se pide:

- a) Desde el punto de vista de la **arquitectura** (tal cual se vería en una tabla de resumen de instrucciones), escribir qué operación hace esta instrucción en notación de **pseudocódigo**.
- b) Justificar qué **formato binario** de instrucción sería el más adecuado para esta instrucción y descomponerla en campos. Habrá que indicar también **todos los operandos** de la instrucción, así como los **modos de direccionamiento** de cada uno de ellos.
- c) Si la instrucción **jal** se encuentra en la dirección de memoria **0x1001000C** y la primera instrucción de la subrutina está alojada en la dirección **0x100100A8**, calcular razonadamente el **valor binario** de la etiqueta **subrut**.

- d) Explicar razonada y detalladamente **todos los cambios** que son necesarios en el camino de datos para poder ejecutar esta instrucción. Los requisitos que debe cumplir la instrucción son los siguientes:
- El hecho de añadir esta nueva instrucción debe ser retrocompatible con todas las instrucciones estándar, que deben seguir siendo ejecutables como hasta ahora.
 - El número total de ciclos para ejecutar la nueva instrucción debe ser el mínimo posible. Para ello se podrán solapar en el mismo ciclo operaciones simultáneas que no interfieran entre sí.
 - El número total de estados de la unidad de control debe ser el mínimo posible.
 - El número total de cambios en el camino de datos debe ser el mínimo posible y también estos cambios han de ser lo más pequeños posible.

Este apartado tiene tres puntos de vista interrelacionados que habrá que explicar:

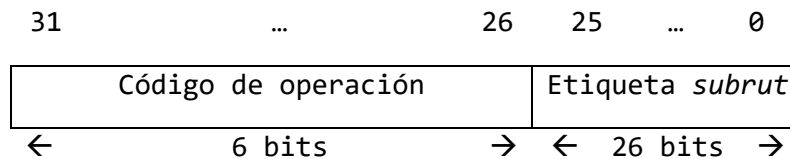
- Por un lado, habrá que **explicar** las modificaciones y **dibujarlas** sobre el camino de datos, tales como añadir cables o ampliar otros existentes, añadir multiplexores o ampliar otros existentes, añadir registros o ampliar otros existentes, añadir puertas lógicas o ampliar las existentes, etc.
- Por otro, habrá que **explicar, ciclo a ciclo**, qué operaciones se realizan (desde el punto de vista de la **organización**, es decir, indicando los componentes concretos del camino de datos involucrados). Para ello, se deberá usar la notación de **pseudocódigo** con el operador de asignación flecha $\leftarrow$ para indicar todas las operaciones que intervienen en dicho ciclo. Además del pseudocódigo:
 - Si el ciclo corresponde a un estado nuevo de la unidad de control, habrá que explicar e indicar el valor de **todas las señales de control relevantes**.
 - Si el ciclo corresponde a un estado existente que ahora es modificado, habrá que explicar e indicar **solo** el valor de las señales de control **nuevas o modificadas**.
 - Si el ciclo corresponde a un estado existente que no se modifica, **no** será necesario explicar ni indicar el valor de las señales de control.
- Finalmente, habrá que ampliar el **diagrama de estados** de la unidad de control, así como la **tabla de verdad** de las salidas de la **unidad de control** para que incluyan los cambios realizados. Ambos se adjuntan con el enunciado.

Solución:

- a) Esta instrucción realiza las siguientes operaciones **desde el punto de vista de la arquitectura**:

```
PC  $\leftarrow$  PC + 4  
$ra  $\leftarrow$  PC  
PC  $\leftarrow$  PC[31-28] || (subrut << 2)
```

- b) Se trata de una instrucción de salto incondicional y tiene un formato J, ya que solo necesita codificar el código de operación y el valor de la etiqueta *subrut* como una pseudodirección de 26 bits:



Por tanto, los operandos de la instrucción son:

- La etiqueta *subrut*, que es un valor binario de 26 bits. Permite calcular el destino del salto mediante un desplazamiento de 2 posiciones hacia la izquierda, así como la concatenación de los 4 bits superiores del registro **PC**. El modo de direccionamiento de este operando se llama pseudodirecto.
- El contador de programa **PC**, con un modo de direccionamiento implícito, que es incrementado en 4 (como en todas las instrucciones), es almacenado en el registro **\$ra** y finalmente es utilizado para el cálculo de la dirección destino del salto.
- El registro de dirección de retorno **\$ra** (registro **\$31** del banco), que es donde se guarda la dirección de la instrucción que se encuentra inmediatamente después de jal en el programa (**PC** + 4). También usa un modo de direccionamiento implícito.

c) Según el enunciado, el valor del contador de programa **PC** es 0x1001000C, ya que esa es la dirección de la instrucción **jal**. Por otro lado, la dirección del destino del salto es 0x100100A8. Para calcular el valor de la etiqueta *subrut*, debemos razonar de manera inversa:

- Dirección de destino del salto: 0x100100A8=0001 0000 0000 0001 0000 0000 1010 1000.
- Los 4 bits superiores de esta dirección (0001) coinciden con los 4 bits del **PC**, lo cual es lo esperable.
- Los 28 bits inferiores (0000 0000 0001 0000 0000 1010 1000) son el resultado de aplicar el desplazamiento de 2 bits hacia la izquierda a la etiqueta *subrut*.
- Para deshacerlo, aplicamos el mismo desplazamiento, pero a la derecha, con lo que obtenemos el valor de *subrut*: 00 0000 0000 0100 0000 0010 1010=0x000402A.

d) Los cambios sugeridos para el camino de datos son los siguientes:

- Ampliamos el multiplexor controlado por la señal **MemaReg**, que es el que establece el dato que se va a escribir en el banco. Su tamaño pasa de “2 a 1” a “4 a 1”, de forma que ahora la entrada 2 procede del registro **PC** (la entrada 3 no se usa). La señal de selección **MemaReg** pasa a ser de 2 bits.
- Por su parte, ampliamos el multiplexor controlado por la señal **RegDest**, que establece en qué registro del banco se produce una escritura. La entrada 2 recibe el valor constante 31, que está cableado en el hardware. Este valor corresponde al número del registro **\$ra** en el banco. La entrada 3 del multiplexor no se usa. A su vez, la señal de selección **RegDest** pasa a ser de 2 bits.

El camino de datos modificado quedaría de la siguiente manera:

y, a su vez, se actualiza **PC** para que contenga la dirección de destino del salto. Con este ciclo, se terminaría la ejecución de la instrucción.

$\$ra \leftarrow PC$
 $PC \leftarrow PC[31-28] \parallel (Instrucción[25-0] \ll 2)$

Las señales de control necesarias son las siguientes:

EscrPC = 1 (se actualiza el **PC** con la dirección de destino del salto)

EscrPC Cond = 0 (no es instrucción condicional)

LeerMem = 0 (no hay que leer la memoria)

EscrMem = 0 (protegemos la memoria contra escrituras indeseadas)

EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)

MemaReg = 10 (elegimos **PC** para escribirlo en el banco de registros)

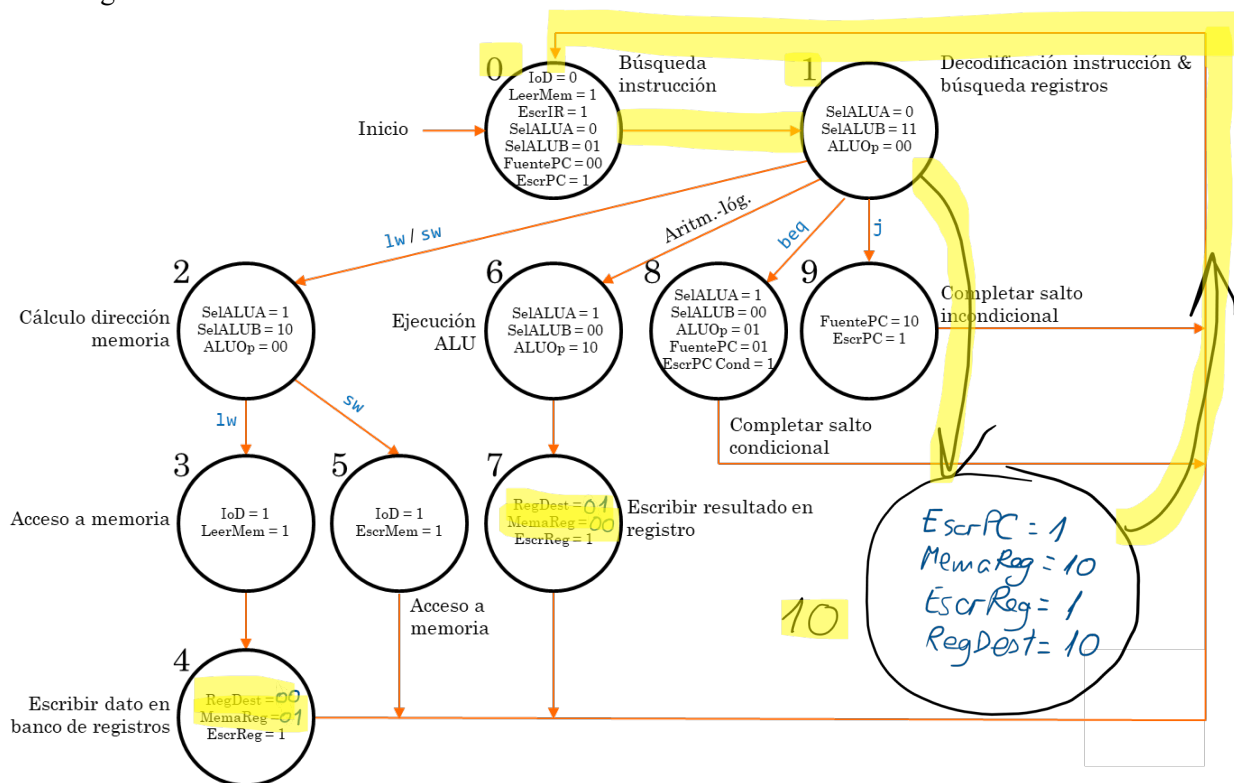
FuentePC = 10 (elegimos **PC[31-28] || (Instrucción[25-0] << 2)** para escribirlo en **PC**)

EscrReg = 1 (escribimos en el banco de registros)

RegDest = 10 (elegimos el registro 31, es decir, **\$ra**)

El resto de las señales son irrelevantes para este ciclo (X).

El nuevo diagrama de estados es:



La secuencia de estados de la nueva instrucción sería: $0 \rightarrow 1 \rightarrow 10 \rightarrow 0$.

La nueva tabla de verdad de las salidas de la unidad de control principal sería:

Estado actual	EscrPC	EscrPCCond	Iod	LeerMem	EscrMem	EscrIR	MemRegI	MemRegO	FuentePC1	FuentePC0	ALUop1	ALUop0	Se1ALUB1	Se1ALUB0	Se1ALUA0	EscrReg	RegDest1	RegDest0
0000	1	0	0	1	0	1	X	X	0	0	0	0	0	1	0	0	X	X
0001	0	0	X	0	0	0	X	X	X	X	0	0	1	1	0	0	X	X
0010	0	0	X	0	0	0	X	X	X	X	0	0	1	0	1	0	X	X
0011	0	0	1	1	0	0	X	X	X	X	X	X	X	X	X	0	X	X
0100	0	0	X	0	0	0	0	1	X	X	X	X	X	X	X	1	0	0
0101	0	0	1	0	1	0	X	X	X	X	X	X	X	X	X	0	X	X
0110	0	0	X	0	0	0	X	X	X	X	1	0	0	0	1	0	X	X
0111	0	0	X	0	0	0	0	0	X	X	X	X	X	X	X	1	0	1
1000	0	1	X	0	0	0	X	X	0	1	0	1	0	0	1	0	X	X
1001	1	0	X	0	0	0	X	X	1	0	X	X	X	X	X	0	X	X
1010	1	0	X	0	0	0	1	0	1	0	X	X	X	X	X	1	1	0

Microoperaciones realizadas por la instrucción, desde el punto de vista de la organización multiciclo:

- 1: $IR \leftarrow M[PC]$
 $PC \leftarrow PC + 4$
- 2: $A \leftarrow Reg[IR[25-21]]$
 $B \leftarrow Reg[IR[20-16]]$
 $Salida\ ALU \leftarrow PC + extSigno16a32(IR[15-0]) \ll 2$
- 3: $\$ra \leftarrow PC$
 $PC \leftarrow PC[31-28] \ || \ (Instrucción[25-0] \ll 2)$

4. [Ejercicio de examen] Sea un procesador MIPS con un camino de datos multiciclo. Se desea añadir una nueva instrucción al repertorio, denominada **jadd**, que realiza un salto incondicional a una determinada instrucción. Un ejemplo de esta instrucción en ensamblador sería el siguiente:

jadd \$s0, \$s4, 12

Esta instrucción hace lo siguiente: Suma los registros **\$s0** y **\$s4** y el valor numérico **12**. El resultado de la suma es la dirección de memoria que corresponde al destino del salto incondicional.

Se pide:

- a) Desde el punto de vista de la **arquitectura** (tal cual se vería en una tabla de resumen de instrucciones), escribir qué operación hace esta instrucción en notación de **pseudocódigo**.
- b) Justificar qué **formato binario** de instrucción sería el más adecuado para esta instrucción y descomponerla en campos. Habrá que indicar también **todos los operandos** de la instrucción, así como los **modos de direccionamiento** de cada uno de ellos.
- c) Escribir la instrucción que se pone en el ejemplo en **formato hexadecimal**.
- d) Explicar razonada y detalladamente **todos los cambios** que son necesarios en el camino de datos para poder ejecutar esta instrucción. Los requisitos que debe cumplir la instrucción son los siguientes:

- El hecho de añadir esta nueva instrucción debe ser retrocompatible con todas las instrucciones estándar, que deben seguir siendo ejecutables como hasta ahora.
- El número total de ciclos para ejecutar la nueva instrucción debe ser el mínimo posible. Para ello se podrán solapar en el mismo ciclo operaciones simultáneas que no interfieran entre sí.
- El número total de estados de la unidad de control debe ser el mínimo posible.
- El número total de cambios en el camino de datos debe ser el mínimo posible y también estos cambios han de ser lo más pequeños posible.

Este apartado tiene tres puntos de vista interrelacionados que habrá que explicar:

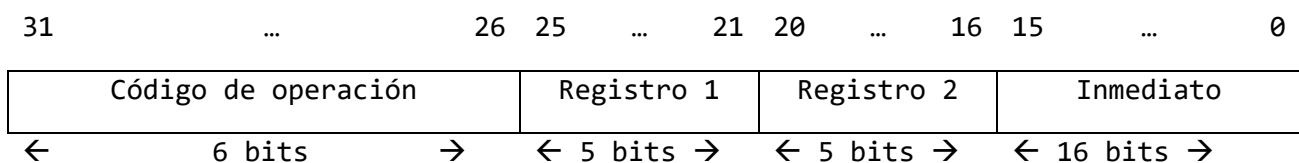
- Por un lado, habrá que **explicar** las modificaciones y **dibujarlas** sobre el camino de datos, tales como añadir cables o ampliar otros existentes, añadir multiplexores o ampliar otros existentes, añadir registros o ampliar otros existentes, añadir puertas lógicas o ampliar las existentes, etc.
- Por otro, habrá que **explicar, ciclo a ciclo**, qué operaciones se realizan (desde el punto de vista de la **organización**, es decir, indicando los componentes concretos del camino de datos involucrados). Para ello, se deberá usar la notación de **pseudocódigo** con el operador de asignación flecha $\leftarrow$ para indicar todas las operaciones que intervienen en dicho ciclo. Además del pseudocódigo:
 - Si el ciclo corresponde a un estado nuevo de la unidad de control, habrá que explicar e indicar el valor de **todas las señales de control relevantes**.
 - Si el ciclo corresponde a un estado existente que ahora es modificado, habrá que explicar e indicar **solo** el valor de las señales de control **nuevas o modificadas**.
 - Si el ciclo corresponde a un estado existente que no se modifica, **no** será necesario explicar ni indicar el valor de las señales de control.
- Finalmente, habrá que ampliar el **diagrama de estados** de la unidad de control, así como la **tabla de verdad** de las salidas de la **unidad de control** para que incluyan los cambios realizados. Ambos se adjuntan con el enunciado.

Solución:

a) Esta instrucción realiza las siguientes operaciones **desde el punto de vista de la arquitectura**:

```
PC  $\leftarrow$  PC + 4 (este valor se pierde)
PC  $\leftarrow$  $s0 + $s4 + extSigno16a32(12)
```

b) El formato binario más adecuado para la instrucción es el I, ya que tenemos que codificar en la instrucción 2 registros y 1 inmediato. De esta forma, la descomposición en campos sería:



Por tanto, los operandos son:

- Registro 1 (**\$s0**), indicado con 5 bits en la instrucción (direccionamiento directo a registro).
- Registro 2 (**\$s4**), indicado con 5 bits en la instrucción (direccionamiento directo a registro).
- Inmediato (12), codificado con 16 bits en complemento a 2 (direccionamiento inmediato).
- Contador de programa **PC**, no mencionado en ninguna parte (direccionamiento implícito).

c) Para pasar esta instrucción a hexadecimal, debemos elegir un código de operación no utilizado por ninguna otra instrucción. Por ejemplo, escogemos el valor **010000**. También hay que recordar que el registro **\$s0** es el 16 (**10000**), mientras que el **\$s4** es el 20 (**10100**). Por último, el inmediato 12 en complemento a 2 de 16 bits es **0000000000001100**. Por tanto:

Código de operación	Registro 1	Registro 2	Inmediato
010000	10000	10100	0000000000001100

Agrupando los bits de 4 en 4, puede verse fácilmente la expresión en hexadecimal de la instrucción máquina:

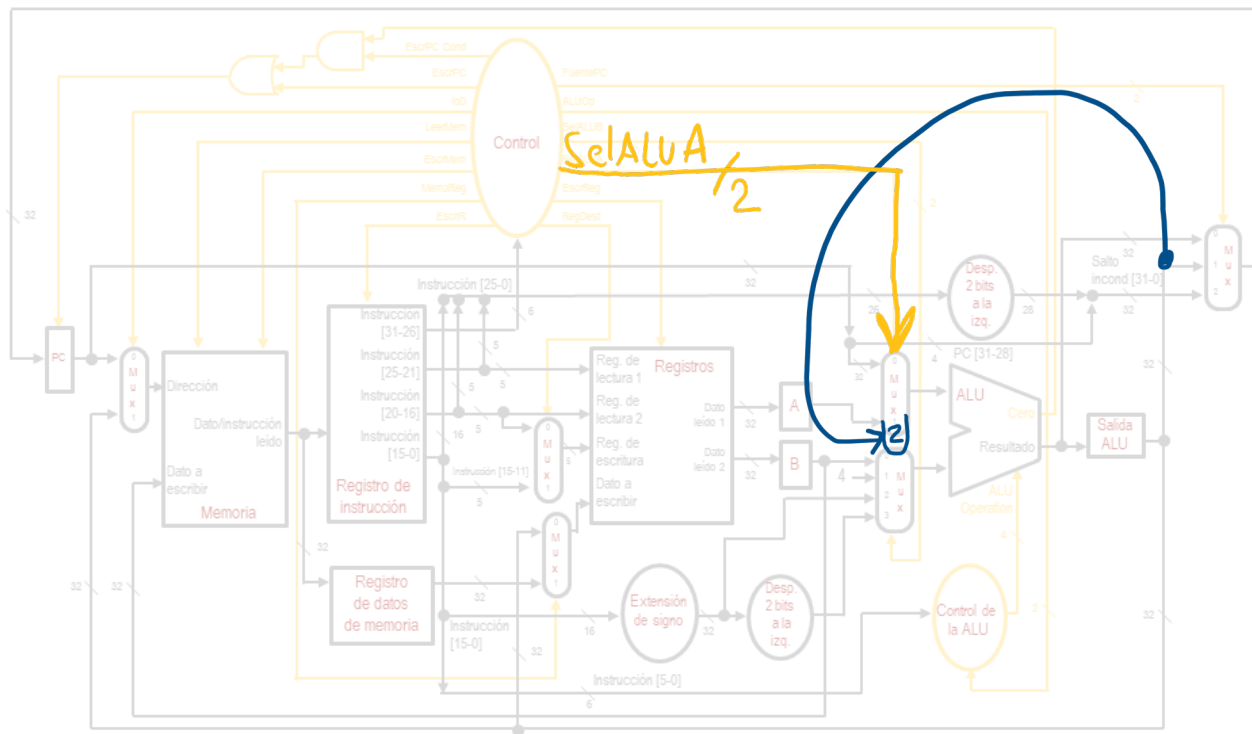
010000 10000 10100 0000000000001100 = 0100 0010 0001 0100 0000 0000 0000 1100 =
0x4214000C

El resultado concreto depende de la elección del código de operación.

d) Se proponen el siguiente cambio en el camino de datos:

- Conectar el registro **Salida ALU** con el multiplexor del primer operando de la ALU, que pasará a ser ahora de 4 entradas (una sin usar). El nuevo cable se conecta en la entrada 2.
- Asimismo, la señal de selección **Se1ALUA** pasa a ser de 2 bits.

El camino de datos modificado quedaría de la siguiente manera:



La ejecución ciclo a ciclo sería la siguiente:

Ciclo 1 (IF)/Estado 0: La lectura de la instrucción se realiza como en el resto de las instrucciones. Las operaciones realizadas son:

$$\begin{aligned} IR &\leftarrow M[PC] \\ PC &\leftarrow PC + 4 \end{aligned}$$

Las señales de control son las habituales para este estado.

Ciclo 2 (ID)/Estado 1: La decodificación de la instrucción se realiza como en el resto de las instrucciones. Las operaciones realizadas son:

$$\begin{aligned} A &\leftarrow \text{Reg}[IR[25-21]] \\ B &\leftarrow \text{Reg}[IR[20-16]] \\ \text{Salida ALU} &\leftarrow PC + \text{extSigno16a32}(IR[15-0]) \ll 2 \end{aligned}$$

El valor del registro **Salida ALU** calculado en este ciclo es irrelevante para esta instrucción, pero aun así se calculan por retrocompatibilidad con el resto de las instrucciones. Los registros **A** y **B** acaban conteniendo los registros **\$s0** y **\$s4**.

Las señales de control son las habituales para este estado.

Ciclo 3 (EX)/Estado 2: Este ciclo va a ser el mismo que siguen las instrucciones **lw** y **sw** cuando suman un registro y un inmediato. En este caso, el registro **A** contiene **\$s0** y los bits 15-0 de **IR** contienen el inmediato 12. Las operaciones realizadas son:

$$\text{Salida ALU} \leftarrow A + \text{extSigno16a32}(\text{IR}[15-0])$$

Las señales de control son las habituales para este estado.

Ciclo 4 (EX)/Estado 10: Este estado es nuevo y en él realizamos la suma que nos falta entre el registro **B** (que vale **\$s4**) y el valor parcial almacenado en **Salida ALU**. Las operaciones realizadas son:

$$\text{PC} \leftarrow \text{Salida ALU} + B$$

Señales de control:

EscrPC = 1 (modificamos el **PC** para realizar el salto)

EscrPC Cond = 0 (no es instrucción condicional)

LeerMem = 0 (no hay que leer la memoria)

EscrMem = 0 (protegemos la memoria contra escrituras indeseadas)

EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)

FuentePC = 00 (seleccionamos el resultado de la ALU)

ALUOp = 00 (sumar)

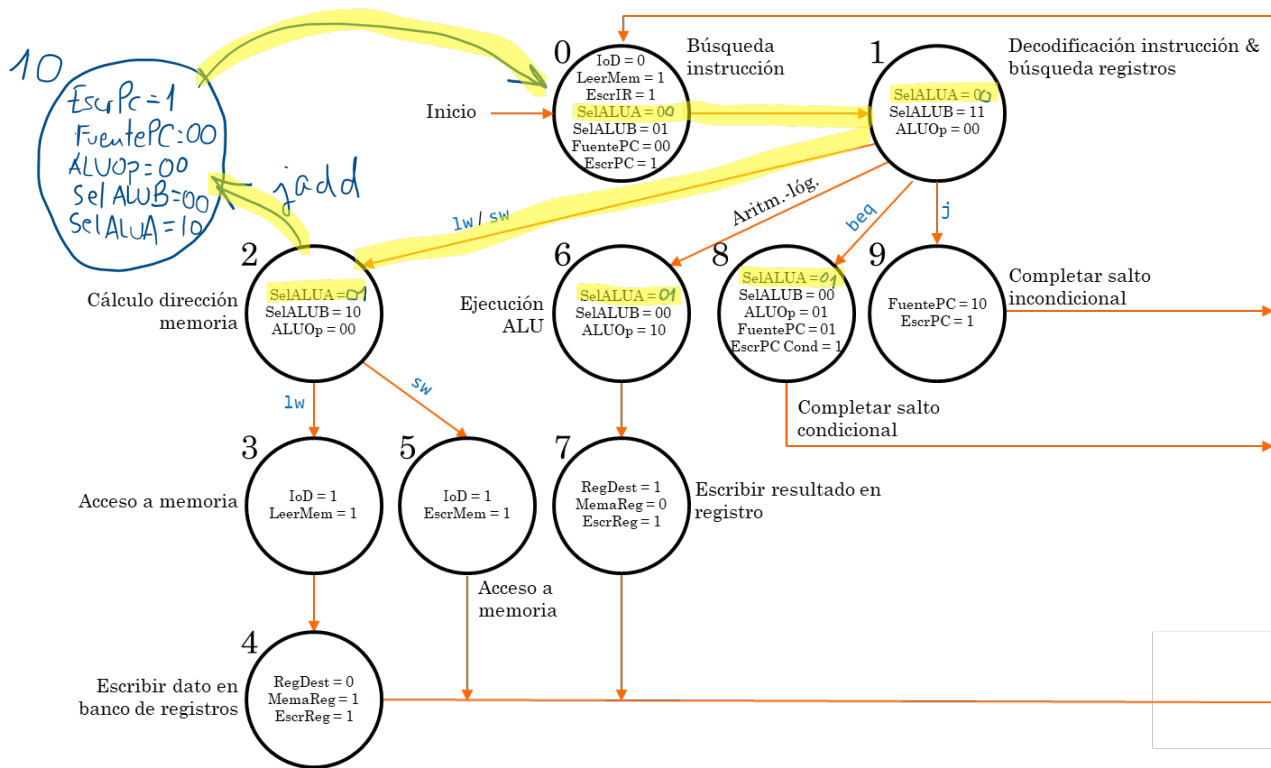
Se1ALUB = 00 (seleccionamos **B** como segundo operando de la ALU)

Se1ALUA = 10 (seleccionamos **Salida ALU** como primer operando de la ALU) → Señal de 2 bits

EscrReg = 0 (protegemos el banco de registros contra escrituras indeseadas)

El resto de las señales son irrelevantes para este ciclo (X).

El nuevo diagrama de estados sería:



La secuencia de estados de la nueva instrucción sería: $0 \rightarrow 1 \rightarrow 2 \rightarrow 10 \rightarrow 0$.

La nueva tabla de verdad de las salidas de la unidad de control principal sería:

Estado actual	EsCrPC	EsCrPCCond	IoD	LeerMem	EsCrMem	EsCrIR	MemaReg	FuentePC1	FuentePC0	ALUOp1	ALUOp0	SelALUB1	SelALUB0	SelALUA1	SelALUA0	EsCrReg	RegDest
0000	1	0	0	1	0	1	X	0	0	0	0	0	1	0	0	0	X
0001	0	0	X	0	0	0	X	X	X	0	0	1	1	0	0	0	X
0010	0	0	X	0	0	0	X	X	X	0	0	1	0	0	1	0	X
0011	0	0	1	1	0	0	X	X	X	X	X	X	X	X	X	0	X
0100	0	0	X	0	0	0	1	X	X	X	X	X	X	X	X	1	0
0101	0	0	1	0	1	0	X	X	X	X	X	X	X	X	X	0	X
0110	0	0	X	0	0	0	X	X	X	1	0	0	0	0	1	0	X
0111	0	0	X	0	0	0	0	X	X	X	X	X	X	X	X	1	1
1000	0	1	X	0	0	0	X	0	1	0	1	0	0	0	1	0	X
1001	1	0	X	0	0	0	X	1	0	X	X	X	X	X	X	0	X
1010	1	0	X	0	0	0	X	0	0	0	0	0	0	1	0	0	X

Microoperaciones realizadas por la instrucción, desde el punto de vista de la organización multiciclo:

- 1: $IR \leftarrow M[PC]$
 $PC \leftarrow PC + 4$
- 2: $A \leftarrow Reg[IR[25-21]]$
 $B \leftarrow Reg[IR[20-16]]$
 $Salida\ ALU \leftarrow PC + extSigno16a32(IR[15-0]) \ll 2$
- 3: $Salida\ ALU \leftarrow A + extSigno16a32(IR[15-0])$
- 4: $PC \leftarrow Salida\ ALU + B$

5. [Ejercicio de examen] Sea un procesador MIPS con un camino de datos multiciclo (se adjunta con el enunciado). Al repertorio seleccionado en clase, se desea añadirle una instrucción inventada llamada **beq32**. Un ejemplo de esta instrucción en ensamblador sería el siguiente:

beq32 **\$s0**, **\$s1**, *dirección32*

Esta instrucción hace lo siguiente:

- Lee los dos registros indicados, en este caso **\$s0** y **\$s1**, y los compara para ver si son iguales.
- Si son iguales, se realiza un salto a la instrucción ubicada en *dirección32*. Nótese que se trata de una dirección de memoria de 32 bits.
- Si no lo son, la ejecución continúa con normalidad con la instrucción ubicada a continuación de esta en la memoria.

Para poder almacenar toda la información relevante a esta instrucción, de manera excepcional se permite que ocupe **dos palabras** en vez de una:

- La primera palabra (ubicada en la dirección a la que apunta el **PC**) contiene el código de operación y los registros.
- La segunda palabra (ubicada a continuación) contiene *dirección32*.

Se pide:

- a) Desde el punto de vista de la **arquitectura** (tal cual se vería en una tabla de resumen de instrucciones), escribir qué operación hace esta instrucción en notación de **pseudocódigo**.
- b) Justificar qué **formato binario** de instrucción sería el más adecuado para esta instrucción y descomponerla en campos.
- c) Indicar cuáles son **todos los operandos** de la instrucción, así como los **modos de direccionamiento** de cada uno de ellos.
- d) Explicar razonada y detalladamente **todos los cambios** que son necesarios en el camino de datos para poder ejecutar esta instrucción. Los requisitos que debe cumplir la instrucción son los siguientes:
 - El hecho de añadir esta nueva instrucción debe ser retrocompatible con todas las instrucciones estándar, que deben seguir siendo ejecutables como hasta ahora.
 - El número total de ciclos para ejecutar la nueva instrucción debe ser el mínimo posible. Para ello se podrán solapar en el mismo ciclo operaciones simultáneas que no interfieran entre sí.
 - El número total de estados de la unidad de control debe ser el mínimo posible.
 - El número total de cambios en el camino de datos debe ser el mínimo posible y también estos cambios han de ser lo más pequeños posible.

Este apartado tiene tres puntos de vista interrelacionados que habrá que explicar:

- Por un lado, habrá que **explicar** las modificaciones y **dibujarlas** sobre el camino de datos, tales como añadir cables o ampliar otros existentes, añadir multiplexores o ampliar otros existentes, añadir registros o ampliar otros existentes, añadir puertas lógicas o ampliar las existentes, etc.
- Por otro, habrá que **explicar, ciclo a ciclo**, qué operaciones se realizan (desde el punto de vista de la **organización**, es decir, indicando los componentes concretos del camino de datos involucrados). Para ello, se deberá usar la notación de **pseudocódigo** con el operador de asignación flecha $\leftarrow$ para indicar todas las operaciones que intervienen en dicho ciclo. Además del pseudocódigo:
 - Si el ciclo corresponde a un estado nuevo de la unidad de control, habrá que explicar e indicar el valor de **todas las señales de control relevantes**.
 - Si el ciclo corresponde a un estado existente que ahora es modificado, habrá que explicar e indicar **solo** el valor de las señales de control **nuevas o modificadas**.
 - Si el ciclo corresponde a un estado existente que no se modifica, **no** será necesario explicar ni indicar el valor de las señales de control.
- Finalmente, habrá que ampliar el **diagrama de estados** de la unidad de control, así como la **tabla de verdad** de las salidas de la **unidad de control** para que incluyan los cambios realizados. Ambos se adjuntan con el enunciado.

Solución:

a) Esta instrucción realiza las siguientes operaciones **desde el punto de vista de la arquitectura**:

```
PC  $\leftarrow$  PC + 4
PC  $\leftarrow$  PC + 4
Si ($s0 = $s1) entonces PC  $\leftarrow$  dirección32
```

Otra manera equivalente de expresar esto mismo sería:

```
Si ($s0 = $s1) entonces PC  $\leftarrow$  dirección32
Si no entonces PC  $\leftarrow$  PC + 8
```

b) Esta instrucción es excepcional en MIPS porque ocupa 2 palabras (8 bytes) en la memoria:

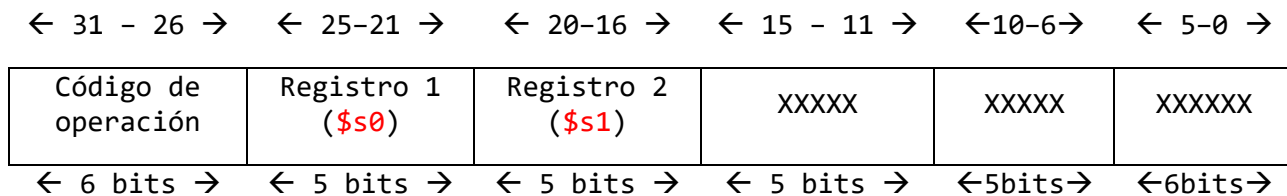
- La dirección a la que apunta PC contiene la primera palabra, que es la que tiene el código de operación. Se llama “palabra de operación”.
- La dirección a la que apunta PC + 4 contiene la segunda palabra, que es la que contiene el valor *dirección32*. Se llama “palabra de extensión”.
- La instrucción siguiente se encuentra ubicada en la dirección PC + 8.

La instrucción propuesta lee dos registros en el banco. Si son iguales, realiza un salto a la instrucción ubicada en *dirección32*, valor que habrá que colocar en el PC. En caso de no cumplirse la condición, hay que incrementar el valor del PC en 8 unidades.

$$\leftarrow 31 - 26 \rightarrow \quad \leftarrow 25 - 21 \rightarrow \quad \leftarrow 20 - 16 \rightarrow \quad \leftarrow 15 - 0 \rightarrow$$

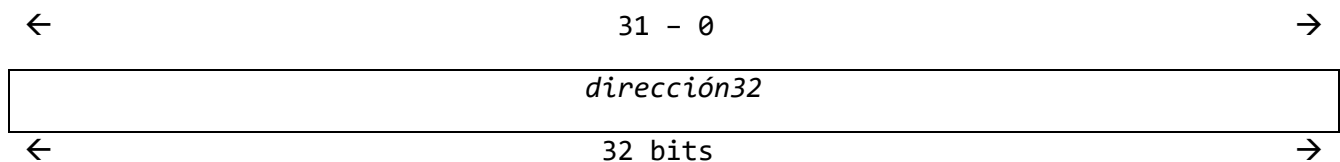
En este formato, en el campo de 16 bits dedicado al inmediato habría codificado cualquier valor, ya que este campo sería ignorado y nos da igual.

Versión 2: También podríamos usar una palabra con formato R:



Con el formato R, los campos dedicados a un tercer registro, a la cantidad de desplazamiento y a la función también tendrían cualquier valor porque serían ignorados.

En cuanto a la **segunda palabra** de la instrucción (la de **extensión**), no corresponde a ningún formato habitual en MIPS (R, I o J). Simplemente contiene la dirección de destino del salto expresada con 32 bits:

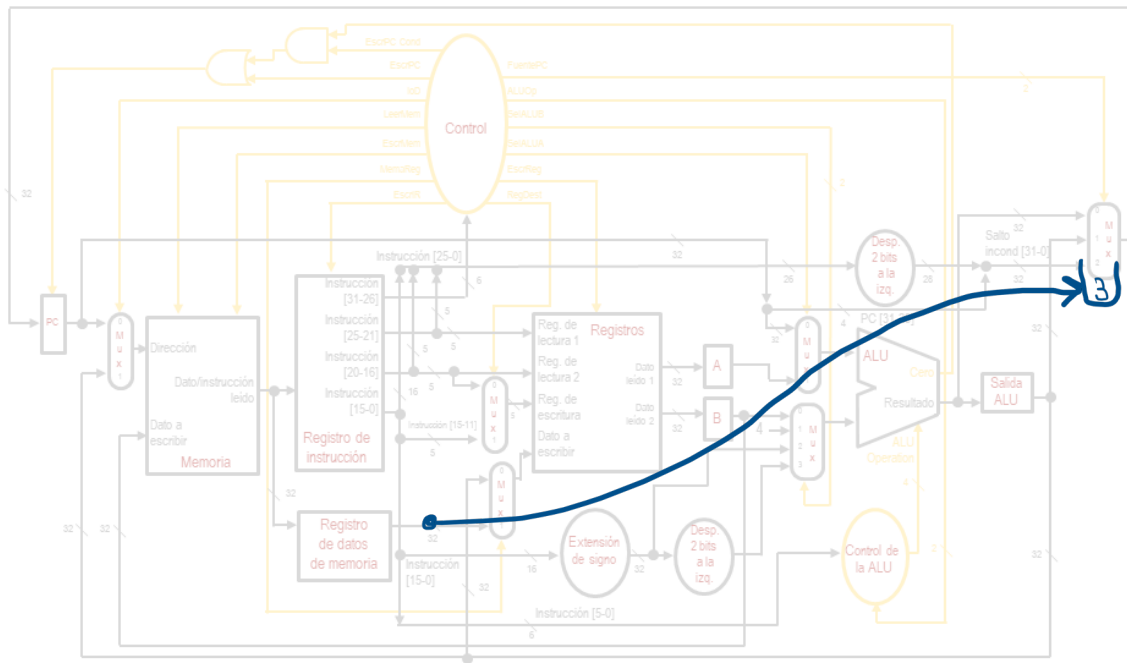


c) En cuanto a los operandos de esta instrucción, son los siguientes:

- Los registros **\$s0** y **\$s1**, cuyos valores han de leerse en el banco para después compararlos entre sí. Por tanto, tienen un direccionamiento directo a registro porque se codifican cada uno con 5 bits en la instrucción y los operandos son precisamente los valores de los registros.
- La *dirección32* es un valor con direccionamiento inmediato porque está codificado en la propia instrucción. De manera excepcional, tiene 32 bits, en vez de los 16 bits habituales.
- El contador de programa **PC**, que recibe el valor *dirección32*. Como no aparece mencionado en la instrucción, tiene un direccionamiento implícito.

d) Los cambios que se proponen en el camino de datos son los siguientes:

- Ampliar el multiplexor controlado por la señal **FuentePC**.
- Añadir un cable desde la salida del registro **MDR** y llevarlo precisamente hasta la entrada 3 del multiplexor controlado por la señal **FuentePC**.



La ejecución ciclo a ciclo sería la siguiente:

Ciclo 1 (IF)/Estado 0: La lectura de la instrucción se realiza como en el resto de las instrucciones. Las operaciones realizadas son:

$IR \leftarrow M[PC]$ // IR contiene la palabra de operación de la instrucción
 $PC \leftarrow PC + 4$

Las señales de control son las habituales para este estado.

Ciclo 2 (ID)/Estado 1: La decodificación de la instrucción se realiza como en el resto de las instrucciones. Las operaciones realizadas son:

$A \leftarrow \text{Reg}[IR[25-21]]$
 $B \leftarrow \text{Reg}[IR[20-16]]$
 $\text{Salida ALU} \leftarrow PC + \text{extSigno16a32}(IR[15-0]) \ll 2$

El valor de **Salida ALU** es irrelevante para esta instrucción, pero aun así se calcula por retrocompatibilidad con el resto de las instrucciones.

Las señales de control son las habituales para este estado.

Ciclo 3 (IF)/Estado 10: En este nuevo estado debemos leer la palabra de extensión, que contiene el valor *dirección32*. Como no podemos guardarla en el registro **IR**, lo hacemos en **MDR**. Mientras tanto, incrementamos el valor del **PC** en otras 4 unidades. Las operaciones realizadas son:

```
MDR ← M[PC]    // MDR contiene la palabra de extensión de la
                // instrucción (dirección32)
PC ← PC + 4
```

Las señales de control necesarias son las siguientes:

```
EscrPC = 1 (al final del ciclo se actualiza el PC)
EscrPC Cond = 0 (no es instrucción condicional)
IoD = 0 (seleccionamos PC como dirección de memoria)
LeerMem = 1 (se lee la memoria)
EscrMem = 0 (si se lee la memoria, no se puede escribir en ella)
EscrIR = 0 (protegemos el registro IR contra escrituras indeseadas)
FuentePC = 00 (seleccionamos PC + 4 como nuevo valor de PC)
ALUOp = 00 (sumar)
SelALUB = 01 (seleccionamos 4 como segundo operando de la ALU)
SelALUA = 0 (seleccionamos PC como primer operando de la ALU)
EscrReg = 0 (protegemos el banco de registros contra escrituras indeseadas)
```

El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 4/Estado 11 (EX): En este ciclo realizamos en la ALU la resta de **A** y **B**, que contienen los registros **\$s0** y **\$s1**. Si la resta es 0, se activará la salida **Cero** de la ALU. Las operaciones realizadas son:

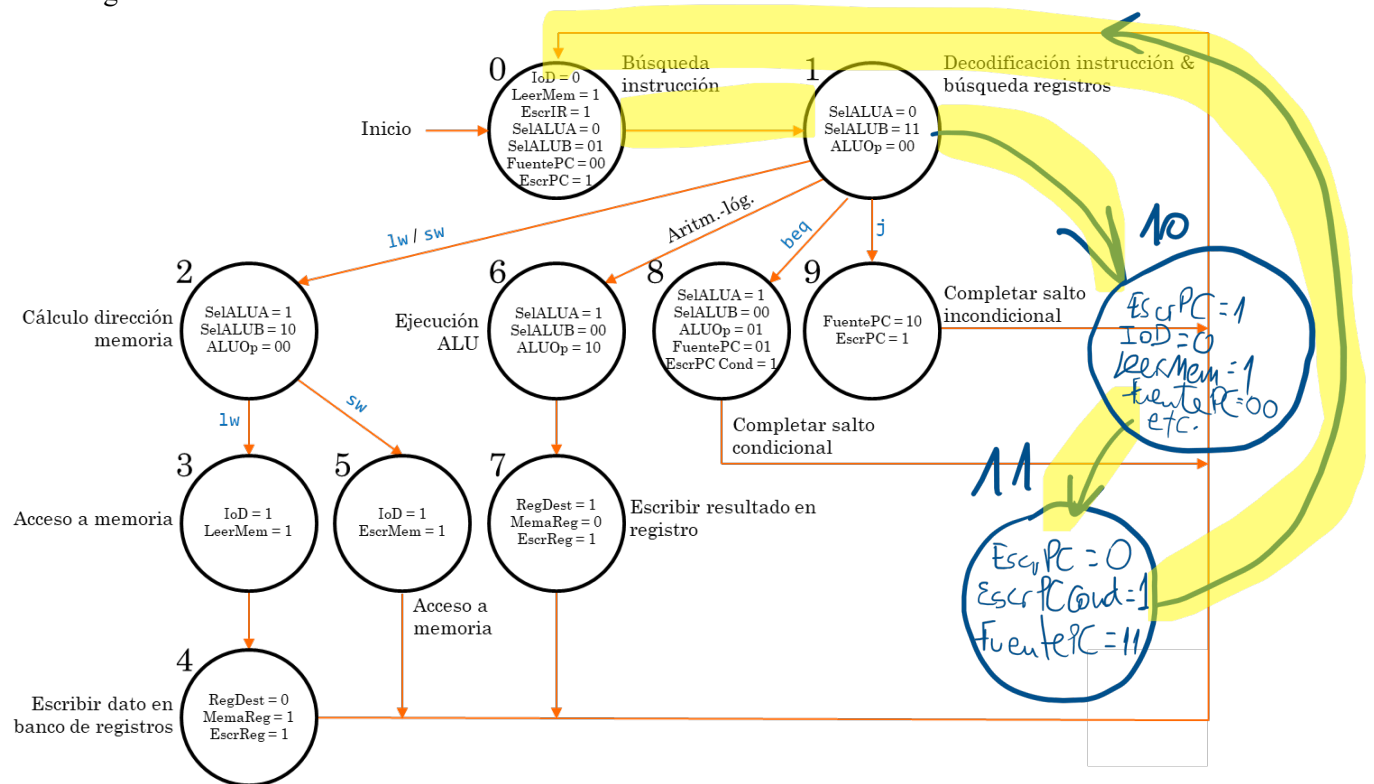
```
Si (A = B) entonces PC ← MDR
```

Las señales de control necesarias son las siguientes:

```
EscrPC = 0 (protegemos el registro PC contra escrituras indeseadas)
EscrPC Cond = 1 (instrucción condicional)
LeerMem = 0 (no se lee la memoria)
EscrMem = 0 (protegemos la memoria contra escrituras indeseadas)
EscrIR = 0 (protegemos el registro IR contra escrituras indeseadas)
FuentePC = 11 (seleccionamos MDR como nuevo valor de PC)
ALUOp = 01 (restar)
SelALUB = 00 (seleccionamos B como segundo operando de la ALU)
SelALUA = 1 (seleccionamos A como primer operando de la ALU)
EscrReg = 0 (protegemos el banco de registros contra escrituras indeseadas)
```

El resto de las señales son irrelevantes para este ciclo (X).

El nuevo diagrama de estados es:



La secuencia de estados de la nueva instrucción sería: $0 \rightarrow 1 \rightarrow 10 \rightarrow 11 \rightarrow 0$.

La nueva tabla de verdad de las salidas de la unidad de control principal sería:

Estado actual	EscrPC	EscrPCCond	IoD	LeerMem	EscrMem	EscrIR	MemaReg	FuentePC1	FuentePC0	ALUOp1	ALUOp0	SelALUB1	SelALUB0	SelALUA0	EscrReg	RegDest
0000	1	0	0	1	0	1	X	0	0	0	0	0	1	0	0	X
0001	0	0	X	0	0	0	X	X	X	0	0	1	1	0	0	X
0010	0	0	X	0	0	0	X	X	X	0	0	1	0	1	0	X
0011	0	0	1	1	0	0	X	X	X	X	X	X	X	X	0	X
0100	0	0	X	0	0	0	1	X	X	X	X	X	X	X	1	0
0101	0	0	1	0	1	0	X	X	X	X	X	X	X	X	0	X
0110	0	0	X	0	0	0	X	X	X	1	0	0	0	1	0	X
0111	0	0	X	0	0	0	0	X	X	X	X	X	X	X	1	1
1000	0	1	X	0	0	0	X	0	1	0	1	0	0	1	0	X
1001	1	0	X	0	0	0	X	1	0	X	X	X	X	X	0	X
1010	1	0	0	1	0	0	X	0	0	0	0	0	1	0	0	X
1011	0	1	X	0	0	0	X	1	1	0	1	0	0	1	0	X

Microoperaciones realizadas por la instrucción, desde el punto de vista de la organización multiciclo:

- 1: $IR \leftarrow M[PC]$
 $PC \leftarrow PC + 4$
- 2: $A \leftarrow Reg[IR[25-21]]$
 $B \leftarrow Reg[IR[20-16]]$
 $Salida\ ALU \leftarrow PC + extSigno16a32(IR[15-0]) \ll 2$
- 3: $MDR \leftarrow M[PC]$

$PC \leftarrow PC + 4$

4: Si ($A = B$) entonces $PC \leftarrow MDR$

6. [Ejercicio de examen] Sea un procesador MIPS con un camino de datos multiciclo (se adjunta con el enunciado). Al repertorio seleccionado en clase, se desea añadirle una instrucción inventada llamada `skip1eq`, donde *skip one if equal* en inglés significa “saltarse una si es igual”. Un ejemplo de esta instrucción en ensamblador sería el siguiente:

`skip1eq ($s0), $s1`

Esta instrucción hace lo siguiente:

- Lee los dos registros indicados, en este caso `$s0` y `$s1`.
- El valor del registro `$s0` es una dirección de memoria, de forma que se realiza la lectura de la palabra contenida en dicha posición.
- Finalmente, comparamos la palabra que acabamos de leer con el valor del registro `$s1`.
 - Si ambos valores son iguales, nos saltamos 1 instrucción, la que va inmediatamente después de esta en el programa, y la ejecución continúa con normalidad con la siguiente. Para `skip1eq`, el hecho de saltarnos 1 instrucción es algo fijo y constante y no se puede modificar de ninguna manera. Tampoco está codificado en la instrucción máquina.
 - Si los valores no son iguales, no se hace nada y la ejecución continúa con normalidad con la instrucción que va inmediatamente después de `skip1eq` en el programa.

Se pide:

- a) Desde el punto de vista de la **arquitectura** (tal cual se vería en una tabla de resumen de instrucciones), escribir qué operación hace esta instrucción en notación de **pseudocódigo**.
- b) Justificar qué **formato binario** de instrucción sería el más adecuado para esta instrucción y descomponerla en campos.
- c) Indicar cuáles son **todos los operandos** de la instrucción, así como los **modos de direccionamiento** de cada uno de ellos.
- d) Explicar razonada y detalladamente **todos los cambios** que son necesarios en el camino de datos para poder ejecutar esta instrucción. Los requisitos que debe cumplir la instrucción son los siguientes:
 - El hecho de añadir esta nueva instrucción debe ser retrocompatible con todas las instrucciones estándar, que deben seguir siendo ejecutables como hasta ahora.
 - El número total de ciclos para ejecutar la nueva instrucción debe ser el mínimo posible. Para ello se podrán solapar en el mismo ciclo operaciones simultáneas que no interfieran entre sí.
 - El número total de estados de la unidad de control debe ser el mínimo posible.
 - El número total de cambios en el camino de datos debe ser el mínimo posible y también estos cambios han de ser lo más pequeños posible.

Este apartado tiene tres puntos de vista interrelacionados que habrá que explicar:

- Por un lado, habrá que **explicar** las modificaciones y **dibujarlas** sobre el camino de datos, tales como añadir cables o ampliar otros existentes, añadir multiplexores o ampliar otros existentes, añadir registros o ampliar otros existentes, añadir puertas lógicas o ampliar las existentes, etc.
- Por otro, habrá que **explicar, ciclo a ciclo**, qué operaciones se realizan (desde el punto de vista de la **organización**, es decir, indicando los componentes concretos del camino de datos involucrados). Para ello, se deberá usar la notación de **pseudocódigo** con el operador de asignación flecha $\leftarrow$ para indicar todas las operaciones que intervienen en dicho ciclo. Además del pseudocódigo:
 - Si el ciclo corresponde a un estado nuevo de la unidad de control, habrá que explicar e indicar el valor de **todas las señales de control relevantes**.
 - Si el ciclo corresponde a un estado existente que ahora es modificado, habrá que explicar e indicar **solo** el valor de las señales de control **nuevas o modificadas**.
 - Si el ciclo corresponde a un estado existente que no se modifica, **no** será necesario explicar ni indicar el valor de las señales de control.
- Finalmente, habrá que ampliar el **diagrama de estados** de la unidad de control, así como la **tabla de verdad** de las salidas de la **unidad de control** para que incluyan los cambios realizados. Ambos se adjuntan con el enunciado.

Solución:

a) Esta instrucción realiza las siguientes operaciones **desde el punto de vista de la arquitectura**:

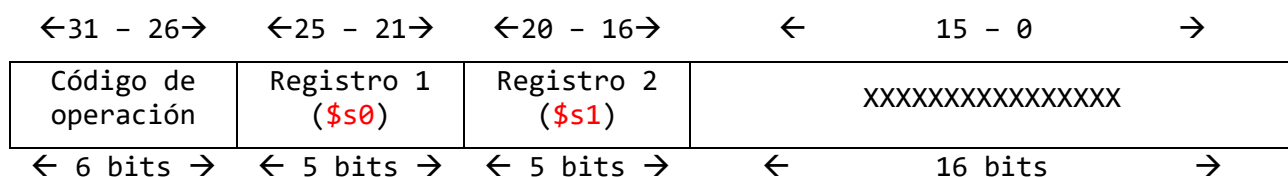
$PC \leftarrow PC + 4$

Si $(M[\$s0] = \$s1)$ entonces $PC \leftarrow PC + 4$

Como este pseudocódigo se realiza desde el punto de vista de la arquitectura, aquí solo pueden aparecer registros conocidos desde el punto de vista de la programación en ensamblador. Así es como aparecería explicada la instrucción en la tabla del Patterson & Hennessy, por ejemplo. Los registros **A**, **B**, **MDR**, **IR**, etc. son de la organización multiciclo y se reservan para el apartado **d**).

b) Esta instrucción necesita codificar 2 registros, con lo que podemos elegir indistintamente entre el formato I y el R.

Formato I:



Formato R:

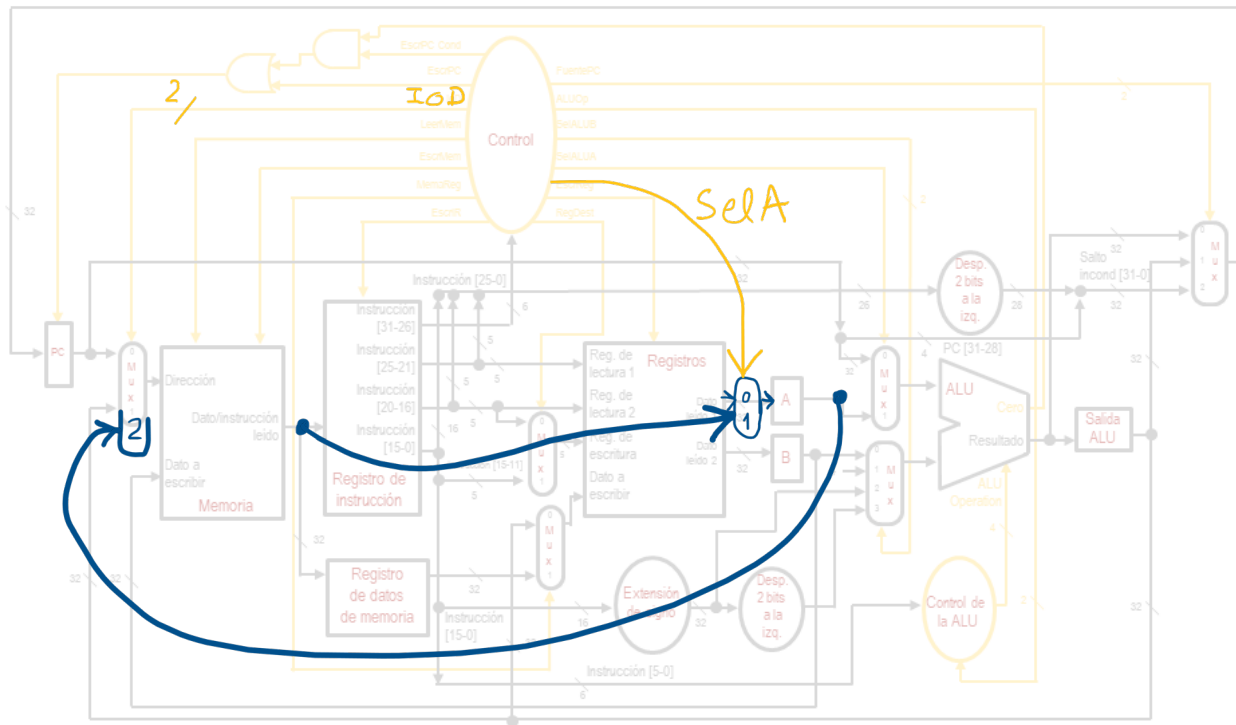
← 31 - 26 →	← 25 - 21 →	← 20 - 16 →	← 15 - 11 →	← 10 - 6 →	← 5 - 0 →
Código de operación	Registro 1 (\$s0)	Registro 2 (\$s1)	XXXXX	XXXXX	XXXXXX
← 6 bits →	← 5 bits →	← 5 bits →	← 5 bits →	← 5 bits →	← 6 bits →

c) Los operandos de esta instrucción son los siguientes:

- Un dato ubicado en la memoria (operando fuente), cuya dirección se especifica a partir de registro \$s0. Por tanto, tiene un direccionamiento indirecto a registro (sin desplazamiento). Nótese que el operando no es \$s0, sino el dato en la memoria al que apunta dicho registro.
- El registro \$s1 (operando fuente), cuyo valor debemos comparar con el dato en memoria. Por tanto, tiene direccionamiento directo a registro.
- Como siempre, el registro PC, que se ve incrementado en 4 (o en 8) unidades, es un operando con direccionamiento implícito.

d) Los cambios que se proponen en el camino de datos son los siguientes (como siempre, hay varias soluciones posibles):

- Añadimos un cable desde la salida del registro A hasta el multiplexor controlado por la señal IoD. El multiplexor tiene ahora una nueva entrada y dicha señal de control pasa a ser de 2 bits (IoD1 IoD0).
- Por otro lado, añadimos un nuevo multiplexor de tamaño 2 a 1 a la izquierda de A. La entrada 0 proviene de Dato leído 1 del banco de registros, mientras que la entrada 1 viene de Dato/Instrucción leído de la memoria. Aparece una nueva señal de control para el multiplexor, que llamaremos SelA.



La ejecución ciclo a ciclo sería la siguiente:

Ciclo 1 (IF)/Estado 0: La lectura de la instrucción se realiza como en el resto de las instrucciones. Las operaciones realizadas son:

$IR \leftarrow M[PC]$ // *IR* contiene la palabra de operación de la instrucción
 $PC \leftarrow PC + 4$

Las señales de control son las habituales para este estado. Debemos recordar que *IoD* ahora es de 2 bits y su valor es 00.

Ciclo 2 (ID)/Estado 1: La decodificación de la instrucción se realiza como en el resto de las instrucciones. Las operaciones realizadas son:

$A \leftarrow \text{Reg}[IR[25-21]]$
 $B \leftarrow \text{Reg}[IR[20-16]]$
 $\text{Salida ALU} \leftarrow PC + \text{extSigno16a32}(IR[15-0]) \ll 2$

Como novedad, la señal *Sela* debe valer 0 para que se almacene el valor leído en el banco en el registro A. Por su parte, el valor de *Salida ALU* es irrelevante para esta instrucción, pero aun así se calcula por retrocompatibilidad con el resto de las instrucciones.

Las señales de control son las habituales para este estado.

Ciclo 3 (EX/MEM)/Estado 10: Este ciclo se realiza con un estado nuevo. Se lee la dirección de memoria que coincide con el registro A (donde se ha guardado *\$s0*) y la palabra leída se vuelve a guardar en A. Asimismo,

aprovechamos para preparar en la ALU el nuevo valor del **PC** en caso de que tengamos que saltamos la siguiente instrucción. Las operaciones realizadas entonces son:

$A \leftarrow M[A]$
 $Salida\ ALU \leftarrow PC + 4$

Las señales de control necesarias son las siguientes:

EscrPC = 0 (protegemos el registro **PC** contra escrituras indeseadas)
EscrPC Cond = 0 (aunque es una instrucción condicional, en este ciclo no debemos tocar **PC**)
IoD = 10 (seleccionamos el valor de **A** como dirección de memoria a la que vamos a acceder)
LeerMem = 1 (leemos en memoria)
EscrMem = 0 (protegemos la memoria contra escrituras indeseadas)
EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)
SeIA = 1 (seleccionamos el valor que acabamos de leer en la memoria para guardarlo en **A**)
ALUOp = 00 (sumar)
SeIALUB = 01 (seleccionamos 4 como segundo operando de la **ALU**)
SeIALUA = 0 (seleccionamos **PC** como primer operando de la **ALU**)
EscrReg = 0 (protegemos el banco de registros contra escrituras indeseadas)

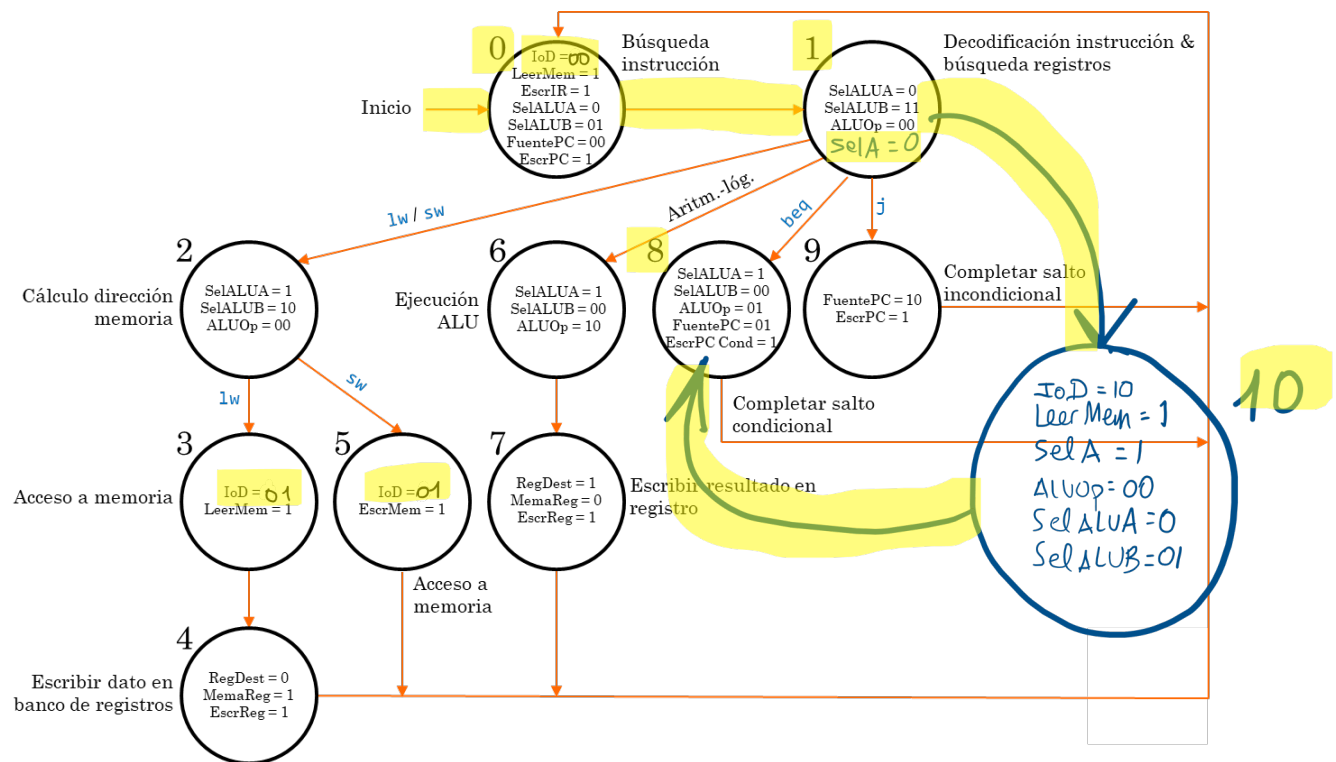
El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 4 (EX)/Estado 8: Este ciclo coincide con la operación normal de la instrucción **beq**. El registro **A** contiene el dato en memoria al que apunta el registro **\$s0**, mientras que **B** contiene el valor del registro **\$s1**. Esos son precisamente los valores que queremos comparar para ver si son iguales. Para ello, se restan en la ALU y, en función del resultado de la resta, se activará o no la señal **Cero**. Solo en ese caso, el valor de **Salida ALU (PC+4)** se guarda en **PC**. Las operaciones realizadas son:

Si (**A** = **B**) entonces $PC \leftarrow Salida\ ALU$

Las señales de control son las habituales para este estado.

El nuevo diagrama de estados es:



La secuencia de estados de la nueva instrucción sería: $0 \rightarrow 1 \rightarrow 10 \rightarrow 8 \rightarrow 0$.

La nueva tabla de verdad de las salidas de la unidad de control principal sería:

Estado actual	EscrPC	EscrPCCond	$IoD1$	$IoD0$	LeerMem	EscrMem	EscrIR	MemaReg	FuentePC1	FuentePC0	ALUOp1	ALUOp0	SelALUB1	SelALUB0	SelALUA0	EscrReg	RegDest	SelA
0000	1	0	0	0	1	0	1	X	0	0	0	0	0	1	0	0	X	X
0001	0	0	X	X	0	0	0	X	X	X	0	0	1	1	0	0	X	0
0010	0	0	X	X	0	0	0	X	X	X	0	0	1	0	1	0	X	X
0011	0	0	0	1	1	0	0	X	X	X	X	X	X	X	X	0	X	X
0100	0	0	X	X	0	0	0	1	X	X	X	X	X	X	X	1	0	X
0101	0	0	0	1	0	1	0	X	X	X	X	X	X	X	X	0	X	X
0110	0	0	X	X	0	0	0	X	X	X	1	0	0	0	1	0	X	X
0111	0	0	X	X	0	0	0	0	X	X	X	X	X	X	X	1	1	X
1000	0	1	X	X	0	0	0	X	0	1	0	1	0	0	1	0	X	X
1001	1	0	X	X	0	0	0	X	1	0	X	X	X	X	X	0	X	X
1010	0	0	1	0	1	0	0	X	X	X	0	0	0	1	0	0	X	1

Microoperaciones realizadas por la instrucción, desde el punto de vista de la organización multiciclo:

- $IR \leftarrow M[PC]$
 $PC \leftarrow PC + 4$
- $A \leftarrow Reg[IR[25-21]]$
 $B \leftarrow Reg[IR[20-16]]$
 $Salida\ ALU \leftarrow PC + extSigno16a32(IR[15-0]) \ll 2$
- $A \leftarrow M[A]$
 $Salida\ ALU \leftarrow PC + 4$
- Si $(A = B)$ entonces $PC \leftarrow Salida\ ALU$

7. [Ejercicio de examen] Se desea añadir una nueva instrucción al repertorio del procesador MIPS en su implementación multiciclo. Se ha bautizado la instrucción como *swra* y tiene la siguiente sintaxis en ensamblador, donde *dirección* es un inmediato de 26 bits codificado en la instrucción:

swra dirección

El funcionamiento de la instrucción es el siguiente:

- Tras leerse la instrucción de la memoria, se toman los 26 bits de *dirección* y se transforman en 32 bits con un procedimiento similar al de la instrucción *j*.
- Se lee la palabra contenida en el registro *\$ra* (31) del banco, que, como puede verse, no aparece por ningún lado mencionado en la instrucción en ensamblador.
- Esta palabra debe escribirse en la dirección de memoria de 32 bits.

Se pide:

- a) Desde el punto de vista de la **arquitectura** (tal cual se vería en una tabla de resumen de instrucciones), escribir qué operación hace esta instrucción en notación de **pseudocódigo**.
- b) Justificar qué **formato binario** de instrucción sería el más adecuado para esta instrucción y descomponerla en campos. Habrá que indicar también **todos los operandos** de la instrucción, así como los modos de direccionamiento de cada uno de ellos.
- c) Explicar razonada y detalladamente **todos los cambios** que son necesarios en el camino de datos para poder ejecutar esta instrucción. Los requisitos que debe cumplir la instrucción son los siguientes:
 - El hecho de añadir esta nueva instrucción debe ser retrocompatible con todas las instrucciones estándar, que deben seguir siendo ejecutables como hasta ahora.
 - El número total de ciclos para ejecutar la nueva instrucción debe ser el mínimo posible. Para ello se podrán solapar en el mismo ciclo operaciones simultáneas que no interfieran entre sí.
 - El número total de estados de la unidad de control debe ser el mínimo posible.
 - El número total de cambios en el camino de datos debe ser el mínimo posible y también estos cambios han de ser lo más pequeños posible.

Este apartado tiene tres puntos de vista interrelacionados que habrá que explicar:

- Por un lado, habrá que **explicar** las modificaciones y **dibujarlas** sobre el camino de datos, tales como añadir cables o ampliar otros existentes, añadir multiplexores o ampliar otros existentes, añadir registros o ampliar otros existentes, añadir puertas lógicas o ampliar las existentes, etc.
- Por otro, habrá que **explicar, ciclo a ciclo**, qué operaciones se realizan (desde el punto de vista de la **organización**, es decir, indicando los componentes concretos del camino de datos involucrados). Para ello, se deberá usar la notación de **pseudocódigo** con el operador de asignación flecha $\leftarrow$ para indicar todas las operaciones que intervienen en dicho ciclo. Además del pseudocódigo:
 - Si el ciclo corresponde a un estado nuevo de la unidad de control, habrá que explicar e indicar el valor de **todas las señales de control relevantes**.

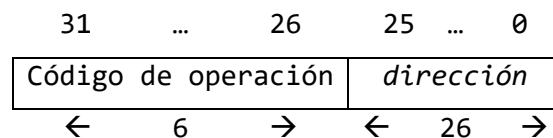
- Si el ciclo corresponde a un estado existente que ahora es modificado, habrá que explicar e indicar **solo** el valor de las señales de control **nuevas o modificadas**.
- Si el ciclo corresponde a un estado existente que no se modifica, **no** será necesario explicar ni indicar el valor de las señales de control.
- Finalmente, habrá que ampliar el **diagrama de estados** de la unidad de control, así como la **tabla de verdad** de las salidas de la **unidad de control** para que incluyan los cambios realizados. Ambos se adjuntan con el enunciado.

Solución:

a) Esta instrucción realiza las siguientes operaciones **desde el punto de vista de la arquitectura**:

$PC \leftarrow PC + 4$
 $M[PC[31-28] \parallel (dirección \ll 2)] \leftarrow \ra

b) De los tres formatos de instrucción de MIPS, el que mejor encaja es el J, que especifica una dirección de memoria mediante una pseudodirección en binario puro con 26 bits. Por tanto, tendremos:



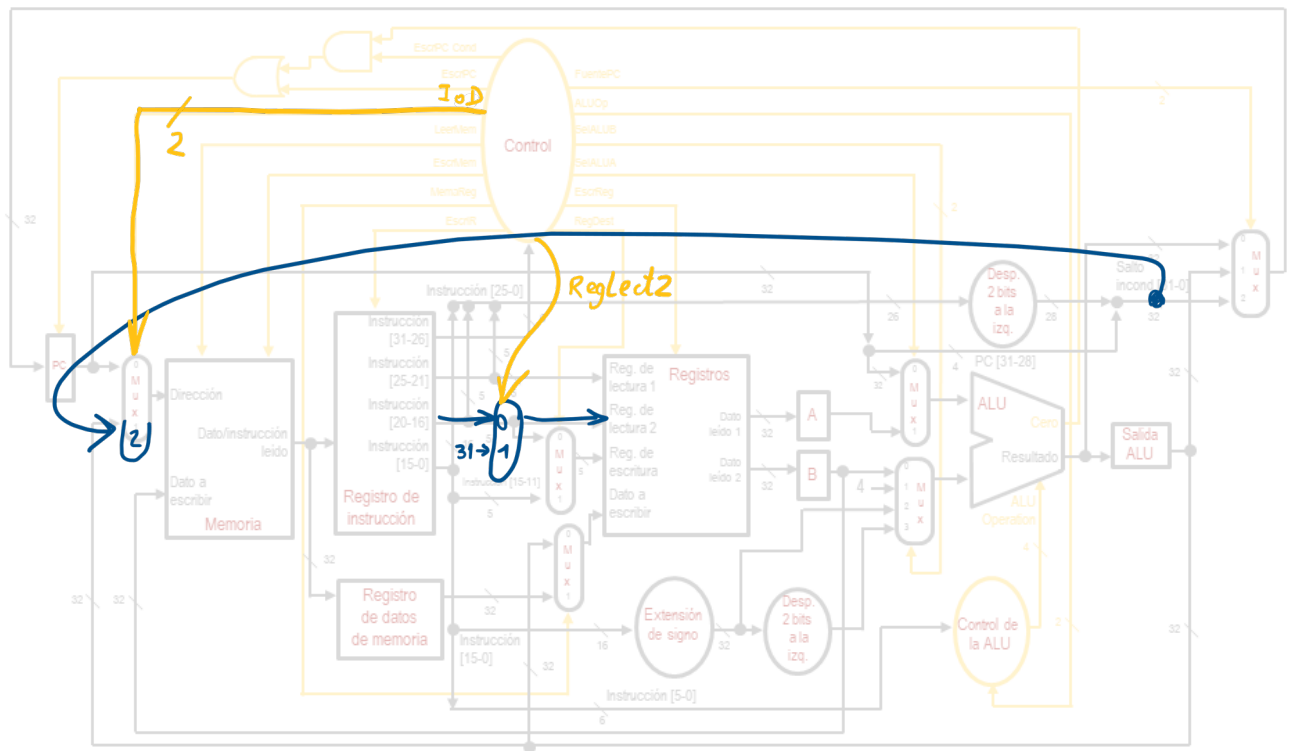
Esta instrucción tiene estos operandos:

- La dirección de memoria en la que hay que escribir, que se especifica mediante la pseudodirección de 26 bits contenida en la instrucción (modo direccionamiento pseudodirecto). Para calcularla se toma el valor de 26 bits, se multiplica por 4 (desplazamiento de 2 posiciones hacia la izquierda) y los 4 bits superiores se copian del valor del registro **PC** (cuando se haga esta operación el registro ya valdrá **PC + 4**). El resultado de esta operación es la dirección de memoria en la que escribimos.
- El registro **\$ra**, que es el 31 del banco de registros. Este es el dato que se ha de escribir en memoria. Como no se menciona en ninguna parte, usa un modo de direccionamiento implícito.
- El registro **PC**, que primero se actualiza sumándole 4 y que luego sirve para calcular la dirección de memoria en la que se escribe, pero que no se menciona en ningún lugar de la instrucción (modo de direccionamiento implícito).

c) Las modificaciones que se proponen son las siguientes:

- Llevamos un cable desde la entrada 2 del multiplexor controlado por la señal **FuentePC** a la entrada de dirección de la memoria, donde ampliamos el multiplexor controlado por **IoD**. Esta señal pasa a ser de 2 bits. En la entrada 2 conectamos el nuevo cable.
- Añadimos un nuevo multiplexor que elige cuál será el Registro de lectura 2 del banco, controlado por la señal **RegLect2**. Si vale 0, se toma el registro codificado por los bits 20-16 del registro **IR**, mientras que, si vale 1, se toma el valor constante 31 (en binario puro, 11111).

El camino de datos modificado quedaría de la siguiente manera:



La ejecución ciclo a ciclo de la nueva instrucción sería la siguiente:

Ciclo 1 (IF)/Estado 0: Este ciclo está sin modificar. La búsqueda de la instrucción se realiza como en el resto de las instrucciones. Las señales de control son las habituales en el estado 0, salvo por el hecho de que ahora **IoD** es de 2 bits.

$IR \leftarrow M[PC]$
 $PC \leftarrow PC + 4$

Las señales de control son las habituales para este estado.

Ciclo 2 (ID)/Estado 1: La decodificación de la instrucción se realiza como en el resto de las instrucciones, salvo por el hecho de que ahora existe la señal **RegLect2**.

$A \leftarrow \text{Reg}[IR[25-21]]$
 $B \leftarrow \text{Reg}[IR[20-16]]$
 $\text{Salida ALU} \leftarrow PC + \text{extSigno16a32}(IR[15-0]) \ll 2$

Los valores de los registros **A**, **B** y **Salida ALU** calculados en este ciclo son irrelevantes para esta instrucción, pero aun así se calculan por retrocompatibilidad con el resto de las instrucciones.

Las señales de control son las habituales para este estado.

Ciclo 3 (EX)/Estado 10: Ahora se debe leer el registro $\$ra$ (31) y se almacena en el registro B .

$B \leftarrow \text{Reg}[31]$

Señales de control:

$\text{EscrPC} = 0$ (protegemos el PC contra escrituras indeseadas)

$\text{EscrPC Cond} = 0$ (no es instrucción condicional)

$\text{LeerMem} = 0$ (no hay que leer la memoria)

$\text{EscrMem} = 0$ (protegemos la memoria contra escrituras indeseadas)

$\text{EscrIR} = 0$ (protegemos el registro IR contra escrituras indeseadas)

$\text{EscrReg} = 0$ (protegemos el banco de registros contra escrituras indeseadas)

$\text{RegLect2} = 1$ (seleccionamos el registro 31 o $\$ra$ como segundo registro que hay que leer en el banco)

El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 4 (MEM)/Estado 11: En este último ciclo, también nuevo, se escribe el valor de B en la memoria.

$M[\text{PC}[31-28] \parallel (\text{IR}[25-0] \ll 2)] \leftarrow B$

Señales de control:

$\text{EscrPC} = 0$ (protegemos el PC contra escrituras indeseadas)

$\text{EscrPC Cond} = 0$ (no es instrucción condicional)

$\text{IoD} = 10$ (seleccionamos la dirección de 32 bits construida a partir de la pseudodirección de 26 bits como dirección de memoria)

$\text{LeerMem} = 0$ (si se escribe en memoria, no puede leerse)

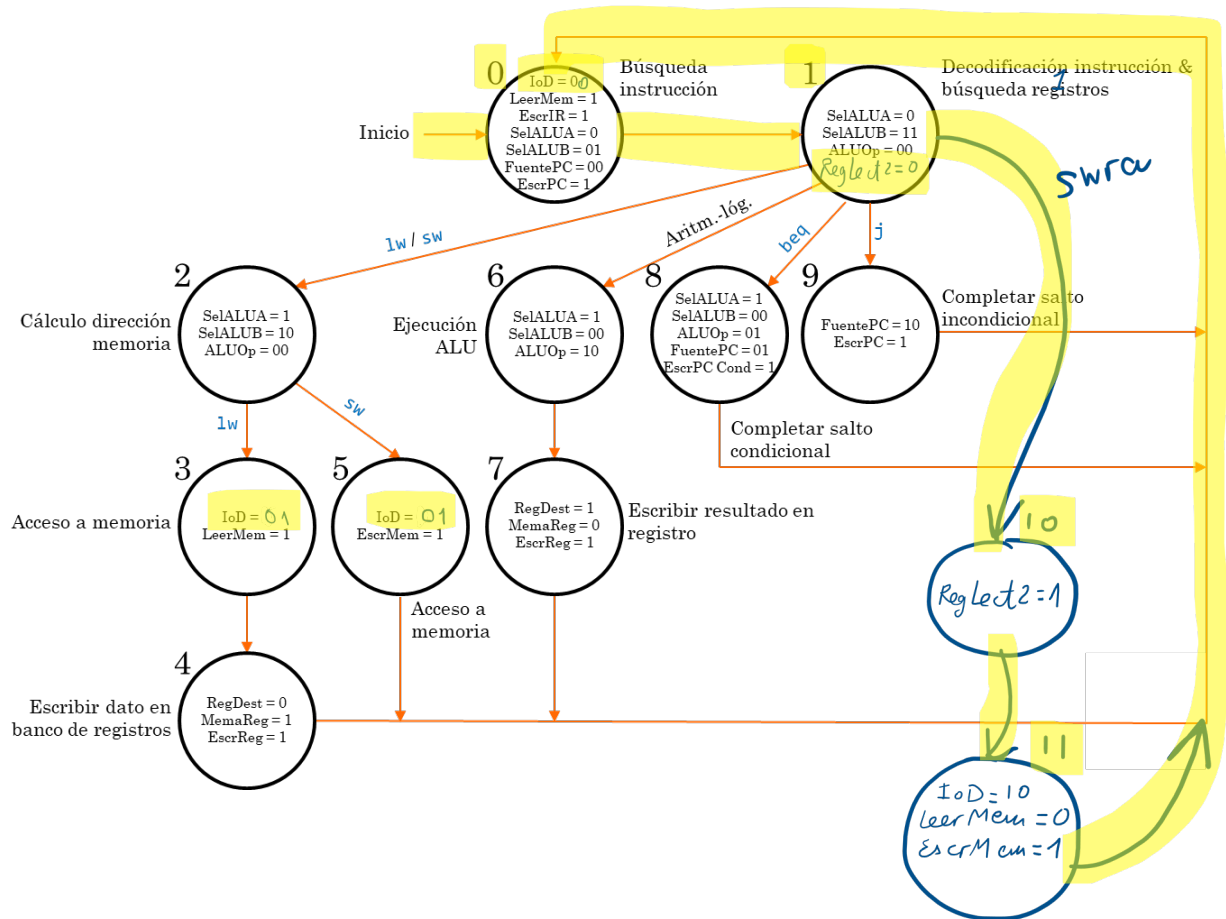
$\text{EscrMem} = 1$ (se escribe en memoria)

$\text{EscrIR} = 0$ (protegemos el registro IR contra escrituras indeseadas)

$\text{EscrReg} = 0$ (protegemos el banco de registros contra escrituras indeseadas)

El resto de las señales son irrelevantes para este ciclo (X).

El nuevo diagrama de estados es el siguiente:



Hemos añadido dos nuevos estados: el 10 y el 11 del 1. También hemos hecho que la señal **IoD** sea de 2 bits, lo que nos ha obligado a modificar también varios estados que no intervienen en esta instrucción, como el 3 y el 5.

La secuencia de estados de la nueva instrucción sería: 0 → 1 → 10 → 11 → 0.

La nueva tabla de verdad de las salidas de la unidad de control principal sería:

Estado actual	EscrPC	EscrPCCond	IoD1	IoD0	LeerMem	EscrMem	EscrIR	MemaReg	FuentePC1	FuentePC0	ALUOp1	ALUOp0	SelALUB1	SelALUB0	SelALUA	EscrReg	RegDest	RegLect2
0000	1	0	0	0	1	0	1	X	0	0	0	0	0	1	0	0	X	X
0001	0	0	X	X	0	0	0	X	X	X	0	0	1	1	0	0	X	0
0010	0	0	X	X	0	0	0	X	X	X	0	0	1	0	1	0	X	X
0011	0	0	0	1	1	0	0	X	X	X	X	X	X	X	X	0	X	X
0100	0	0	X	X	0	0	0	1	X	X	X	X	X	X	X	1	0	X
0101	0	0	0	1	0	1	0	X	X	X	X	X	X	X	X	0	X	X
0110	0	0	X	X	0	0	0	X	X	X	1	0	0	0	1	0	X	X
0111	0	0	X	X	0	0	0	0	X	X	X	X	X	X	X	1	1	X
1000	0	1	X	X	0	0	0	X	0	1	0	1	0	0	1	0	X	X
1001	1	0	X	X	0	0	0	0	X	1	0	X	X	X	X	0	X	X
1010	0	0	X	X	0	0	0	X	X	X	X	X	X	X	X	0	X	1
1011	0	0	1	0	0	1	0	X	X	X	X	X	X	X	X	0	X	0

La antigua señal **IoD** ahora corresponde a **IoD0**. Hemos añadido las filas correspondiente a los estados 10 (1010) y 11 (1011), así como las columnas correspondientes a las nuevas señales **IoD1** y **RegLect2**.

Para los nuevos estados, las señales **EscrPC**, **EscrPC Cond**, **LeerMem**, **EscrMem**, **EscrIR** y **EscrReg** se han mantenido a 0 para evitar que se modifique la memoria, el registro de instrucción o el banco de registros, salvo que en el estado 11 se activa **EscrMem**. El resto de las señales de control se han marcado a X al no intervenir en la instrucción y ser su valor irrelevante.

Microoperaciones realizadas por la instrucción, **desde el punto de vista de la organización multiciclo**:

```

1: IR ← M[PC]
   PC ← PC + 4
2: A ← Reg[IR[25-21]]
   B ← Reg[IR[20-16]]
   Salida ALU ← PC + extSigno16a32(IR[15-0])<<2
3: B ← Reg[31]
4: M[PC[31-28] || (IR[25-0]<<2)] ← B

```

8. **[Ejercicio de examen]** Se desea añadir una nueva instrucción al repertorio del procesador MIPS en su implementación multiciclo. Se ha bautizado la instrucción como **lwra** y tiene la siguiente sintaxis en ensamblador, donde *dirección* es un inmediato de 26 bits codificado en la instrucción:

lwra dirección

El funcionamiento de la instrucción es el siguiente:

- Tras leerse la instrucción de la memoria, se toman los 26 bits de *dirección* y se transforman en 32 bits con un procedimiento similar al de la instrucción **j**.
- La dirección de 32 bits obtenida se lee de la memoria.
- La palabra leída en dicha dirección de memoria se almacena en el registro **\$ra** (31) del banco, que, como puede verse, no aparece por ningún lado mencionado en la instrucción en ensamblador.

Se pide:

- a) Desde el punto de vista de la **arquitectura** (tal cual se vería en una tabla de resumen de instrucciones), escribir qué operación hace esta instrucción en notación de **pseudocódigo**.
- b) Justificar qué **formato binario** de instrucción sería el más adecuado para esta instrucción y descomponerla en campos. Habrá que indicar también **todos los operandos** de la instrucción, así como los **modos de direccionamiento** de cada uno de ellos.
- c) Explicar razonada y detalladamente **todos los cambios** que son necesarios en el camino de datos para poder ejecutar esta instrucción. Los requisitos que debe cumplir la instrucción son los siguientes:
 - El hecho de añadir esta nueva instrucción debe ser retrocompatible con todas las instrucciones estándar, que deben seguir siendo ejecutables como hasta ahora.
 - El número total de ciclos para ejecutar la nueva instrucción debe ser el mínimo posible. Para ello se podrán solapar en el mismo ciclo operaciones simultáneas que no interfieran entre sí.
 - El número total de estados de la unidad de control debe ser el mínimo posible.

- El número total de cambios en el camino de datos debe ser el mínimo posible y también estos cambios han de ser lo más pequeños posible.

Este apartado tiene tres puntos de vista interrelacionados que habrá que explicar:

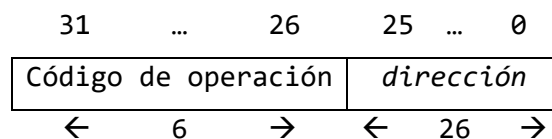
- Por un lado, habrá que **explicar** las modificaciones y **dibujarlas** sobre el camino de datos, tales como añadir cables o ampliar otros existentes, añadir multiplexores o ampliar otros existentes, añadir registros o ampliar otros existentes, añadir puertas lógicas o ampliar las existentes, etc.
- Por otro, habrá que **explicar, ciclo a ciclo**, qué operaciones se realizan (desde el punto de vista de la **organización**, es decir, indicando los componentes concretos del camino de datos involucrados). Para ello, se deberá usar la notación de **pseudocódigo** con el operador de asignación flecha $\leftarrow$ para indicar todas las operaciones que intervienen en dicho ciclo. Además del pseudocódigo:
 - Si el ciclo corresponde a un estado nuevo de la unidad de control, habrá que explicar e indicar el valor de **todas las señales de control relevantes**.
 - Si el ciclo corresponde a un estado existente que ahora es modificado, habrá que explicar e indicar **solo** el valor de las señales de control **nuevas o modificadas**.
 - Si el ciclo corresponde a un estado existente que no se modifica, **no** será necesario explicar ni indicar el valor de las señales de control.
- Finalmente, habrá que ampliar el **diagrama de estados** de la unidad de control, así como la **tabla de verdad** de las salidas de la **unidad de control** para que incluyan los cambios realizados. Ambos se adjuntan con el enunciado.

Solución:

a) Esta instrucción realiza las siguientes operaciones **desde el punto de vista de la arquitectura**:

```
PC  $\leftarrow$  PC + 4
$ra  $\leftarrow$  M[PC[31-28] || (dirección<<2)]
```

b) De los tres formatos de instrucción de MIPS, el que mejor encaja es el J, que especifica una dirección de memoria mediante una pseudodirección en binario puro con 26 bits. Por tanto, tendremos:



Esta instrucción tiene estos operandos:

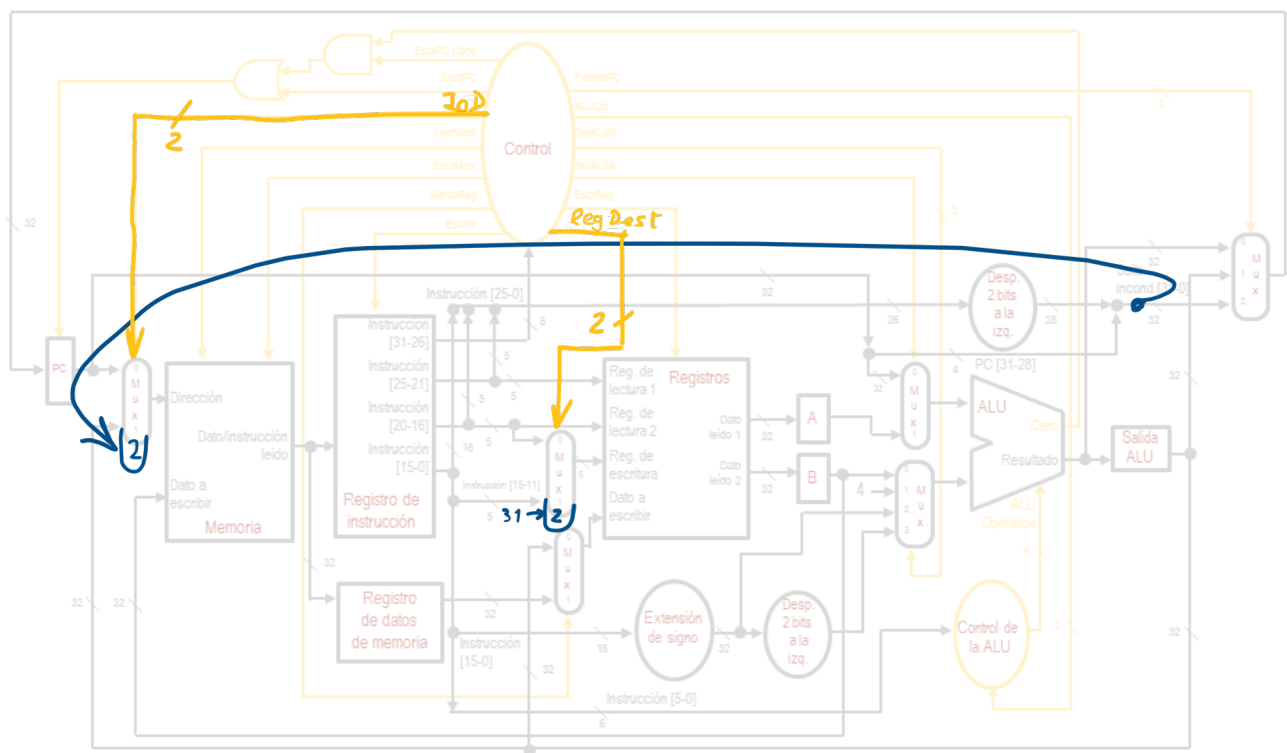
- La dirección de memoria que hay que leer, que se especifica mediante la pseudodirección de 26 bits contenida en la instrucción (modo direccionamiento pseudodirecto). Para calcularla se toma el valor de 26 bits, se multiplica por 4 (desplazamiento de 2 posiciones hacia la izquierda) y los 4 bits superiores se copian del valor del registro **PC** (cuando se haga esta operación el registro ya valdrá **PC + 4**). El resultado de esta operación es la dirección de memoria que leemos.

- El registro **\$ra**, que es el 31 del banco de registros. Allí es donde se escribe el dato leído en memoria. Como no se menciona en ninguna parte, usa un modo de direccionamiento implícito.
- El registro **PC**, que primero se actualiza sumándole 4 y que luego sirve para calcular la dirección de memoria en la que se lee, pero que no se menciona en ningún lugar de la instrucción (modo de direccionamiento implícito).

c) Las modificaciones que se proponen son las siguientes:

- Llevamos un cable desde la entrada 2 del multiplexor controlado por la señal **FuentePC** hasta la entrada de dirección de la memoria, donde ampliamos el multiplexor allí existente. De esta forma, la señal **IoD** pasa a ser de 2 bits. El nuevo cable se conecta en la entrada 2.
- El multiplexor controlado por la señal **RegDest** se amplía y en la nueva entrada 2 colocamos un valor constante de 31 (en binario puro, 11111). De esta forma, la señal **RegDest** también pasa a ser de 2 bits.

El camino de datos modificado quedaría de la siguiente manera:



La ejecución ciclo a ciclo de la nueva instrucción sería la siguiente:

Ciclo 1 (IF)/Estado 0: Este ciclo está sin modificar. La búsqueda de la instrucción se realiza como en el resto de las instrucciones. Las señales de control son las habituales en el estado 0, salvo por el hecho de que ahora **IoD** es de 2 bits.

IR $\leftarrow$ **M**[**PC**]
PC $\leftarrow$ **PC** + 4

Las señales de control son las habituales para este estado.

Ciclo 2 (ID)/Estado 1: La decodificación de la instrucción se realiza como en el resto de las instrucciones.

$A \leftarrow \text{Reg}[\text{IR}[25-21]]$
 $B \leftarrow \text{Reg}[\text{IR}[20-16]]$
 $\text{Salida ALU} \leftarrow \text{PC} + \text{extSigno16a32}(\text{IR}[15-0]) \ll 2$

Los valores de los registros **A**, **B** y **Salida ALU** calculados en este ciclo son irrelevantes para esta instrucción, pero aun así se calculan por retrocompatibilidad con el resto de las instrucciones.

Las señales de control son las habituales para este estado.

Ciclo 3 (MEM)/Estado 10: En este nuevo estado, calculamos la dirección de memoria a partir de la pseudodirección de 26 bits y se lee dicha posición. El dato leído lo almacenamos en el registro **MDR**.

$\text{MDR} \leftarrow \text{M}[\text{PC}[31-28] \ || \ (\text{IR}[25-0] \ll 2)]$

Señales de control:

EscrPC = 0 (protegemos el **PC** contra escrituras indeseadas)
EscrPC Cond = 0 (no es instrucción condicional)
IoD = 10 (seleccionamos la dirección de 32 bits construida a partir de la pseudodirección de 26 bits como dirección de memoria)
LeerMem = 1 (se lee la memoria)
EscrMem = 0 (si se lee la memoria, no se puede escribir en ella)
EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)
EscrReg = 0 (protegemos el banco de registros contra escrituras indeseadas)
El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 4 (WB)/Estado 11: En este último estado, también nuevo, se escribe el valor contenido en el registro **MDR** en el registro \$ra (31) del banco de registros.

$\text{Reg}[31] \leftarrow \text{MDR}$

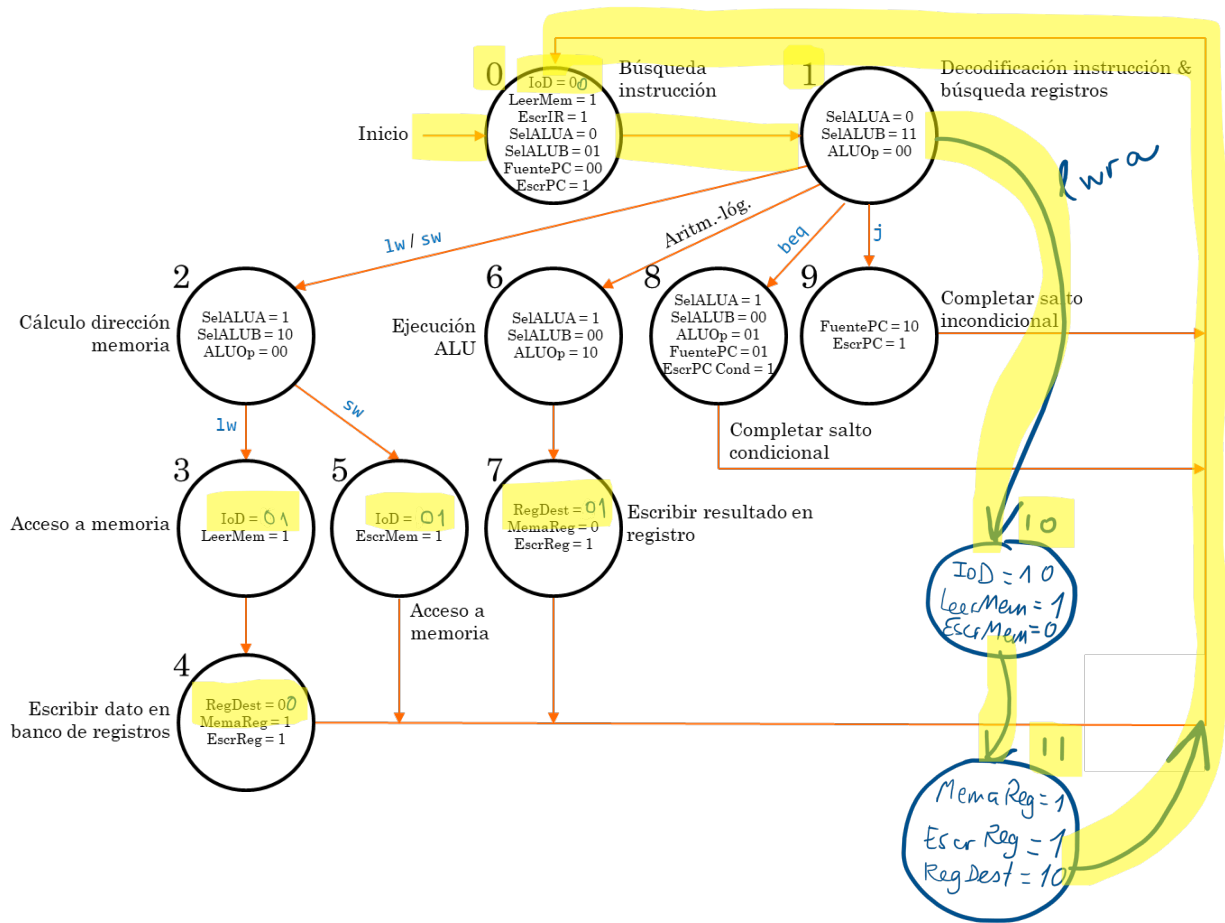
Señales de control:

EscrPC = 0 (protegemos el **PC** contra escrituras indeseadas)
EscrPC Cond = 0 (no es instrucción condicional)
LeerMem = 0 (no hay que leer la memoria)
EscrMem = 0 (protegemos la memoria contra escrituras indeseadas)
EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)
MemaReg = 1 (seleccionamos **MDR** como dato que hay que escribir en el banco)
EscrReg = 1 (activamos la escritura en el banco de registros)

RegDest = 10 (seleccionamos el registro 31 o **\$ra** como registro en el que hay que escribir)

El resto de las señales son irrelevantes para este ciclo (X).

El nuevo diagrama de estados es el siguiente:



Hemos añadido dos nuevos estados: el 10 y el 11. También hemos hecho que las señales **IoD** y **RegDest** sean de 2 bits, lo que nos ha obligado a modificar también varios estados que no intervienen en esta instrucción, como el 3, el 4, el 5 y el 7.

La secuencia de estados de la nueva instrucción sería: $0 \rightarrow 1 \rightarrow 10 \rightarrow 11 \rightarrow 0$.

La nueva tabla de verdad de las salidas de la unidad de control principal sería:

Estado actual	EscrPC	EscrPCCond	IoD1	IoD0	LeerMem	EscrMem	EscrIR	MemReg	FuentePC1	FuentePC0	ALUp1	ALUp0	Se1ALUB1	Se1ALUB0	Se1ALUA	EscrReg	RegDest1	RegDest0
0000	1	0	0	0	1	0	1	X	0	0	0	0	0	1	0	0	X	X
0001	0	0	X	X	0	0	0	X	X	X	0	0	1	1	0	0	X	X
0010	0	0	X	X	0	0	0	X	X	X	0	0	1	0	1	0	X	X
0011	0	0	0	1	1	0	0	X	X	X	X	X	X	X	X	0	X	X
0100	0	0	X	X	0	0	0	1	X	X	X	X	X	X	X	1	0	0
0101	0	0	0	1	0	1	0	X	X	X	X	X	X	X	X	0	X	X
0110	0	0	X	X	0	0	0	X	X	X	1	0	0	0	1	0	X	X
0111	0	0	X	X	0	0	0	0	X	X	X	X	X	X	X	1	0	1
1000	0	1	X	X	0	0	0	X	0	1	0	1	0	0	1	0	X	X
1001	1	0	X	X	0	0	0	X	1	0	X	X	X	X	X	0	X	X
1010	0	0	1	0	1	0	0	X	X	X	X	X	X	X	X	0	X	X
1011	0	0	X	X	0	0	0	1	X	X	X	X	X	X	X	1	1	0

La antigua señal **IoD** ahora corresponde a **IoD0**, de igual forma que la antigua señal **RegDest** ahora es **RegDest0**. Hemos añadido las filas correspondiente a los estados 10 (**1010**) y 11 (**1011**), así como las columna correspondientes a las nuevas señales **IoD1** y **RegDest1**.

Para los nuevos estados, las señales **EscrPC**, **EscrPC Cond**, **LeerMem**, **EscrMem**, **EscrIR** y **EscrReg** se han mantenido a 0 para evitar que se modifique la memoria, el registro de instrucción o el banco de registros, salvo que en el estado 10 se activa **LeerMem**. El resto de las señales de control se han marcado a X al no intervenir en la instrucción y ser su valor irrelevante.

Microoperaciones realizadas por la instrucción, desde el punto de vista de la organización multiciclo:

- 1: **IR** $\leftarrow$ **M[PC]**
PC $\leftarrow$ **PC** + 4
- 2: **A** $\leftarrow$ **Reg[IR[25-21]]**
B $\leftarrow$ **Reg[IR[20-16]]**
Salida ALU $\leftarrow$ **PC** + extSigno16a32(**IR**[15-0])<<2
- 3: **MDR** $\leftarrow$ **M[PC[31-28] || (**IR**[25-0]<<2)]**
- 4: **Reg[31]** $\leftarrow$ **MDR**

9. [Ejercicio de examen] Sea un procesador MIPS con un camino de datos multiciclo. Se desea añadir una nueva instrucción al repertorio, denominada **swap**, que intercambia los valores de dos registros del banco. Un ejemplo de esta instrucción en ensamblador sería el siguiente:

swap \$s0, \$t0

Esta instrucción hace lo siguiente: Guarda una copia de seguridad del valor de **\$t0**. A continuación, el valor de **\$s0** se escribe en **\$t0** y, finalmente, el valor salvado de **\$t0** se escribe en **\$s0**. (Si se prefiere, se pueden invertir los registros en estas operaciones).

Se pide:

- a) Desde el punto de vista de la **arquitectura** (tal cual se vería en una tabla de resumen de instrucciones), escribir qué operación hace esta instrucción en notación de **pseudocódigo**.

- b) Justificar qué **formato binario** de instrucción sería el más adecuado para esta instrucción y descomponerla en campos. Habrá que indicar también **todos los operandos** de la instrucción, así como los **modos de direccionamiento** de cada uno de ellos.
- c) Escribir la instrucción que se pone en el ejemplo en **formato hexadecimal**.
- d) Explicar razonada y detalladamente **todos los cambios** que son necesarios en el camino de datos para poder ejecutar esta instrucción. Los requisitos que debe cumplir la instrucción son los siguientes:

IMPORTANTE: Para guardar la copia de seguridad del valor de uno de los registros **NO** se podrá usar ningún registro del banco ni tampoco ninguna posición de la memoria.

- El hecho de añadir esta nueva instrucción debe ser retrocompatible con todas las instrucciones estándar, que deben seguir siendo ejecutables como hasta ahora.
- El número total de ciclos para ejecutar la nueva instrucción debe ser el mínimo posible. Para ello se podrán solapar en el mismo ciclo operaciones simultáneas que no interfieran entre sí.
- El número total de estados de la unidad de control debe ser el mínimo posible.
- El número total de cambios en el camino de datos debe ser el mínimo posible y también estos cambios han de ser lo más pequeños posible.

Este apartado tiene tres puntos de vista interrelacionados que habrá que explicar:

- Por un lado, habrá que **explicar** las modificaciones y **dibujarlas** sobre el camino de datos, tales como añadir cables o ampliar otros existentes, añadir multiplexores o ampliar otros existentes, añadir registros o ampliar otros existentes, añadir puertas lógicas o ampliar las existentes, etc.
- Por otro, habrá que **explicar, ciclo a ciclo**, qué operaciones se realizan (desde el punto de vista de la **organización**, es decir, indicando los componentes concretos del camino de datos involucrados). Para ello, se deberá usar la notación de **pseudocódigo** con el operador de asignación flecha **←** para indicar todas las operaciones que intervienen en dicho ciclo. Además del pseudocódigo:
 - Si el ciclo corresponde a un estado nuevo de la unidad de control, habrá que explicar e indicar el valor de **todas las señales de control relevantes**.
 - Si el ciclo corresponde a un estado existente que ahora es modificado, habrá que explicar e indicar **solo** el valor de las señales de control **nuevas o modificadas**.
 - Si el ciclo corresponde a un estado existente que no se modifica, **no** será necesario explicar ni indicar el valor de las señales de control.
- Finalmente, habrá que ampliar el **diagrama de estados** de la unidad de control, así como la **tabla de verdad** de las salidas de la **unidad de control** para que incluyan los cambios realizados. Ambos se adjuntan con el enunciado.

IMPORTANTE: Se recuerda que el valor leído de cualquier registro, incluidos los del banco, está accesible en la primera mitad del ciclo, mientras que la escritura de un valor en un registro se produce siempre al final del ciclo.

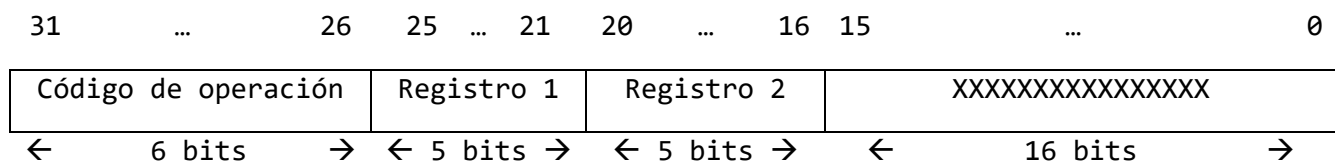
Solución:

a) Esta instrucción realiza las siguientes operaciones **desde el punto de vista de la arquitectura**:

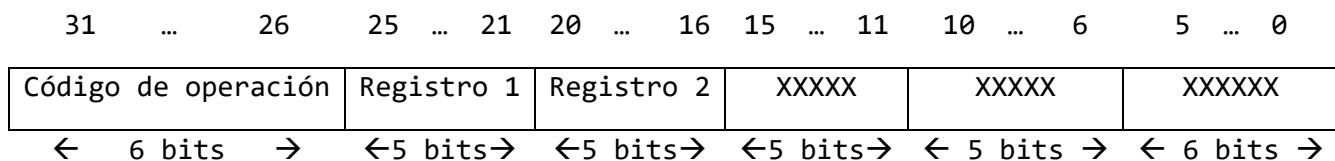
$PC \leftarrow PC + 4$
 $\$s0 \leftrightarrow \$t0$

b) Los formatos binarios más adecuados pueden ser el I o el R, ya que ambos permiten especificar en la instrucción dos registros. El formato J no nos serviría de ninguna de las maneras.

Tomando el formato I, la descomposición en campos de la instrucción sería la siguiente:



Por su lado, tomando el formato R, la descomposición en campos de la instrucción sería la siguiente:

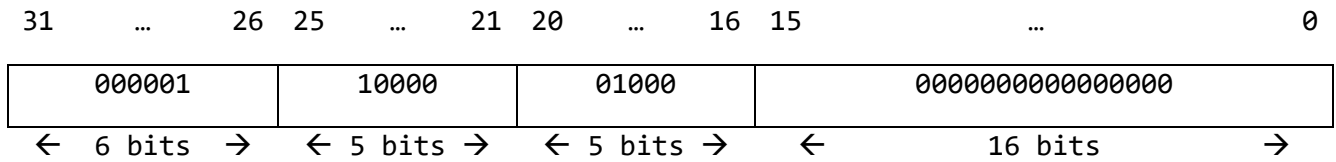


En ambos casos, en los bits 25 a 21 estaría codificado con 5 bits el número del registro $\$s0$ (16), mientras que en los bits 20 a 16 estaría el número del registro $\$t0$ (8). De esta forma, se leerán estos dos registros de manera natural en el ciclo 2 de la instrucción. Por otro lado, el resto de los bits pueden tomar cualquier valor (los hemos representado como X), ya que se ignorarán.

En cuanto a los operandos de la instrucción, serían los siguientes:

- El primer registro (en este caso $\$s0$), que es fuente y destino. Su direccionamiento es directo a registro porque nos interesa su valor.
- El segundo registro (en este caso $\$t0$), que es también fuente y destino. Su direccionamiento es directo a registro por la misma razón que en el caso anterior.
- El contador de programa (PC), que es incrementado en 4 unidades para que apunte a la siguiente instrucción. Su direccionamiento es implícito.

c) Se trata de una instrucción inventada, así que buscaremos un código de operación que esté libre, por ejemplo, el 000001. El registro $\$s0$ es el 10000, mientras que el $\$t0$ es el 01000. Por otro lado, los campos no utilizados de la instrucción pueden tomar cualquier valor, pero, por simplicidad, consideraremos que contienen ceros. Por tanto, tomando el formato I, tenemos:

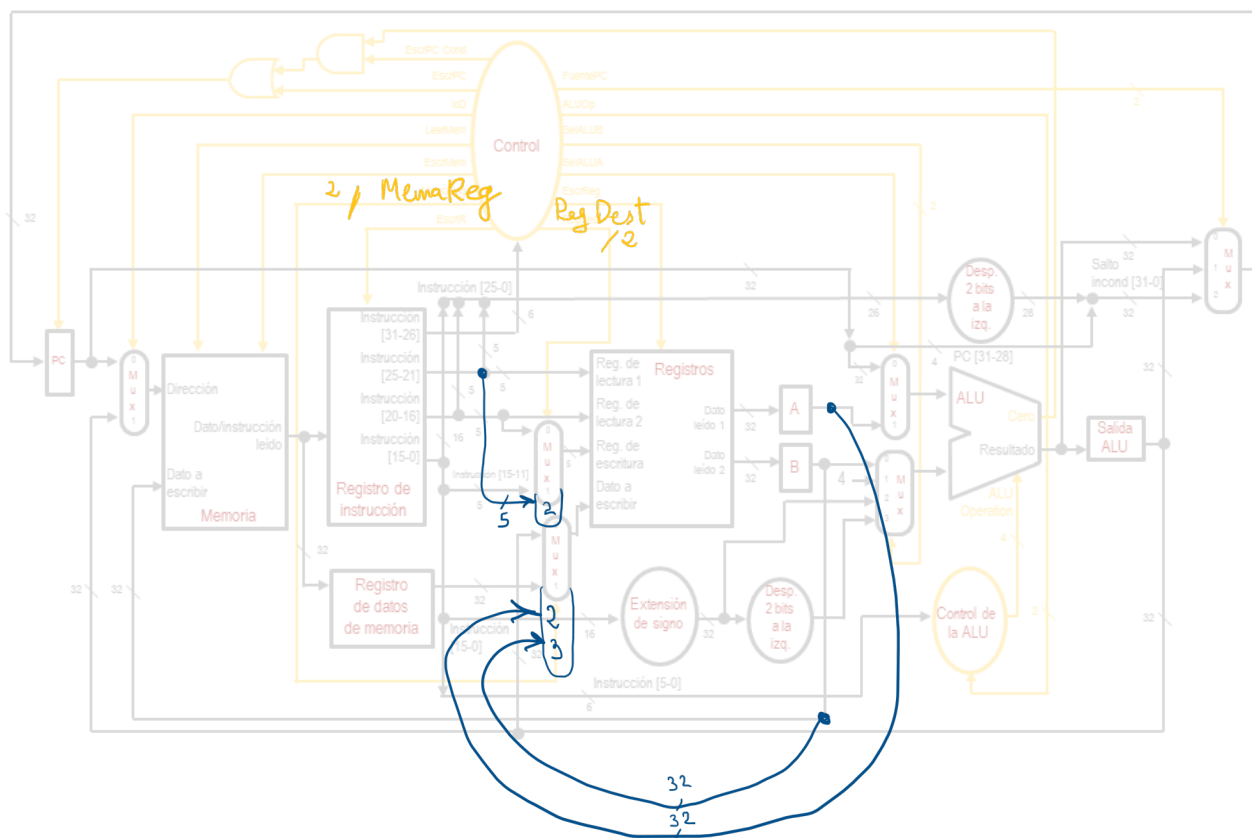


Luego: 000001 10000 01000 0000000000000000 = 0000 0110 0000 1000 0000 0000 0000 0000
= 0x06080000

d) Los cambios sugeridos para el camino de datos son los siguientes:

- Ampliamos el multiplexor controlado por la señal **MemaReg**, que es el que establece el dato que se va a escribir en el banco. Su tamaño pasa de “2 a 1” a “4 a 1”, de forma que ahora la entrada 2 proceda del registro **A**, mientras que la entrada 3 proceda del registro **B**. La señal de selección **MemaReg** pasa a ser de 2 bits.
- Por su parte, ampliamos el multiplexor controlado por la señal **RegDest**, que establece en qué registro del banco se produce una escritura. La entrada 2 procede de los bits 25 a 21 del registro **IR**. A su vez, la señal de selección **RegDest** pasa a ser de 2 bits.

El camino de datos modificado quedaría de la siguiente manera:



La ejecución ciclo a ciclo sería la siguiente:

Ciclo 1 (IF)/Estado 0: La lectura de la instrucción se realiza como en el resto de las instrucciones. Las operaciones realizadas son:

$$\begin{aligned} \text{IR} &\leftarrow \text{M}[\text{PC}] \\ \text{PC} &\leftarrow \text{PC} + 4 \end{aligned}$$

Las señales de control son las habituales para este estado.

Ciclo 2 (ID)/Estado 1: La decodificación de la instrucción se realiza como en el resto de las instrucciones. Las operaciones realizadas son:

$$\begin{aligned} \text{A} &\leftarrow \text{Reg}[\text{IR}[25-21]] \\ \text{B} &\leftarrow \text{Reg}[\text{IR}[20-16]] \\ \text{Salida ALU} &\leftarrow \text{PC} + \text{extSigno16a32}(\text{IR}[15-0]) \ll 2 \end{aligned}$$

El valor del registro **Salida ALU** calculado en este ciclo es irrelevante para esta instrucción, pero aun así se calculan por retrocompatibilidad con el resto de las instrucciones. Al finalizar el ciclo, el registro **A** vale **\$s0**, mientras que **B** vale **\$t0**.

Las señales de control son las habituales para este estado.

Ciclo 3 (WB)/Estado 10: Aquí comienza la parte distintiva de esta operación. Se lleva el valor del registro **A** (que contiene **\$s0**) a la entrada de Dato a escribir del banco de registros y se activa la escritura para escribirlo en **\$t0**, es decir:

$$\text{Reg}[\text{IR}[20-16]] \leftarrow \text{A}$$

Las señales de control necesarias son las siguientes:

EscrPC = 0 (protegemos el **PC** contra escrituras indeseadas)

EscrPC Cond = 0 (no es instrucción condicional)

LeerMem = 0 (no hay que leer la memoria)

EscrMem = 0 (protegemos la memoria contra escrituras indeseadas)

EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)

MemaReg = 10 (elegimos **A** para escribirlo en el banco de registros)

EscrReg = 1 (escribimos en el banco de registros)

RegDest = 00 (elegimos el segundo registro de la instrucción para escribir en él en el banco)

El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 4 (WB)/Estado 11: Al finalizar el ciclo anterior, ya hemos conseguido que el registro $\$t0$ del banco valga lo mismo que $\$s0$. En el ciclo actual se vuelve a leer $\$t0$ en el banco y su nuevo valor ($\$s0$) se escribirá en B al finalizar este ciclo. Pero antes de que se reescriba el registro B (que contiene todavía el valor original de $\$t0$), se lleva a la entrada de Dato a escribir del banco de registros y se activa la escritura para guardarlo en $\$s0$, es decir:

$$\text{Reg}[\text{IR}[25-21]] \leftarrow B$$

Las señales de control necesarias son las siguientes:

$\text{EscrPC} = 0$ (protegemos el PC contra escrituras indeseadas)

$\text{EscrPC Cond} = 0$ (no es instrucción condicional)

$\text{LeerMem} = 0$ (no hay que leer la memoria)

$\text{EscrMem} = 0$ (protegemos la memoria contra escrituras indeseadas)

$\text{EscrIR} = 0$ (protegemos el registro IR contra escrituras indeseadas)

$\text{MemaReg} = 11$ (elegimos B para escribirlo en el banco de registros)

$\text{EscrReg} = 1$ (escribimos en el banco de registros)

$\text{RegDest} = 10$ (elegimos el primer registro de la instrucción para escribir en él en el banco)

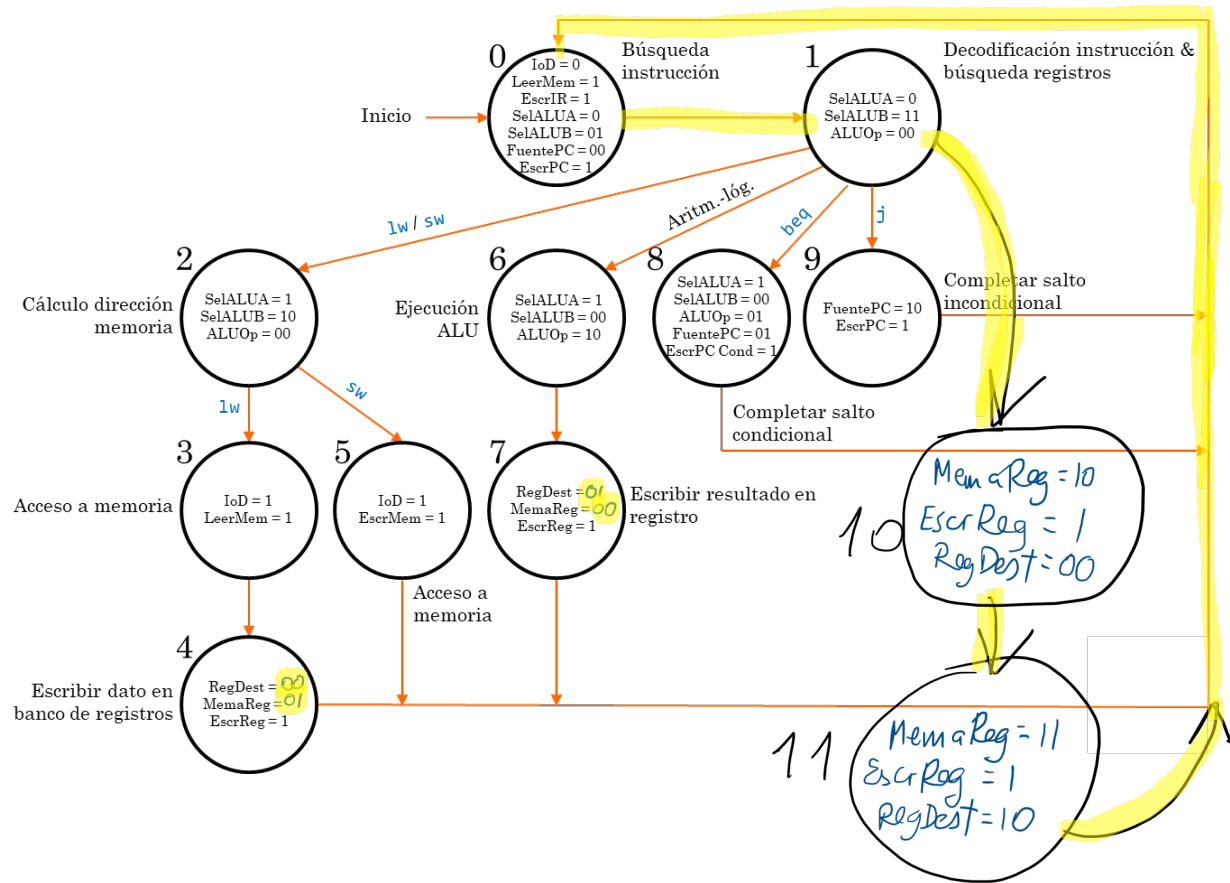
El resto de las señales son irrelevantes para este ciclo (X).

NOTA: Por simplicidad y tal y como indica explícitamente el enunciado, estamos suponiendo siempre que las lecturas de cualquier registro son rápidas y se producen en la primera parte del ciclo, mientras que las escrituras son más lentas y se producen en la segunda mitad del ciclo.

- Durante el ciclo 2, se leen los registros $\$s0$ y $\$t0$ en el banco y se escriben en A y en B .
- Durante el ciclo 3, puesto que el registro de instrucción IR no ha cambiado, se vuelven a leer los registros $\$s0$ y $\$t0$ en el banco, cuyos valores todavía no han cambiado. Estos valores inalterados se escriben al final del ciclo 3 de nuevo en A y en B .
- Pero también durante el ciclo 3 se lee el valor de A (que vale $\$s0$ desde el ciclo 2) y al final de este ciclo se escribe en el registro $\$t0$ del banco.
- Por tanto, al finalizar el ciclo 3 existe una disparidad de valores: el registro $\$t0$ del banco vale ahora lo mismo que $\$s0$, mientras que el registro B contiene todavía el valor antiguo de $\$t0$. Por eso B sirve como copia de seguridad de $\$t0$.
- Al comenzar el ciclo 4, se lee de nuevo el valor de $\$t0$ en el banco, pero ahora vale $\$s0$, y ese es el valor que se escribe en B al final de este ciclo. Pero antes de que sea efectiva esta escritura, al valor de B (el antiguo de $\$t0$) le da tiempo a escribirse en el registro $\$s0$ del banco.

Si se quiere estar más seguro de que estas lecturas y escrituras de los registros A y B se van a realizar correctamente, se podría añadir una señal de habilitación de escritura para cada uno de los registros, de tal forma que solo se permitiera la escritura cuando fuera estrictamente necesaria.

El nuevo diagrama de estados es:



La secuencia de estados de la nueva instrucción sería: 0 → 1 → 10 → 11 → 0.

La nueva tabla de verdad de las salidas de la unidad de control principal sería:

Estado actual	EscrPC	EscrPCCond	IoD	LeerMem	EscrMem	EscrIR	MemaReg1	MemaReg0	FuentePC1	FuentePC0	ALUOp1	ALUOp0	SelALUB1	SelALUB0	SelALUA0	EscrReg	RegDest1	RegDest0
0000	1	0	0	1	0	1	X	X	0	0	0	0	0	1	0	0	X	X
0001	0	0	X	0	0	0	X	X	X	X	0	0	1	1	0	0	X	X
0010	0	0	X	0	0	0	X	X	X	X	0	0	1	0	1	0	X	X
0011	0	0	1	1	0	0	X	X	X	X	X	X	X	X	0	0	X	X
0100	0	0	X	0	0	0	0	1	X	X	X	X	X	X	1	0	0	0
0101	0	0	1	0	1	0	X	X	X	X	X	X	X	X	X	0	X	X
0110	0	0	X	0	0	0	X	X	X	X	1	0	0	0	1	0	X	X
0111	0	0	X	0	0	0	0	0	X	X	X	X	X	X	1	0	1	1
1000	0	1	X	0	0	0	X	X	0	1	0	1	0	0	1	0	X	X
1001	1	0	X	0	0	0	X	X	1	0	X	X	X	X	0	0	X	X
1010	0	0	X	0	0	0	1	0	X	X	X	X	X	X	1	0	0	0
1011	0	0	X	0	0	0	1	1	X	X	X	X	X	X	1	1	0	0

Microoperaciones realizadas por la instrucción, desde el punto de vista de la organización multiciclo:

- 1: $IR \leftarrow M[PC]$
 $PC \leftarrow PC + 4$
- 2: $A \leftarrow Reg[IR[25-21]]$

```

B ← Reg[IR[20-16]]
Salida ALU ← PC + extSigno16a32(IR[15-0])<<2
3: Reg[IR[20-16]] ← A
4: Reg[IR[25-21]] ← B

```

Solución alternativa:

Este ejercicio puede realizarse de muchas otras maneras, todas más o menos equivalentes. Una de las más recurrentes en el examen ha sido utilizar la ALU, sumando un nuevo valor 0, de forma que la copia de uno de los registros se guarde en **Salida ALU**. Esto es una buena idea, ya que el registro **Salida ALU** está conectado de manera natural con la entrada de Dato a escribir del banco. Además, al alejar la copia de los registros **A** y **B**, no tenemos ese problema de sincronismo que se ha mencionado antes y que, como se suele decir coloquialmente, está un poco “cogido con alfileres”.

¿Y cómo se suma 0? Se han visto dos posibles maneras. En una, se puede aprovechar el inmediato infrutilizado de la instrucción (en el formato I), de tal forma que ahora no podría tomar cualquier valor, sino que tendría que valer exactamente 0000000000000000. Así, el registro **A** (que contiene **\$s0**) se sumaría al inmediato extendido de signo, exactamente igual a como se hace en el estado 2 de las instrucciones **lw** y **sw**, con lo que se reaprovecharía este estado del diagrama.

La segunda forma que se ha visto consiste en ampliar el multiplexor de la entrada del primer operando de la ALU para que permita sumar un 0 grabado en el hardware. De esta forma, sería el registro **B** (que contiene **\$t0**) el que se sumaría al 0.

Se deja para vosotrxs pensar cómo se resolvería así este ejercicio.

10. [Ejercicio de examen] Sea un procesador MIPS con un camino de datos multiciclo (se adjunta en una hoja aparte). Al repertorio seleccionado en clase, se desea añadir una instrucción inventada, que llamaremos **swesp** (almacenamiento especial). Un ejemplo de esta instrucción en ensamblador sería el siguiente:

swesp etiqueta

La instrucción del ejemplo hace lo siguiente:

- La **etiqueta**, que corresponde a un valor de 26 bits, se transforma en una dirección completa de memoria siguiendo el mismo procedimiento que la instrucción **j**.
- Suponiendo que en el comienzo de su ejecución la instrucción **swesp** se encuentra almacenada en la dirección de memoria a la que apunta **PC**, se escribe este valor **PC** en la posición de memoria calculada a partir de la **etiqueta**. Nótese que **no** se escribe **PC+4**, sino **PC**.
- La ejecución del programa continúa con normalidad con la instrucción que va inmediatamente después de esta.

Se pide:

- a) Desde el punto de vista de la arquitectura (tal cual se vería en una tabla de resumen de instrucciones), escribir qué operación hace esta instrucción en notación de **pseudocódigo**.
- b) Justificar razonadamente qué **formato binario de instrucción** sería el más adecuado para esta instrucción y descomponerla en campos. **No** se pide expresarla en hexadecimal.
- c) Indicar cuáles son todos los **operandos** de la instrucción, así como los **modos de direccionamiento** de cada uno de ellos.
- d) Explicar **razonada y detalladamente** todos los cambios que son necesarios en el camino de datos para poder ejecutar esta instrucción. Los requisitos que debe cumplir la instrucción son los siguientes:
- El hecho de añadir esta nueva instrucción debe ser retrocompatible con todas las instrucciones estándar, que deben seguir siendo ejecutables como hasta ahora.
 - El número total de ciclos para ejecutar la nueva instrucción debe ser el mínimo posible. Para ello se podrán solapar en el mismo ciclo operaciones simultáneas que no interfieran entre sí.
 - El número total de estados de la unidad de control debe ser el mínimo posible.
 - El número total de cambios en el camino de datos debe ser el mínimo posible y también estos cambios han de ser lo más pequeños posible.
 - La prioridad de los criterios de minimización es la siguiente: número total de ciclos (más prioritario), número total de estados, número y tamaño de los cambios (menos prioritario).

Este apartado tiene tres puntos de vista interrelacionados que habrá que explicar:

- Por un lado, habrá que **explicar** las modificaciones y **dibujarlas** sobre el camino de datos, tales como añadir cables o ampliar otros existentes, añadir multiplexores o ampliar otros existentes, añadir registros o ampliar otros existentes, añadir puertas lógicas o ampliar las existentes, etc.
- Por otro, habrá que **explicar, ciclo a ciclo**, qué operaciones se realizan (desde el punto de vista de la organización, es decir, indicando los componentes concretos del camino de datos multiciclo involucrados). Para ello, se deberá usar la **notación de pseudocódigo** con el operador de asignación flecha **←** para indicar **todas las operaciones** que intervienen en dicho ciclo.

Además del pseudocódigo (que se pide para todos los ciclos):

- Si el ciclo corresponde a un estado nuevo de la unidad de control, habrá que explicar e indicar el valor de **todas las señales de control relevantes**.
- Si el ciclo corresponde a un estado existente que ahora es modificado, habrá que explicar e indicar **solo** el valor de las señales de control **nuevas o modificadas**.
- Si el ciclo corresponde a un estado existente que no se modifica, **no** se deberá explicar ni indicar el valor de las señales de control.

- Finalmente, habrá que ampliar el **diagrama de estados** de la unidad de control, así como la **tabla de verdad** de las salidas de la **unidad de control** para que incluyan los cambios realizados. Ambos se adjuntan en la hoja del diagrama del camino de datos.

Solución:

a) La notación de pseudocódigo desde el punto de vista de la arquitectura sería:

```
PC ← PC + 4
M[PC[31-28] || etiqueta<<2] ← PC - 4
```

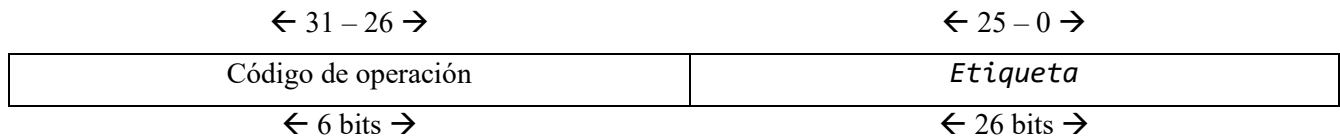
Otra manera de expresar lo mismo (aunque tiene un matiz ligeramente diferente) sería:

```
M[PC[31-28] || etiqueta<<2] ← PC
PC ← PC + 4
```

En esta segunda manera, estamos suponiendo que los bits 31-28 del **PC** no cambian al sumarle 4 al **PC**, operación que ya sabéis que se realiza muy pronto, en el ciclo 1 de la instrucción.

b) En cuanto al formato de instrucción, necesitamos codificar una pseudodirección de 26 bits que corresponde a la *etiqueta*. El único formato que lo permite es el J.

Formato J:



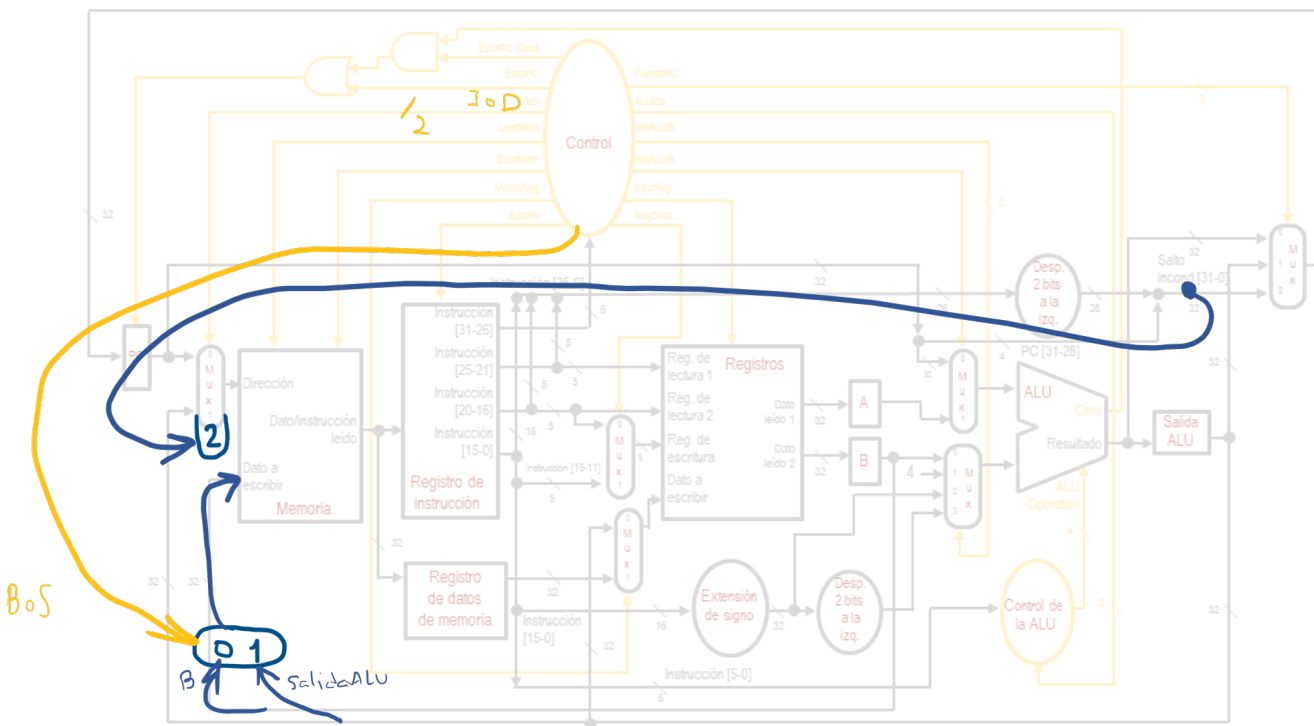
c) Los operandos de la instrucción son:

- El registro **PC**, que no solo se ve incrementado en 4 unidades, como en todas las instrucciones, sino que también sirve como operando fuente para la escritura que se hace en memoria. Su direccionamiento es implícito porque no aparece por ninguna parte en la instrucción.
- De igual forma, el 4 que se le suma al **PC**, para luego restárselo, tampoco aparece por ninguna parte en la instrucción. Por tanto, su direccionamiento también es implícito.
- La dirección de memoria en la que se produce la escritura, es decir, el valor (**PC[31-28] || etiqueta<<2**), es el tercer operando. Nótese que la *etiqueta* como tal no es un operando, sino el medio para calcular esta dirección de memoria, el verdadero operando. Su direccionamiento es pseudodirecto, porque corresponde a una pseudodirección de 26 bits que escribimos en ensamblador con la *etiqueta*.

d) Se proponen los siguientes cambios en el camino de datos:

- Ampliamos el multiplexor controlado por la señal **IoD**, que ahora pasa a ser de 2 bits. En la entrada 2, conectamos un cable que procede del punto en el que se ha calculado ($PC[31-28] \parallel \text{etiqueta} < 2$). Este punto se encuentra a la derecha del camino de datos, entre el desplazador de 2 bits a la izquierda y el multiplexor controlado por la señal **FuentePC**.
- Añadiremos un nuevo multiplexor 2 a 1 para elegir el dato a escribir de la memoria. En la entrada 0, ponemos el cable que procede del registro **B**, mientras que en la entrada 1 seleccionamos el cable que procede del registro **Salida ALU**. La señal de selección del nuevo multiplexor la llamaremos **BoS**.

El camino de datos modificado quedaría de la siguiente manera:



La ejecución ciclo a ciclo sería la siguiente:

Ciclo 1 (IF)/Estado 0: La lectura de la instrucción se realiza como en el resto de las instrucciones. Las operaciones realizadas son:

$IR \leftarrow M[PC]$ // **IR** contiene la palabra de operación de la instrucción
 $PC \leftarrow PC + 4$

Las señales de control son las habituales (ver tabla). Debemos recordar que **IoD** ahora es de 2 bits y su valor es 00.

Ciclo 2 (ID)/Estado 1: La decodificación de la instrucción se realiza como en el resto de las instrucciones. Los valores que se escriben en **A**, **B** y **Salida ALU** son irrelevantes para esta instrucción, pero se mantiene la escritura por retrocompatibilidad con el resto de las instrucciones:

$A \leftarrow \text{Reg}[\text{IR}[25-21]]$
 $B \leftarrow \text{Reg}[\text{IR}[20-16]]$
 $\text{Salida ALU} \leftarrow \text{PC} + \text{extSigno16a32}(\text{IR}[15-0]) \ll 2$

Las señales de control son las habituales (ver tabla).

Ciclo 3 (EX)/Estado 10: Este tercer ciclo corresponde a un nuevo estado. Puesto que en el ciclo 1 ya habíamos incrementado el valor del **PC**, necesitamos deshacer esa suma para poder escribir el valor que tenía el registro **PC** al comienzo de la instrucción. Por ello, la operación realizada es:

$\text{Salida ALU} \leftarrow \text{PC} - 4$

Las señales de control necesarias son las siguientes:

EscrPC = 0 (protegemos el **PC** contra escrituras indeseadas)
EscrPC Cond = 0 (no es instrucción condicional)
LeerMem = 0 (no se lee la memoria)
EscrMem = 0 (no se escribe en memoria)
EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)
ALUOp = 01 (resta en la ALU)
Se1ALUA = 0 (elegimos **PC** como minuendo)
Se1ALUB = 01 (elegimos 4 como sustraendo)
EscrReg = 0 (protegemos el banco de registros contra escrituras indeseadas)

El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 4 (MEM)/Estado 11: Este será el ciclo final de la instrucción, en el que se produce la escritura en memoria. Recordemos que la pseudodirección que se expresa con la *etiqueta* en ensamblador se encuentra en los bits 25-0 del registro **IR**. Por tanto, la operación realizada es:

$M[\text{PC}[31-28] \parallel \text{IR}[25-0] \ll 2] \leftarrow \text{Salida ALU}$

Las señales de control necesarias son las siguientes:

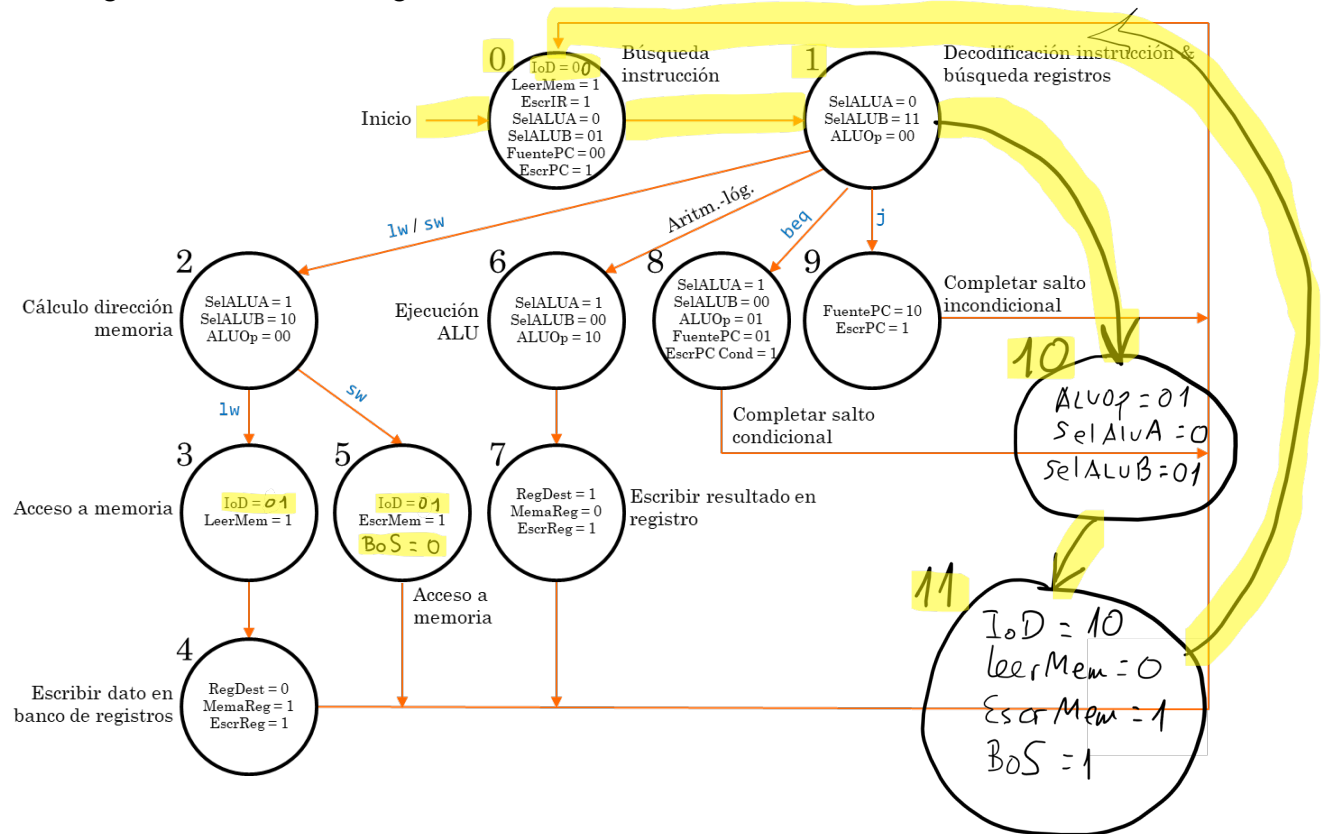
EscrPC = 0 (protegemos el **PC** contra escrituras indeseadas)
EscrPC Cond = 0 (no es instrucción condicional)
IoD = 10 (seleccionamos la dirección calculada a partir de la pseudodirección de 26 bits)
LeerMem = 0 (si se escribe en memoria, no puede leerse)
EscrMem = 1 (se escribe en memoria)
BoS = 1 (se selecciona **Salida ALU** como dato a escribir en la memoria)

$EscrIR = 0$ (protegemos el registro **IR** contra escrituras indeseadas)

$EscrReg = 0$ (protegemos el banco de registros contra escrituras indeseadas)

El resto de las señales son irrelevantes para este ciclo (X).

El nuevo diagrama de estados es el siguiente:



Hemos añadido dos nuevos estados: el 10 y el 11. También hemos hecho que la señal **IoD** sea de 2 bits, lo que nos ha obligado a modificar también varios estados que no intervienen en esta instrucción, como el 3 y el 5, además del 0. También en el estado 5 hemos tenido que indicar que **BoS** = 0 para poder realizar correctamente la escritura en **SW**.

La secuencia de estados de la nueva instrucción sería: $0 \rightarrow 1 \rightarrow 10 \rightarrow 11 \rightarrow 0$.

La nueva tabla de verdad de las salidas de la unidad de control principal sería:

Estado actual	EscrPC	EscrPCCond	Iod1	Iod0	LeerMem	EscrMem	EscrIR	MemReg	FuentePC1	FuentePC0	ALUp1	ALUp0	Se1ALUB1	Se1ALUB0	Se1ALUA	EscrReg	RegDest	Bos
0000	1	0	0	0	1	0	1	X	0	0	0	0	0	1	0	0	X	X
0001	0	0	X	X	0	0	0	X	X	X	0	0	1	1	0	0	X	X
0010	0	0	X	X	0	0	0	X	X	X	0	0	1	0	1	0	X	X
0011	0	0	0	1	1	0	0	X	X	X	X	X	X	X	X	0	X	X
0100	0	0	X	X	0	0	0	1	X	X	X	X	X	X	X	1	0	X
0101	0	0	0	1	0	1	0	X	X	X	X	X	X	X	X	0	X	0
0110	0	0	X	X	0	0	0	X	X	X	1	0	0	0	1	0	X	X
0111	0	0	X	X	0	0	0	0	X	X	X	X	X	X	X	1	1	X
1000	0	1	X	X	0	0	0	X	0	1	0	1	0	0	1	0	X	X
1001	1	0	X	X	0	0	0	X	1	0	X	X	X	X	X	0	X	X
1010	0	0	X	X	0	0	0	X	X	X	0	1	0	1	0	0	X	X
1011	0	0	1	0	0	1	0	X	X	X	X	X	X	X	X	0	X	1

Microoperaciones realizadas por la instrucción, desde el punto de vista de la organización multiciclo:

- 1: $IR \leftarrow M[PC]$
 $PC \leftarrow PC + 4$
- 2: $A \leftarrow Reg[IR[25-21]]$
 $B \leftarrow Reg[IR[20-16]]$
 $Salida\ ALU \leftarrow PC + extSigno16a32(IR[15-0]) \ll 2$
- 3: $Salida\ ALU \leftarrow PC - 4$
- 4: $M[PC[31-28] || (IR[25-0] \ll 2)] \leftarrow Salida\ ALU$

11. [Ejercicio de examen] Sea un procesador MIPS con un camino de datos multiciclo estándar. Se desea añadir una nueva instrucción al repertorio, denominada **add3**, que realice la suma de tres registros y guarde el resultado en un cuarto registro. Un ejemplo de esta instrucción en ensamblador sería el siguiente:

add3 \$s0, \$s1, \$s2, \$s3

donde el registro **\$s0** sería el destino y los otros tres serían los fuentes.

Se pide:

- a) Desde el punto de vista de la **arquitectura** (tal cual se vería en una tabla de resumen de instrucciones), escribir qué operación hace esta instrucción en notación de **pseudocódigo**.
- b) Justificar qué **formato binario** de instrucción sería el más adecuado para esta instrucción y descomponerla en campos. Habrá que indicar también **todos los operandos** de la instrucción, así como los **modos de direccionamiento** de cada uno de ellos.
- c) Escribir la instrucción que se pone en el ejemplo en **formato hexadecimal**.
- d) Explicar razonada y detalladamente **todos los cambios** que son necesarios en el camino de datos para poder ejecutar esta instrucción. Los requisitos que debe cumplir la instrucción son los siguientes:

- El hecho de añadir esta nueva instrucción debe ser retrocompatible con todas las instrucciones estándar, que deben seguir siendo ejecutables como hasta ahora.
- El número total de ciclos para ejecutar la nueva instrucción debe ser el mínimo posible. Para ello se podrán solapar en el mismo ciclo operaciones simultáneas que no interfieran entre sí.
- El número total de estados de la unidad de control debe ser el mínimo posible.
- El número total de cambios en el camino de datos debe ser el mínimo posible y también estos cambios han de ser lo más pequeños que sea posible.
- No se podrá modificar el banco de registros.

Este apartado tiene tres puntos de vista interrelacionados que habrá que explicar:

- Por un lado, habrá que **explicar** las modificaciones y **dibujarlas** sobre el camino de datos, tales como añadir cables o ampliar otros existentes, añadir multiplexores o ampliar otros existentes, añadir registros o ampliar otros existentes, añadir puertas lógicas o ampliar las existentes, etc.
- Por otro, habrá que **explicar, ciclo a ciclo**, qué operaciones se realizan (desde el punto de vista de la **organización**, es decir, indicando los componentes concretos del camino de datos involucrados). Para ello, se deberá usar la notación de **pseudocódigo** con el operador de asignación flecha $\leftarrow$ para indicar todas las operaciones que intervienen en dicho ciclo. Además del pseudocódigo:
 - Si el ciclo corresponde a un estado nuevo de la unidad de control, habrá que explicar e indicar el valor de **todas las señales de control relevantes**.
 - Si el ciclo corresponde a un estado existente que ahora es modificado, habrá que explicar e indicar **solo** el valor de las señales de control **nuevas o modificadas**.
 - Si el ciclo corresponde a un estado existente que no se modifica, **no** será necesario explicar ni indicar el valor de las señales de control.
- Finalmente, habrá que ampliar el **diagrama de estados** de la unidad de control, así como la **tabla de verdad** de las salidas de la **unidad de control** para que incluyan los cambios realizados. Ambos se adjuntan con el enunciado.

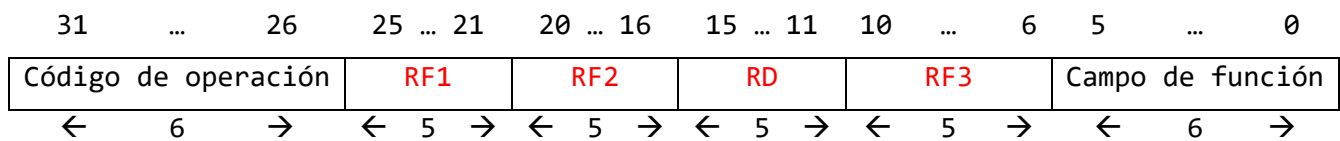
Solución:

a) Esta instrucción realiza las siguientes operaciones **desde el punto de vista de la arquitectura**:

```
PC ← PC + 4
$s0 ← $s1 + $s2 + $s3
```

b) De los tres formatos de instrucción de MIPS, el que mejor encaja es el R, que especifica tres registros codificando cada uno con 5 bits. La nueva instrucción usa un cuarto registro, que podría colocarse en el hueco dedicado a las instrucciones de desplazamiento (shamt), que también es de 5 bits.

Por lo tanto, el nuevo formato, que llamaremos R2, sería el siguiente:



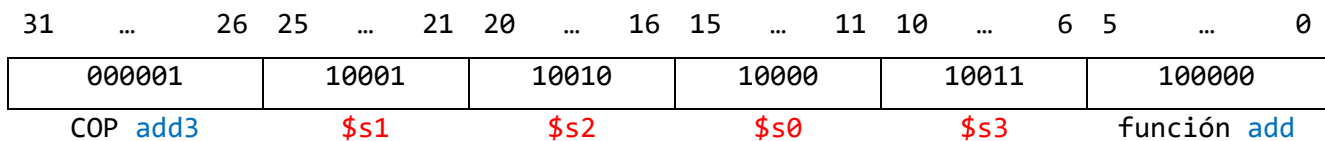
Donde:

- Código de operación (31-26): identifica esta instrucción. Para diferenciarla del resto de instrucciones aritmético-lógicas de formato R (el normal), sería recomendable asignarle un código único. De la tabla de códigos de operación del listado resumen de instrucciones elegimos uno sin usar, por ejemplo: 000001.
- RF1 (25-21): es el registro fuente 1. En este caso es \$s1 (registro 17), que se codifica en binario puro como 10001.
- RF2 (20-16): es el registro fuente 2, que sería \$s2 (registro 18). Luego vale 10010.
- RD (15-11): es el registro destino que almacenará el resultado. Lo mantenemos en esta posición por retrocompatibilidad. Sería el registro \$s0 (16), luego vale 10000.
- RF3 (10-6): es el registro fuente 3, donde antes se codificaba el valor de shamt. En el ejemplo sería el registro \$s3 (19). Luego vale 10011.
- Campo de función: es el campo que usan las instrucciones aritmético-lógicas de formato R para indicar la operación concreta. Una posibilidad sería colocar el valor que tiene la instrucción normal add (100000). Ambas se diferenciarían por el código de operación. Hay otras maneras de hacerlo.

Los cinco operandos de add3 son:

- Visibles:
 - Fuente: registros \$s1, \$s2 y \$s3
 - Destino: registro \$s0
 - Cada uno de estos registros usa un modo de direccionamiento directo a registro, es decir, se codifican en la instrucción en binario puro con 5 bits (número de 0 a 31).
- Invisibles:
 - El contador de programa PC, al que se le suma 4 para que apunte a la siguiente instrucción ($PC \leftarrow PC + 4$, donde el 4 está grabado en el hardware). Usa un modo de direccionamiento implícito.

c) La instrucción en código máquina sería:



Para pasar la instrucción a hexadecimal, agrupamos los bits de 4 en 4, con lo que resulta:

000001 10001 10010 10000 10011 100000 = 0000 0110 0011 0010 1000 0100 1110 0000 = 0x063284E0

La solución de este apartado depende del código de operación y del campo de función elegidos en el apartado anterior, con lo que no hay solución única.

d) Los cambios propuestos para el camino de datos son los siguientes:

Por un lado, hemos añadido un nuevo multiplexor controlado por la señal **RegLect2** (1 bit):

- Si vale 0, se selecciona el registro codificado en los bits 20-16 de la instrucción, como hasta ahora.
- Si vale 1, se selecciona el registro codificado en los bits 10-6, el tercer registro fuente de **add3**.

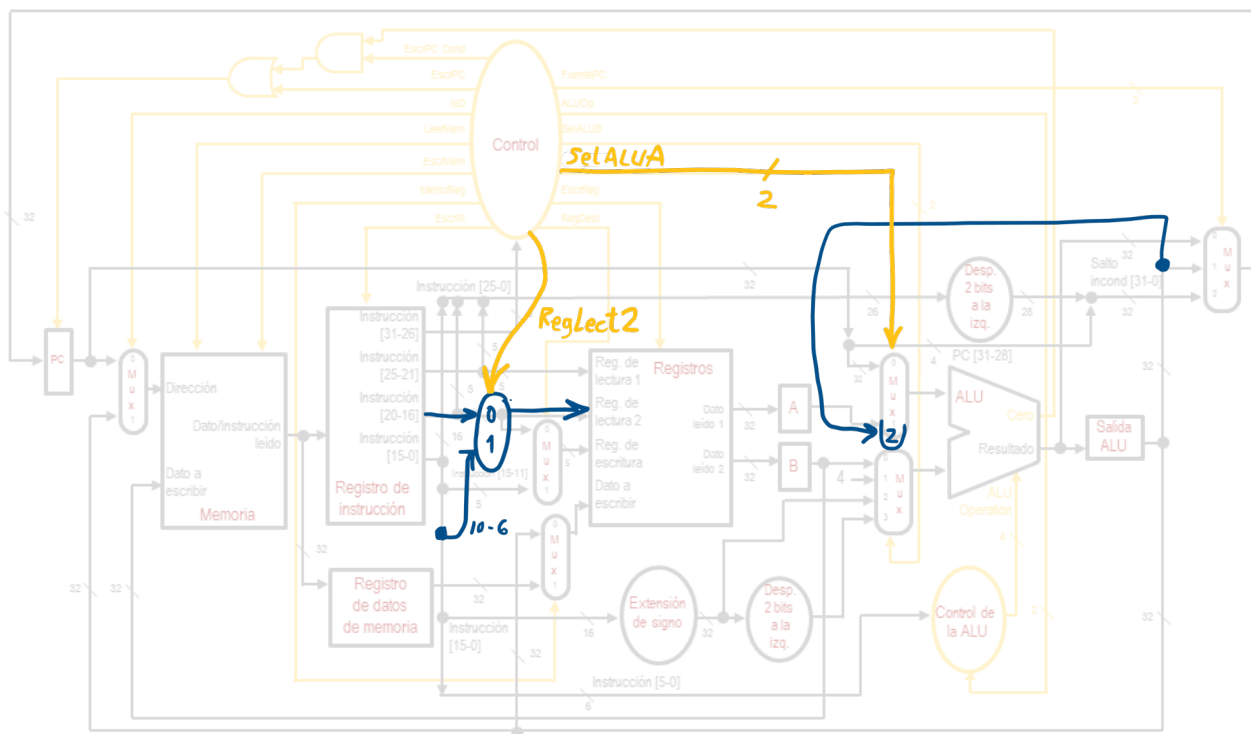
La salida de este multiplexor se proporciona a la entrada del Registro de Lectura 2 del banco de registros.

Por otra parte, ampliamos el multiplexor que selecciona el primer operando de la ALU, para lo cual la señal **SelALUA** pasa a ser de 2 bits (**SelALUA1 SelALUA0**). Las entradas del multiplexor son:

- Entrada 0 (00): **PC**
- Entrada 1 (01): **A**
- Entrada 2 (10): **Salida ALU**
- Entrada 3 (11): libre

Hemos tenido que añadir un cable que conecte la salida del registro **Salida ALU** a la entrada 2 del multiplexor.

El camino de datos modificado quedaría de la siguiente manera:



La ejecución ciclo a ciclo de la nueva instrucción sería la siguiente:

Ciclo 1 (IF)/Estado 0: Este ciclo está sin modificar. La búsqueda de la instrucción se realiza como en el resto de las instrucciones. Las señales de control son las habituales en el estado 0.

$$\begin{aligned} \text{IR} &\leftarrow \text{M}[\text{PC}] \\ \text{PC} &\leftarrow \text{PC} + 4 \end{aligned}$$

Las señales de control son las habituales para este estado.

Ciclo 2 (ID)/Estado 1: La decodificación de la instrucción se realiza como en el resto de las instrucciones.

$$\begin{aligned} \text{A} &\leftarrow \text{Reg}[\text{IR}[25-21]] \\ \text{B} &\leftarrow \text{Reg}[\text{IR}[20-16]] \\ \text{Salida ALU} &\leftarrow \text{PC} + \text{extSigno16a32}(\text{IR}[15-0]) \ll 2 \end{aligned}$$

A contendrá el valor del registro base **\$s1**, mientras que **B** contendrá el valor del registro destino **\$s0**. El cálculo de **Salida ALU** representa el hipotético destino de un salto condicional y esta instrucción no es de ese tipo, con lo que este valor no nos sirve para nada en este caso.

Las señales de control son las habituales para este estado, salvo por la nueva señal **RegLect2**, que vale 0, y la señal **SelALUA**, que es de 2 bits y vale 00.

El valor almacenado en **Salida ALU** es el destino de un hipotético salto condicional y no sirve para nada en esta instrucción. Sí son de provecho los valores almacenados en los registros **A** y **B**, que, para la instrucción del ejemplo, serían los de **\$s1** y **\$s2**, respectivamente.

Durante este ciclo, se proporciona el código de operación de la instrucción a la unidad de control. Al final del ciclo, ya sabemos que se trata de la instrucción **add3**.

Ciclo 3 (EX1)/Estado 6: En este primer ciclo de ejecución se suman los valores de **A** y **B** y el resultado se almacena en **Salida ALU**:

$$\text{Salida ALU} \leftarrow \text{A} + \text{B}$$

Para ello, **SelALUA** debe valer 01, mientras que **SelALUB** vale 00.

Hasta aquí esto es igual que en el estado 6 normal. Vamos a modificar este estado para leer también el tercer registro fuente y guardarlo en **B**, haciendo **RegLect2** = 1:

$$\text{B} \leftarrow \text{Reg}[\text{IR}[10-6]]$$

Como la escritura se hace al final del ciclo, se toma el valor correcto de **B** en la ALU. Los valores al final del ciclo son:

$$\begin{aligned} \text{Salida ALU} &\leftarrow \$s1 + \$s2 \\ \text{B} &\leftarrow \$s3 \end{aligned}$$

Para que la ALU haga la suma, tenemos **ALUOp** = 10. Este valor obliga a la unidad de control de la ALU a ver el campo de función en **IR[5-0]**, que vale 100000. Por la tabla de verdad de la unidad de control de la ALU (que no hay que modificar en este ejercicio), la ALU recibe la orden de sumar ambos operandos.

Nótese que sigue garantizándose la retrocompatibilidad para la ejecución de las instrucciones aritmético-lógicas habituales (las de 3 registros), que seguirían funcionando como hasta ahora. La única diferencia sería el valor del registro **B** al final del ciclo, ya que antes era $\text{Reg}[\text{IR}[20-16]]$ y ahora será $\text{Reg}[\text{IR}[10-6]]$.

Las señales de control son las habituales para este estado.

Ciclo 4 (EX2)/Estado 10: La instrucción **add3** tendría un segundo ciclo de ejecución, que esta vez sí tendría que ser un nuevo estado (el 10). La operación realizada en la ALU sería:

$$\text{Salida ALU} \leftarrow \text{Salida ALU} + \text{B}$$

Para ello, necesitamos que **SelALUA** valga **10** (para seleccionar **Salida ALU**), que **SelALUB** valga **00**, que **ALUOp** sea **10** y que el campo de función situado en $\text{IR}[5-0]$ valga **100000** (operación sumar).

Al terminar este ciclo, el valor almacenado es:

$$\text{Salida ALU} \leftarrow \$s1 + \$s2 + \$s3$$

Este es el valor que hay que guardar en el registro destino (**\$s0**).

Señales de control:

EscrPC = 0 (protegemos el **PC** contra escrituras indeseadas)

EscrPC Cond = 0 (no es instrucción condicional)

LeerMem = 0 (no hay que leer la memoria)

EscrMem = 0 (protegemos la memoria contra escrituras indeseadas)

EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)

ALUOp = 10 (sumar, ya que el campo de función vale 100000)

SelALUB = 00 (seleccionamos **B** como segundo operando de la ALU)

SelALUA = 10 (seleccionamos **Salida ALU** como primer operando de la ALU)

EscrReg = 0 (protegemos el banco de registros contra escrituras indeseadas)

El resto de las señales son irrelevantes para este ciclo (X).

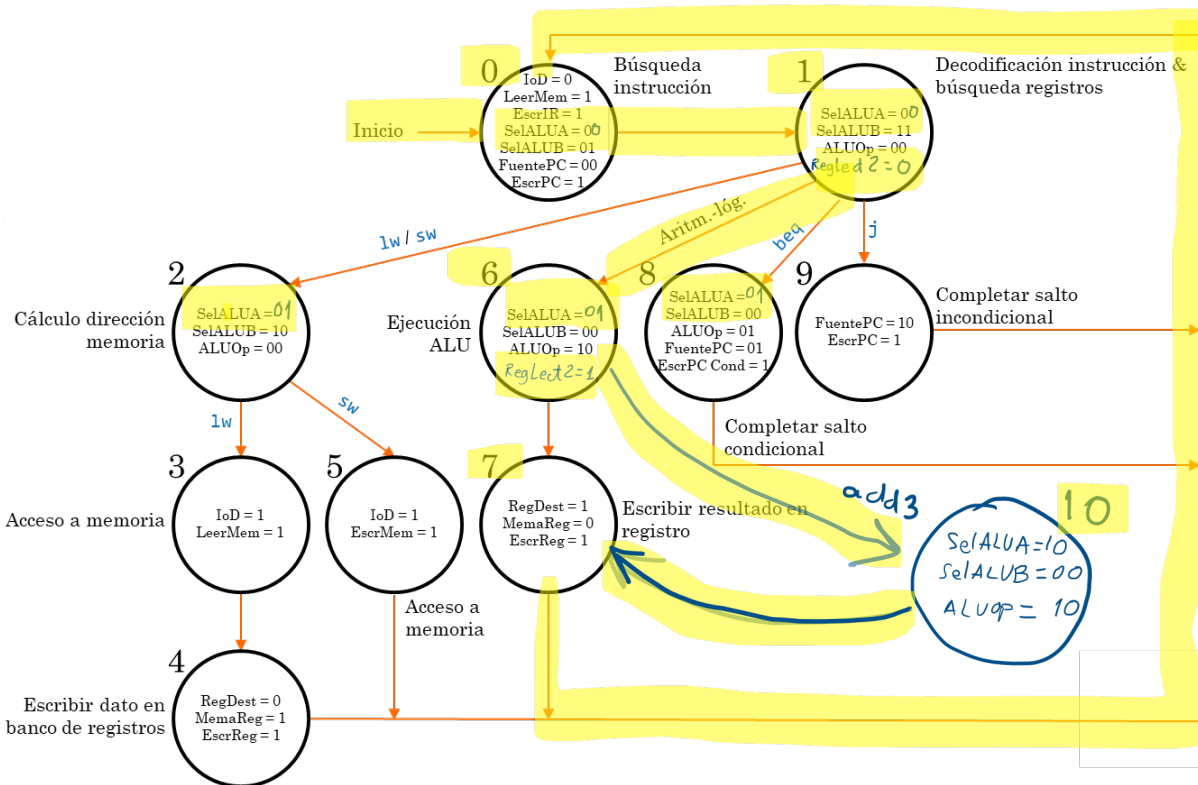
Ciclo 5 (WB)/Estado 7: En este ciclo, el último, se guarda el resultado en el banco de registros.

$$\text{Reg}[\text{IR}[15-11]] \leftarrow \text{Salida ALU}$$

Las señales de control son las habituales para este estado.

Por tanto, hemos ejecutado la nueva instrucción **add3** en 5 ciclos (no en 4, como el resto de aritmético-lógicas). Para ello, hemos añadido solo un nuevo estado a la unidad de control, que hemos llamado estado 10 (1010).

El nuevo diagrama de estados sería el siguiente:



Hemos añadido un nuevo estado 10 después del 6 y antes del 7. También hemos hecho que la señal **SelALUA** sea de 2 bits, lo que nos ha obligado a modificar también varios estados que no intervienen en esta instrucción, como el 2 y el 8. Por último, hemos añadido la señal **RegLect2**, que interviene en los estados 1 y 6.

La secuencia de estados de la nueva instrucción sería: $0 \rightarrow 1 \rightarrow 6 \rightarrow 10 \rightarrow 7 \rightarrow 0$.

La nueva tabla de verdad de las salidas de la unidad de control principal sería:

Estado actual	EscrPC	EscrPCCond	IoD	LeerMem	EscrMem	EscrIR	MemReg	FuentePC1	FuentePC0	ALUOp1	ALUOp0	SelALUB1	SelALUB0	SelALUA1	SelALUA0	EscrReg	RegDest	RegLect2
0000	1	0	0	1	0	1	X	0	0	0	0	0	1	0	0	0	X	X
0001	0	0	X	0	0	0	X	X	X	0	0	1	1	0	0	0	X	0
0010	0	0	X	0	0	0	X	X	X	0	0	1	0	0	1	0	X	X
0011	0	0	1	1	0	0	X	X	X	X	X	X	X	X	X	0	X	X
0100	0	0	X	0	0	0	1	X	X	X	X	X	X	X	X	1	0	X
0101	0	0	1	0	1	0	X	X	X	X	X	X	X	X	X	0	X	X
0110	0	0	X	0	0	0	X	X	X	1	0	0	0	0	1	0	X	1
0111	0	0	X	0	0	0	0	X	X	X	X	X	X	X	X	1	1	X
1000	0	1	X	0	0	0	X	0	1	0	1	0	0	0	1	0	X	X
1001	1	0	X	0	0	0	X	1	0	X	X	X	X	X	X	0	X	X
1010	0	0	X	0	0	0	X	X	X	1	0	0	0	1	0	0	X	X

Hemos añadido la fila del nuevo estado 10, así como dos nuevas columnas: la de **SelALUA1** y la de **RegLect2**. Por su parte, la señal anteriormente denominada **SelALUA** pasa a ser **SelALUA0**.

Para el nuevo estado, las señales **EscrPC**, **EscrPC Cond**, **LeerMem**, **EscrMem**, **EscrIR** y **EscrReg** se han mantenido a 0 para evitar que se modifique la memoria, el registro de instrucción o el banco de registros o que se produzca un salto de instrucción indeseado. El resto de las señales de control, sin contar **ALUOp**, **SelALUB** y **SelALUA**, se han marcado a X al no intervenir en la instrucción y ser su valor irrelevante.

Microoperaciones realizadas por la instrucción, desde el punto de vista de la organización multiciclo:

```

1: IR ← M[PC]
   PC ← PC + 4
2: A ← Reg[IR[25-21]]
   B ← Reg[IR[20-16]]
   Salida ALU ← PC + extSigno16a32(IR[15-0])<<2
3: Salida ALU ← A + B
   B ← Reg[IR[10-6]]
4: Salida ALU ← Salida ALU + B
5: Reg[IR[15-11]] ← Salida ALU

```

12. [Ejercicio de examen] Se desea modificar el camino de datos multiciclo del procesador MIPS para añadir un nuevo modo de direccionamiento para los operandos de las instrucciones de carga `lw`, en particular el llamado “direccionamiento indirecto a registro con postincremento”. Disponer de este direccionamiento sería muy útil para los programas que incluyen un bucle que recorre correlativamente todos los elementos de un vector, desde el primero hasta el último.

Supongamos que el registro `$s0` apunta al primer elemento del vector. La notación en ensamblador de este direccionamiento sería `($s0)+`. La primera parte de la instrucción se realizaría como una carga normal, aunque con desplazamiento cero. Sin embargo, se le añadiría el siguiente comportamiento a la instrucción: se incrementaría automáticamente el valor del registro según el tamaño de palabra, de tal manera que el registro quede apuntando al siguiente elemento del vector. El ejemplo paradigmático sería el siguiente:

`lw $s1, ($s0)+`

Se pide:

- a) Desde el punto de vista de la **arquitectura** (tal cual se vería en una tabla de resumen de instrucciones), escribir qué operación hace esta instrucción en notación de **pseudocódigo**.
- b) Justificar qué **formato binario** de instrucción sería el más adecuado para esta instrucción y descomponerla en campos. Habrá que indicar también **todos los operandos** de la instrucción, así como los **modos de direccionamiento** de cada uno de ellos.
- c) La instrucción más parecida a esta del MIPS estándar es `lw $s1, 0($s0)`. Sabiendo que tiene por código de operación el valor `100011`, ¿podría usarse también este código de operación para la nueva instrucción? Razonar la respuesta.
- d) Explicar razonada y detalladamente **todos los cambios** que son necesarios en el camino de datos para poder ejecutar esta instrucción. Los requisitos que debe cumplir la instrucción son los siguientes:
 - El hecho de añadir esta nueva instrucción debe ser retrocompatible con todas las instrucciones estándar, que deben seguir siendo ejecutables como hasta ahora.
 - El número total de ciclos para ejecutar la nueva instrucción debe ser el mínimo posible. Para ello se podrán solapar en el mismo ciclo operaciones simultáneas que no interfieran entre sí.
 - El número total de estados de la unidad de control debe ser el mínimo posible.

- El número total de cambios en el camino de datos debe ser el mínimo posible y también estos cambios han de ser lo más pequeños posible.

Este apartado tiene tres puntos de vista interrelacionados que habrá que explicar:

- Por un lado, habrá que **explicar** las modificaciones y **dibujarlas** sobre el camino de datos, tales como añadir cables o ampliar otros existentes, añadir multiplexores o ampliar otros existentes, añadir registros o ampliar otros existentes, añadir puertas lógicas o ampliar las existentes, etc.
- Por otro, habrá que **explicar, ciclo a ciclo**, qué operaciones se realizan (desde el punto de vista de la **organización**, es decir, indicando los componentes concretos del camino de datos involucrados). Para ello, se deberá usar la notación de **pseudocódigo** con el operador de asignación flecha $\leftarrow$ para indicar todas las operaciones que intervienen en dicho ciclo. Además del pseudocódigo:
 - Si el ciclo corresponde a un estado nuevo de la unidad de control, habrá que explicar e indicar el valor de **todas las señales de control relevantes**.
 - Si el ciclo corresponde a un estado existente que ahora es modificado, habrá que explicar e indicar **solo** el valor de las señales de control **nuevas o modificadas**.
 - Si el ciclo corresponde a un estado existente que no se modifica, **no** será necesario explicar ni indicar el valor de las señales de control.
- Finalmente, habrá que ampliar el **diagrama de estados** de la unidad de control, así como la **tabla de verdad** de las salidas de la **unidad de control** para que incluyan los cambios realizados. Ambos se adjuntan con el enunciado.

Solución:

a) Esta instrucción realiza las siguientes operaciones **desde el punto de vista de la arquitectura**:

```
PC ← PC + 4
$s1 ← M[$s0]
$s0 ← $s0 + 4
```

b) Hay al menos dos maneras de resolver este ejercicio, una con menos cambios en el camino de datos, pero más ciclos, y otra con más cambios, pero menos ciclos. Explicaremos esta última.

El formato de instrucción es el I, ya que utilizamos dos registros y un inmediato. En esta instrucción no es necesario ningún inmediato, ya que al valor del registro base no hay que sumarle ningún inmediato. Por tanto, los 16 bits de la instrucción podrían contener cualquier valor, porque se van a ignorar.

31	...	26	25	...	21	20	...	16	15	...	0
Código de operación				Registro base				Registro destino			
←	6	→	←	5	→	←	5	→	←	16	→

Los operandos de esta instrucción son cuatro:

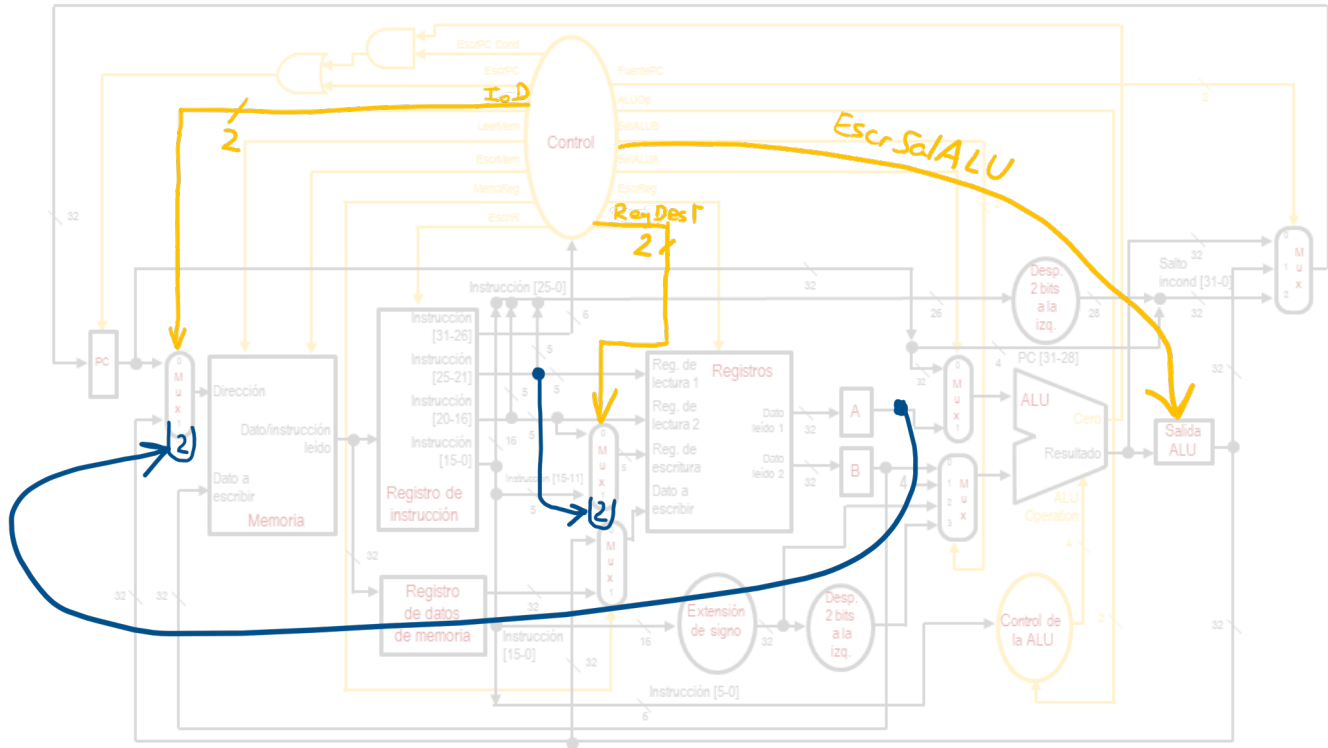
- Un operando fuente que hay que leer de la memoria, al que se hace referencia mediante un registro base (\$s0). El modo de direccionamiento sería de tipo indirecto a registro.
- Un registro destino (\$s1), en el que se escribirá el dato leído, y al que se hace referencia con un modo de direccionamiento directo a registro.
- El propio registro base (\$s0) verá incrementado su valor en el tamaño de palabra (4) al terminar la operación. Esta es la parte del postincremento del modo de direccionamiento.
- El registro PC también es operando porque se actualiza al valor PC + 4. Este operando está oculto en la instrucción, ya que esta suma se codifica en el código de operación, con lo que el modo de direccionamiento es el implícito.

c) La nueva instrucción no puede compartir el mismo código de operación que la instrucción lw normal, porque entonces serían indistinguibles para la unidad de control. De la hoja resumen de las instrucciones MIPS podría escogerse una combinación de 6 bits que estuviera sin usar para asignarla a la nueva instrucción.

d) Las modificaciones que se proponen son las siguientes:

- El registro base \$s0 está codificado en los bits 25-21 de la instrucción. Necesitamos leer este registro en el banco, lo cual ya se puede hacer porque está conectado a la entrada 1 del mismo. Pero también hay que modificar su valor para hacer el postincremento. Por ello, debemos conectarlo también a la entrada de Registro de escritura del banco. Para ello, necesitamos ampliar el multiplexor controlado por la señal RegDest, que pasaría a tener 3 entradas, y por tanto la señal sería ahora de 2 bits (RegDest1 RegDest0).
- Colocamos un nuevo cable que conecte el registro A (que almacena el valor del registro base \$s0) con el multiplexor que hay en la entrada de dirección de la memoria, que pasa a tener 3 entradas. La señal IoD pasa a ser de 2 bits (IoD1 IoD0).
- El registro Salida ALU necesita una señal que habilite o deshabilite la escritura, que llamaremos EscrSalALU. Nótese que hasta ahora siempre se producía una escritura en dicho registro, con lo que no podía almacenarse un valor que fuera necesario más de un ciclo de reloj.

El camino de datos modificado quedaría de la siguiente manera:



La ejecución ciclo a ciclo de la nueva instrucción sería la siguiente:

Ciclo 1 (IF)/Estado 0: Este ciclo está sin modificar. La búsqueda de la instrucción se realiza como en el resto de las instrucciones. Las señales de control son las habituales en el estado 0, salvo por el hecho de que ahora **IoD** es de 2 bits.

$IR \leftarrow M[PC]$
 $PC \leftarrow PC + 4$

Las señales de control son las habituales para este estado.

Ciclo 2 (ID)/Estado 1: La decodificación de la instrucción se realiza como en el resto de las instrucciones, salvo por la nueva señal **EscrSalALU**.

$A \leftarrow \text{Reg}[IR[25-21]]$
 $B \leftarrow \text{Reg}[IR[20-16]]$
 $\text{Salida ALU} \leftarrow PC + \text{extSigno16a32}(IR[15-0]) \ll 2$

Los valores de los registros **B** y **Salida ALU** calculados en este ciclo son irrelevantes para esta instrucción, pero aun así se calculan por retrocompatibilidad con el resto de las instrucciones.

Las señales de control son las habituales para este estado.

Ciclo 3 (MEM)/Estado 10: En este nuevo ciclo se produce la lectura en memoria de la dirección a la que apunta el registro base, contenido en A. Mientras se produce la lectura, aprovechamos también para hacer el postincremento en la ALU.

$MDR \leftarrow M[A]$
 $Salida\ ALU \leftarrow A + 4$

Señales de control:

$EscrPC = 0$ (protegemos el PC contra escrituras indeseadas)
 $EscrPC\ Cond = 0$ (no es instrucción condicional)
 $IoD = 10$ (seleccionamos A como dirección de memoria)
 $LeerMem = 1$ (se lee la memoria)
 $EscrMem = 0$ (si se lee la memoria, no se puede escribir en ella)
 $EscrIR = 0$ (protegemos el registro IR contra escrituras indeseadas)
 $ALUOp = 00$ (sumar)
 $Se1ALUB = 01$ (seleccionamos 4 como segundo operando de la ALU)
 $Se1ALUA = 1$ (seleccionamos A como segundo operando de la ALU)
 $EscrReg = 0$ (protegemos el banco de registros contra escrituras indeseadas)
 $EscrSalALU = 1$ (escribimos en Salida ALU el resultado que nos da la ALU)

El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 4 (WB)/Estado 4: En este ciclo se produce la carga, es decir, el valor leído en memoria se guarda en el registro destino. Es el estado 4 del diagrama de estados (coincide con una carga normal). Por otro lado, necesitamos que el valor almacenado en el registro Salida ALU no se pierda porque lo necesitamos en el siguiente ciclo. Por eso, la señal $EscrSalALU$ está deshabilitada.

$Reg[IR[20-16]] \leftarrow MDR$

Las señales de control son las habituales para este estado.

Ciclo 5 (WB)/Estado 11: En este último ciclo, se guarda el valor incrementado en el registro base. Para ello, se produce la escritura en el banco de registros.

$Reg[IR[25-21]] \leftarrow Salida\ ALU$

Señales de control:

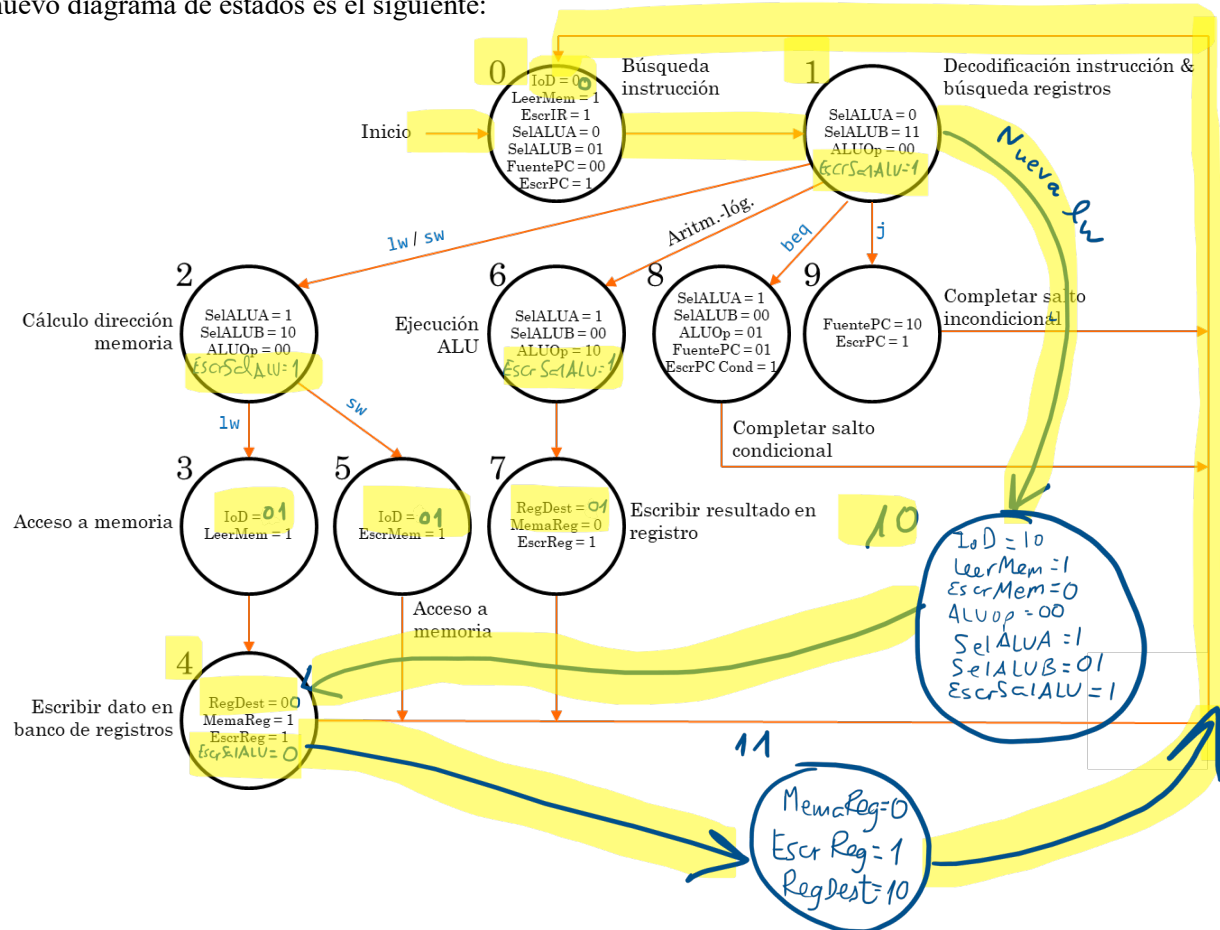
$EscrPC = 0$ (protegemos el PC contra escrituras indeseadas)
 $EscrPC\ Cond = 0$ (no es instrucción condicional)
 $LeerMem = 0$ (no hay que leer la memoria)
 $EscrMem = 0$ (protegemos la memoria contra escrituras indeseadas)
 $EscrIR = 0$ (protegemos el registro IR contra escrituras indeseadas)
 $MemaReg = 0$ (seleccionamos Salida ALU como valor que hay que escribir en el banco)

EscrReg = 1 (activamos la escritura en el banco de registros)

RegDest = 10 (seleccionamos **IR**[25-21] como el registro en el que hay que escribir)

El resto de las señales son irrelevantes para este ciclo (X).

El nuevo diagrama de estados es el siguiente:



Hemos añadido dos nuevos estados: el 10 y el 11 y hemos reaprovechado el 4. También hemos hecho que las señales **IoD** y **RegDest** sean de 2 bits, lo que nos ha obligado a modificar también varios estados, como el 0, 3, el 4, el 5 y el 7. Al introducir la nueva señal **EscrS<1>ALU**, debemos modificar asimismo los estados 1, 2 y 4.

La secuencia de estados de la nueva instrucción sería: $0 \rightarrow 1 \rightarrow 10 \rightarrow 4 \rightarrow 11 \rightarrow 0$.

La nueva tabla de verdad de las salidas de la unidad de control principal sería:

Estado actual	EscrPC	EscrPCCond	IoD1	IoD0	LeerMem	EscrMem	EscrIR	MemReg	FuentePC1	FuentePC0	ALUOp1	ALUOp0	Se1ALUB1	Se1ALUB0	Se1ALUA	EscrReg	RegDest1	RegDest0	EscrSalALU
0000	1	0	0	0	1	0	1	X	0	0	0	0	0	1	0	0	X	X	X
0001	0	0	X	X	0	0	0	X	X	X	0	0	1	1	0	0	X	X	1
0010	0	0	X	X	0	0	0	X	X	X	0	0	1	0	1	0	X	X	1
0011	0	0	0	1	1	0	0	X	X	X	X	X	X	X	X	0	X	X	X
0100	0	0	X	X	0	0	0	1	X	X	X	X	X	X	X	1	0	0	0
0101	0	0	0	1	0	1	0	X	X	X	X	X	X	X	X	0	X	X	X
0110	0	0	X	X	0	0	0	X	X	X	1	0	0	0	1	0	X	X	1
0111	0	0	X	X	0	0	0	0	X	X	X	X	X	X	X	1	0	1	X
1000	0	1	X	X	0	0	0	X	0	1	0	1	0	0	1	0	X	X	X
1001	1	0	X	X	0	0	0	X	1	0	X	X	X	X	X	0	X	X	X
1010	0	0	1	0	1	0	0	X	X	X	0	0	0	1	1	0	X	X	1
1011	0	0	X	X	0	0	0	0	X	X	X	X	X	X	X	0	1	0	X

La antigua señal **IoD** ahora corresponde a **IoD0**, al igual que la antigua **RegDest** ahora es **RegDest0**. Hemos añadido las filas correspondiente a los estados 10 (**1010**) y 11 (**1011**), así como las columnas correspondientes a las nuevas señales **IoD1** y **RegDest1** y **EscrSalALU**.

Para los nuevos estados, las señales **EscrPC**, **EscrPC Cond**, **LeerMem**, **EscrMem**, **EscrIR** y **EscrReg** se han mantenido a 0 para evitar que se modifique la memoria, el registro de instrucción o el banco de registros, salvo que en el estado 10 se activa **LeerMem** y en el 11 se activa **EscrReg**. El resto de las señales de control se han marcado a X al no intervenir en la instrucción y ser su valor irrelevante.

Microoperaciones realizadas por la instrucción, **desde el punto de vista de la organización multiciclo**:

```

1: IR ← M[PC]
   PC ← PC + 4
2: A ← Reg[IR[25-21]]
   B ← Reg[IR[20-16]]
   Salida ALU ← PC + extSigno16a32(IR[15-0])<<2
3: MDR ← M[A]
   Salida ALU ← A + 4
4: Reg[IR[20-16]] ← MDR
5: Reg[IR[25-21]] ← Salida ALU

```

13. [Ejercicio de examen] Se desea añadir una nueva instrucción al repertorio del procesador MIPS en su implementación multiciclo. Se ha bautizado la instrucción como **inc** y tiene la siguiente sintaxis en ensamblador, donde *dirección* es un inmediato de 26 bits codificado en la instrucción:

inc *dirección*

El funcionamiento de la instrucción es el siguiente:

- Tras leerse la instrucción de la memoria, se toman los 26 bits de *dirección* y se transforman en 32 bits con un procedimiento similar al de la instrucción **j**.
- La dirección de 32 bits obtenida se lee de la memoria.
- La palabra leída en dicha dirección de memoria se incrementa en 4 y el resultado se almacena en la misma dirección de memoria.

Se pide:

- a) Desde el punto de vista de la **arquitectura** (tal cual se vería en una tabla de resumen de instrucciones), escribir qué operación hace esta instrucción en notación de **pseudocódigo**.
- b) Justificar qué **formato binario** de instrucción sería el más adecuado para esta instrucción y descomponerla en campos. Habrá que indicar también **todos los operandos** de la instrucción, así como los **modos de direccionamiento** de cada uno de ellos.
- c) Explicar razonada y detalladamente **todos los cambios** que son necesarios en el camino de datos para poder ejecutar esta instrucción. Los requisitos que debe cumplir la instrucción son los siguientes:
 - El hecho de añadir esta nueva instrucción debe ser retrocompatible con todas las instrucciones estándar, que deben seguir siendo ejecutables como hasta ahora.
 - El número total de ciclos para ejecutar la nueva instrucción debe ser el mínimo posible. Para ello se podrán solapar en el mismo ciclo operaciones simultáneas que no interfieran entre sí.
 - El número total de estados de la unidad de control debe ser el mínimo posible.
 - El número total de cambios en el camino de datos debe ser el mínimo posible y también estos cambios han de ser lo más pequeños posible.

Este apartado tiene tres puntos de vista interrelacionados que habrá que explicar:

- Por un lado, habrá que **explicar** las modificaciones y **dibujarlas** sobre el camino de datos, tales como añadir cables o ampliar otros existentes, añadir multiplexores o ampliar otros existentes, añadir registros o ampliar otros existentes, añadir puertas lógicas o ampliar las existentes, etc.
- Por otro, habrá que **explicar, ciclo a ciclo**, qué operaciones se realizan (desde el punto de vista de la **organización**, es decir, indicando los componentes concretos del camino de datos involucrados). Para ello, se deberá usar la notación de **pseudocódigo** con el operador de asignación flecha $\leftarrow$ para indicar todas las operaciones que intervienen en dicho ciclo. Además del pseudocódigo:
 - Si el ciclo corresponde a un estado nuevo de la unidad de control, habrá que explicar e indicar el valor de **todas las señales de control relevantes**.
 - Si el ciclo corresponde a un estado existente que ahora es modificado, habrá que explicar e indicar **solo** el valor de las señales de control **nuevas o modificadas**.
 - Si el ciclo corresponde a un estado existente que no se modifica, **no** será necesario explicar ni indicar el valor de las señales de control.
- Finalmente, habrá que ampliar el **diagrama de estados** de la unidad de control, así como la **tabla de verdad** de las salidas de la **unidad de control** para que incluyan los cambios realizados. Ambos se adjuntan con el enunciado.

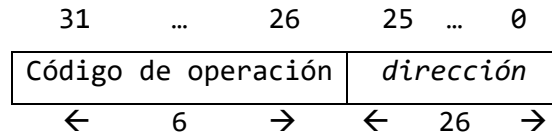
Solución:

a) Esta instrucción realiza las siguientes operaciones **desde el punto de vista de la arquitectura**:

$$PC \leftarrow PC + 4$$

$$M[PC[31-28] \parallel (dirección \ll 2)] \leftarrow M[PC[31-28] \parallel (dirección \ll 2)] + 4$$

b) De los tres formatos de instrucción de MIPS, el que mejor encaja es el J, que especifica una dirección de memoria mediante una pseudodirección en binario puro con 26 bits. Por tanto, tendremos:



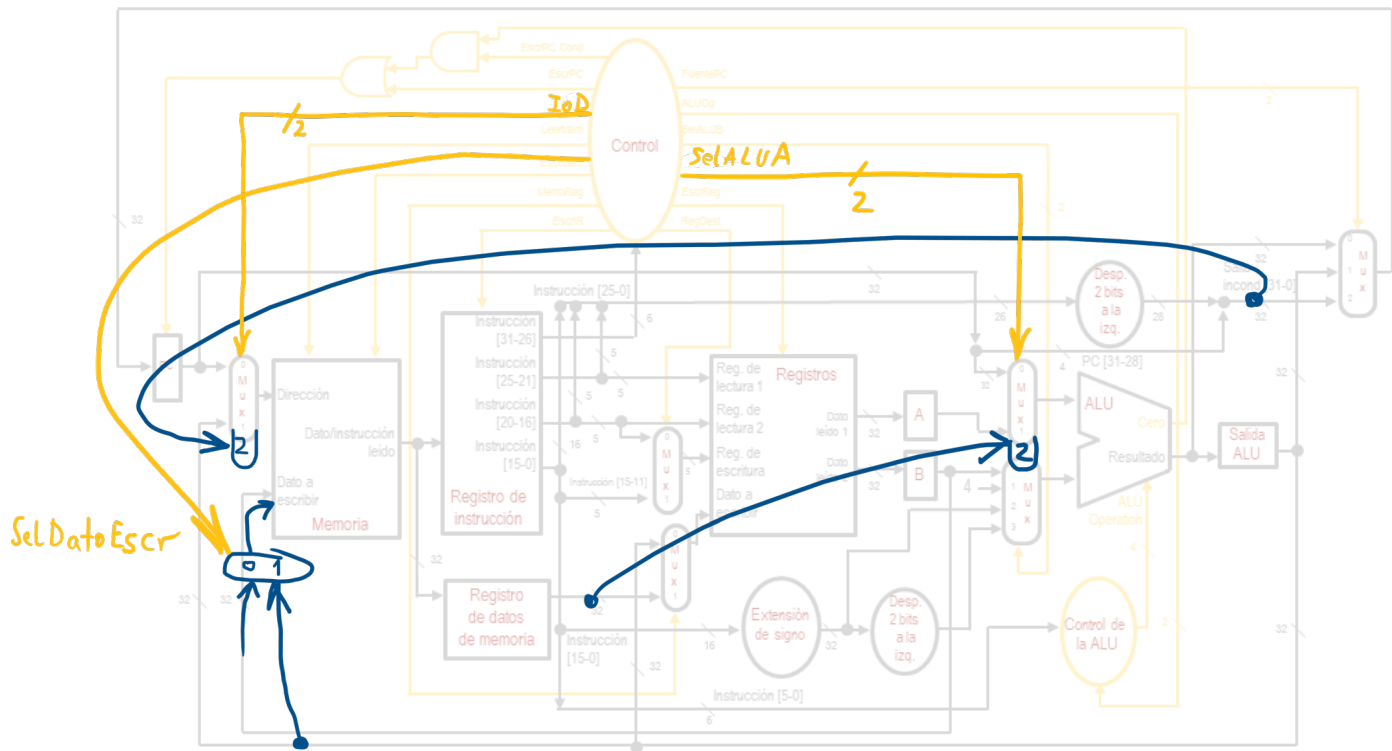
Esta instrucción tiene estos operandos:

- La dirección de memoria que va a ver incrementado su valor en 4, que se especifica mediante la pseudodirección de 26 bits contenida en la instrucción. Por lo tanto, usa un modo de direccionamiento pseudodirecto: se toma el valor de 26 bits, se multiplica por 4 (desplazamiento de 2 posiciones hacia la izquierda) y los 4 bits superiores se copian del valor del registro **PC** (cuando se haga esta operación el registro ya valdrá **PC + 4**). El resultado de esta operación es la dirección de memoria que vamos a incrementar, es decir, necesitaremos leer su valor, le sumaremos 4 y escribiremos el resultado en la misma dirección.
- El registro **PC**, que se actualiza de la manera habitual para sumarle 4 (modo de direccionamiento implícito).

c) Las modificaciones que se proponen son las siguientes:

- Por un lado, colocamos un nuevo cable desde la entrada 2 del multiplexor controlado por la señal **FuentePC** hasta la entrada de dirección de la memoria, donde ampliamos el multiplexor controlado por la señal **IoD**, que pasa a ser de 2 bits. La entrada 2 de este multiplexor ampliado es la que recibe el nuevo cable.
- Colocamos un nuevo cable desde el registro **MDR** hasta el primer operando de la ALU, para lo cual ampliamos el multiplexor controlado por la señal **Se1ALUA**, que pasa a ser de 2 bits. El nuevo cable se conecta a la entrada 2.
- Añadimos un nuevo multiplexor en la entrada de Dato a escribir de la memoria, controlado por una nueva señal **Se1DatoEscr**. A la entrada 0 conectamos el registro **B**, mientras que a la entrada 1 conectamos el registro **MDR**.

El camino de datos modificado quedaría de la siguiente manera:



La ejecución ciclo a ciclo de la nueva instrucción sería la siguiente:

Ciclo 1 (IF)/Estado 0: Este ciclo está sin modificar. La búsqueda de la instrucción se realiza como en el resto de las instrucciones. Las señales de control son las habituales en el estado 0, salvo por el hecho de que ahora tanto **IoD** como **SelALUA** son de 2 bits.

$IR \leftarrow M[PC]$
 $PC \leftarrow PC + 4$

Las señales de control son las habituales para este estado.

Ciclo 2 (ID)/Estado 1: La decodificación de la instrucción se realiza como en el resto de las instrucciones.

$A \leftarrow \text{Reg}[IR[25-21]]$
 $B \leftarrow \text{Reg}[IR[20-16]]$
 $\text{Salida ALU} \leftarrow PC + \text{extSigno16a32}(IR[15-0]) \ll 2$

Los valores de los registros **A**, **B** y **Salida ALU** calculados en este ciclo son irrelevantes para esta instrucción, pero aun así se calculan por retrocompatibilidad con el resto de las instrucciones.

Las señales de control son las habituales para este estado.

Ciclo 3 (MEM)/Estado 10: En este nuevo estado, calculamos la dirección de memoria a partir de la pseudodirección de 26 bits y se lee dicha posición. El dato leído lo almacenamos en el registro **MDR**.

$$\text{MDR} \leftarrow M[\text{PC}[31-28] \parallel (\text{IR}[25-0] \ll 2)]$$

Señales de control:

EscrPC = 0 (protegemos el **PC** contra escrituras indeseadas)

EscrPC Cond = 0 (no es instrucción condicional)

IoD = 10 (seleccionamos la dirección de 32 bits construida a partir de la pseudodirección de 26 bits como dirección de memoria)

LeerMem = 1 (se lee la memoria)

EscrMem = 0 (si se lee la memoria, no se puede escribir en ella)

EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)

EscrReg = 0 (protegemos el banco de registros contra escrituras indeseadas)

El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 4 (EX)/Estado 11: En este nuevo estado, se incrementa el valor del registro **MDR** en 4 en la ALU y el resultado se almacena en el registro **Salida ALU**.

$$\text{Salida ALU} \leftarrow \text{MDR} + 4$$

Señales de control:

EscrPC = 0 (protegemos el **PC** contra escrituras indeseadas)

EscrPC Cond = 0 (no es instrucción condicional)

LeerMem = 0 (no hay que leer la memoria)

EscrMem = 0 (protegemos la memoria contra escrituras indeseadas)

EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)

ALUOp = 00 (sumar)

Se1ALUB = 01 (seleccionamos 4 como segundo operando de la ALU)

Se1ALUA = 10 (seleccionamos **MDR** como primer operando de la ALU)

EscrReg = 0 (protegemos el banco de registros contra escrituras indeseadas)

El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 5 (MEM)/Estado 12: En este último estado, también nuevo, se escribe en memoria el valor que acabamos de calcular en **Salida ALU**.

$$M[\text{PC}[31-28] \parallel (\text{IR}[25-0] \ll 2)] \leftarrow \text{Salida ALU}$$

Señales de control:

EscrPC = 0 (protegemos el **PC** contra escrituras indeseadas)

EscrPC Cond = 0 (no es instrucción condicional)

IoD = 10 (seleccionamos la dirección de 32 bits construida a partir de la pseudodirección de 26 bits como dirección de memoria)

LeerMem = 0 (si se escribe en memoria, no puede leerse)

EscrMem = 1 (se escribe en memoria)

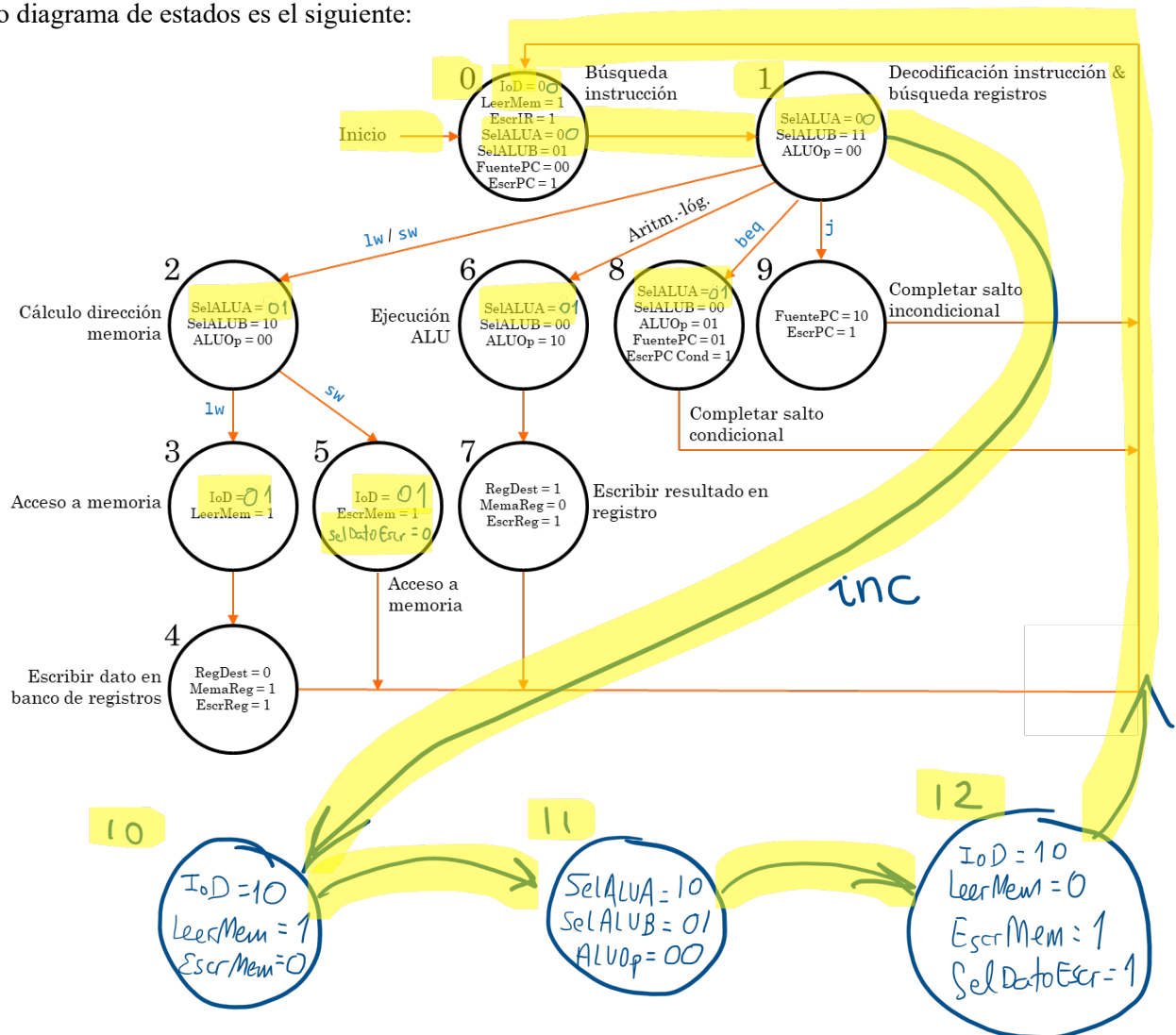
EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)

EscrReg = 0 (protegemos el banco de registros contra escrituras indeseadas)

SelDatoEscr = 1 (seleccionamos **SalidaALU** como dato que hay que escribir en memoria)

El resto de las señales son irrelevantes para este ciclo (X).

El nuevo diagrama de estados es el siguiente:



Hemos añadido tres nuevos estados: el 10, el 11 y el 12, que vendrían después del 1. También hemos hecho que las señales **IoD** y **SelALUA** sean de 2 bits, lo que nos ha obligado a modificar también varios estados, como el 0, el 1, 2, el 3, el 5, el 6 y el 8. Por último, hemos añadido la señal **SelDatoEscr**, que interviene en los estados 5 y 12.

La secuencia de estados de la nueva instrucción sería: $0 \rightarrow 1 \rightarrow 10 \rightarrow 11 \rightarrow 12 \rightarrow 0$.

La nueva tabla de verdad de las salidas de la unidad de control principal sería:

Estado actual	EscrPC	EscrPCCond	IoD1	IoD0	LeerMem	EscrMem	EscrIR	MemReg	FuentePC1	FuentePC0	ALUOp1	ALUOp0	SelALUB1	SelALUB0	SelALUA1	SelALUA0	EscrReg	RegDest	SelDatoEscr
0000	1	0	0	0	1	0	1	X	0	0	0	0	0	1	0	0	0	X	X
0001	0	0	X	X	0	0	0	X	X	X	0	0	1	1	0	0	0	X	X
0010	0	0	X	X	0	0	0	X	X	X	0	0	1	0	0	1	0	X	X
0011	0	0	0	1	1	0	0	X	X	X	X	X	X	X	X	X	0	X	X
0100	0	0	X	X	0	0	0	1	X	X	X	X	X	X	X	X	1	0	X
0101	0	0	0	1	0	1	0	X	X	X	X	X	X	X	X	X	0	X	0
0110	0	0	X	X	0	0	0	X	X	X	1	0	0	0	0	1	0	X	X
0111	0	0	X	X	0	0	0	0	X	X	X	X	X	X	X	X	1	1	X
1000	0	1	X	X	0	0	0	X	0	1	0	1	0	0	0	1	0	X	X
1001	1	0	X	X	0	0	0	X	1	0	X	X	X	X	X	X	0	X	X
1010	0	0	1	0	1	0	0	X	X	X	X	X	X	X	X	X	0	X	X
1011	0	0	X	X	0	0	0	X	X	X	0	0	0	1	1	0	0	X	X
1100	0	0	1	0	0	1	0	X	X	X	X	X	X	X	X	X	0	X	1

La antigua señal **IoD** ahora corresponde a **IoD0**, de igual forma que la antigua señal **SelALUA** ahora es **SelALUA0**. Hemos añadido las filas correspondiente a los estados 10 (1010), 11 (1011) y 12 (1100), así como la columna correspondiente a las nuevas señales **IoD1**, **SelALUA1** y **SelDatoEscr**.

Para los nuevos estados, las señales **EscrPC**, **EscrPC Cond**, **LeerMem**, **EscrMem**, **EscrIR** y **EscrReg** se han mantenido a 0 para evitar que se modifique la memoria, el registro de instrucción o el banco de registros, salvo que en el estado 10 se activa **LeerMem** y en el 12, **EscrMem**. El resto de las señales de control se han marcado a X al no intervenir en la instrucción y ser su valor irrelevante.

Microoperaciones realizadas por la instrucción, desde el punto de vista de la organización multiciclo:

- 1: $IR \leftarrow M[PC]$
 $PC \leftarrow PC + 4$
- 2: $A \leftarrow Reg[IR[25-21]]$
 $B \leftarrow Reg[IR[20-16]]$
 $Salida\ ALU \leftarrow PC + extSigno16a32(IR[15-0]) \ll 2$
- 3: $MDR \leftarrow M[PC[31-28] \ || \ (IR[25-0] \ll 2)]$
- 4: $Salida\ ALU \leftarrow MDR + 4$
- 5: $M[PC[31-28] \ || \ (IR[25-0] \ll 2)] \leftarrow Salida\ ALU$

14. [Ejercicio de examen] Sea un procesador MIPS con un camino de datos multiciclo (se adjunta en una hoja aparte). Al repertorio seleccionado en clase, se desea añadir una variante de la instrucción de carga, que llamaremos **l2w** (carga de 2 palabras). Un ejemplo de esta instrucción en ensamblador sería el siguiente:

l2w \$t0, \$t1, (\$s0)

La instrucción del ejemplo hace lo siguiente:

- El registro **\$s0** apunta a una palabra que se encuentra en la memoria. Se lee esta palabra en memoria y se realiza la carga en el registro **\$t0**.
- Se lee a continuación en memoria la palabra en la dirección inmediatamente consecutiva (es decir, **\$s0+4**). Esta palabra se carga en el registro **\$t1**.
- El valor del registro **\$s0** permanece constante en todo el proceso.

Se pide:

- a) Desde el punto de vista de la arquitectura (tal cual se vería en una tabla de resumen de instrucciones), escribir qué operación hace esta instrucción en notación de **pseudocódigo**.
- b) Justificar razonadamente qué **formato binario de instrucción** sería el más adecuado para esta instrucción y descomponerla en campos. **No** se pide expresarla en hexadecimal.
- c) Indicar cuáles son todos los **operandos** de la instrucción, así como los **modos de direccionamiento** de cada uno de ellos.
- d) Explicar **razonada y detalladamente** todos los cambios que son necesarios en el camino de datos para poder ejecutar esta instrucción. Los requisitos que debe cumplir la instrucción son los siguientes:
 - El hecho de añadir esta nueva instrucción debe ser retrocompatible con todas las instrucciones estándar, que deben seguir siendo ejecutables como hasta ahora.
 - El número total de ciclos para ejecutar la nueva instrucción debe ser el mínimo posible. Para ello se podrán solapar en el mismo ciclo operaciones simultáneas que no interfieran entre sí.
 - El número total de estados de la unidad de control debe ser el mínimo posible.
 - El número total de cambios en el camino de datos debe ser el mínimo posible y también estos cambios han de ser lo más pequeños posible.
 - La prioridad de los criterios de minimización es la siguiente: número total de ciclos (más prioritario), número total de estados, número y tamaño de los cambios (menos prioritario).

Este apartado tiene tres puntos de vista interrelacionados que habrá que explicar:

- Por un lado, habrá que **explicar** las modificaciones y **dibujarlas** sobre el camino de datos, tales como añadir cables o ampliar otros existentes, añadir multiplexores o ampliar otros

existentes, añadir registros o ampliar otros existentes, añadir puertas lógicas o ampliar las existentes, etc.

- Por otro, habrá que **explicar, ciclo a ciclo**, qué operaciones se realizan (desde el punto de vista de la organización, es decir, indicando los componentes concretos del camino de datos multiciclo involucrados). Para ello, se deberá usar la **notación de pseudocódigo** con el operador de asignación flecha $\leftarrow$ para indicar **todas las operaciones** que intervienen en dicho ciclo.

Además del pseudocódigo:

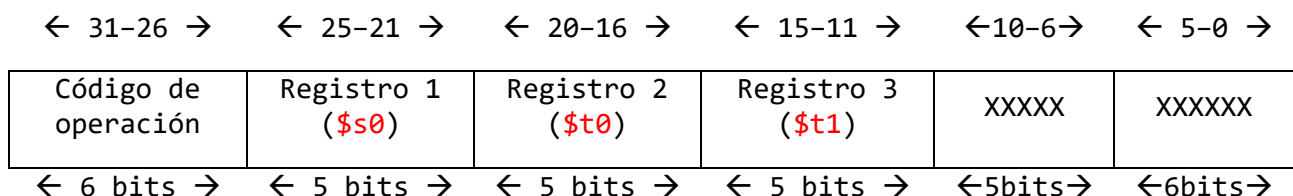
- Si el ciclo corresponde a un estado nuevo de la unidad de control, habrá que explicar e indicar el valor de **todas las señales de control relevantes**.
 - Si el ciclo corresponde a un estado existente que ahora es modificado, habrá que explicar e indicar **solo** el valor de las señales de control **nuevas o modificadas**.
 - Si el ciclo corresponde a un estado existente que no se modifica, **no** será necesario explicar ni indicar el valor de las señales de control.
- Finalmente, habrá que ampliar el **diagrama de estados** de la unidad de control, así como la **tabla de verdad** de las salidas de la **unidad de control** para que incluyan los cambios realizados. Ambos se adjuntan en la hoja del diagrama del camino de datos.

Solución:

a) Desde el punto de vista de la arquitectura, esta instrucción realiza las siguientes operaciones:

```
PC ← PC + 4
$t0 ← M[$s0]
$t1 ← M[$s0 + 4]
```

b) Como tenemos que codificar 3 registros en la instrucción, el único formato que podemos usar es el R.



c) Los operandos de la instrucción son:

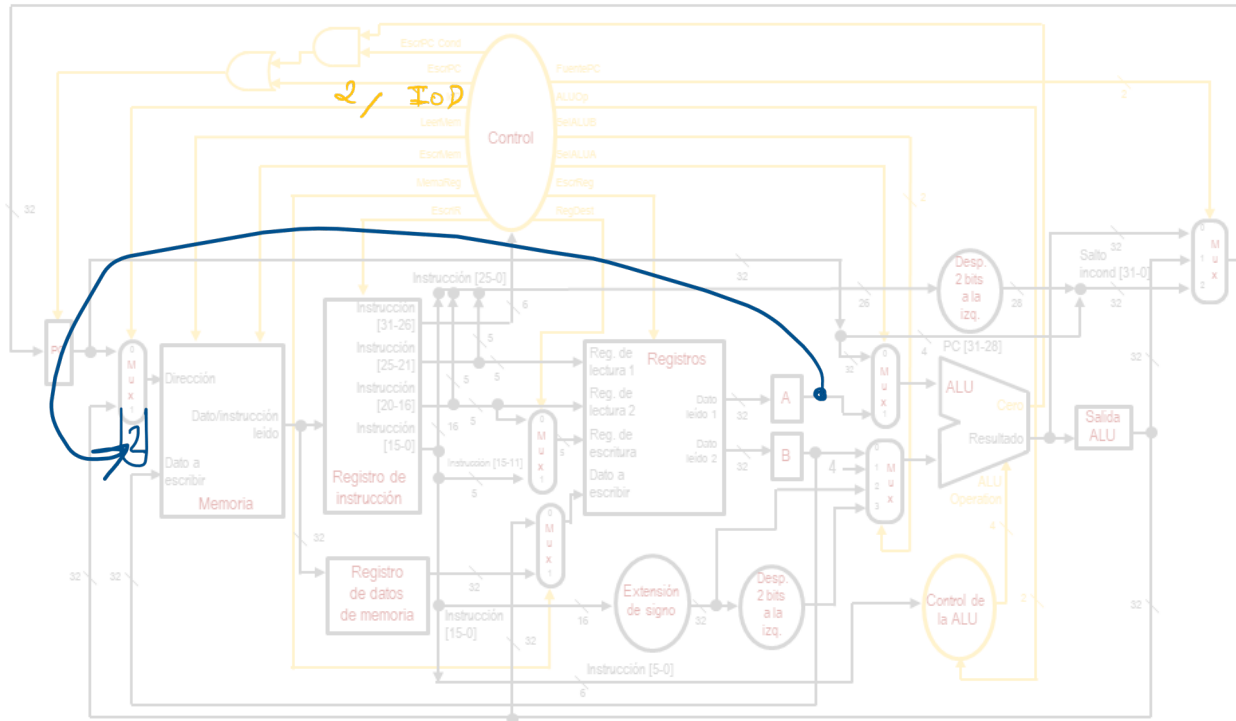
- Una palabra que se lee en memoria en la dirección $\$s0$, con direccionamiento indirecto a registro. El registro $\$s0$ no es el operando, sino el puntero al operando.
- Otra palabra que se lee en memoria en la dirección $\$s0 + 4$, con un direccionamiento indirecto a registro con desplazamiento, donde el desplazamiento es implícito.
- El registro destino $\$t0$, con direccionamiento directo a registro.
- El registro destino $\$t1$, también con direccionamiento directo a registro.

- El contador de programa **PC**, que, junto con el 4 que se le suma, tiene direccionamiento implícito.

d) Para poder ejecutar esta instrucción, se proponen los siguientes cambios en el camino de datos:

- Añadir una nueva entrada al multiplexor controlado por la señal de control **IoD**, que pasaría a ser de 2 bits.
- Conectar con un cable la salida del registro **A** con la entrada 2 del multiplexor que acabamos de ampliar.

El camino de datos modificado quedaría de la siguiente manera:



La ejecución ciclo a ciclo sería la siguiente:

Ciclo 1 (IF)/Estado 0: La lectura de la instrucción se realiza como en el resto de las instrucciones. Las operaciones realizadas son:

$IR \leftarrow M[PC]$ // **IR** contiene la palabra de operación de la instrucción
 $PC \leftarrow PC + 4$

Las señales de control son las habituales. Debemos recordar que **IoD** ahora es de 2 bits y su valor es **00**.

Ciclo 2 (ID)/Estado 1: La decodificación de la instrucción se realiza como en el resto de las instrucciones. El registro **A** contiene el valor de **\$s0**, mientras que el valor de **B** es irrelevante. Las operaciones realizadas son:

$A \leftarrow \text{Reg}[IR[25-21]]$
 $B \leftarrow \text{Reg}[IR[20-16]]$
 $\text{Salida ALU} \leftarrow PC + \text{extSigno16a32}(IR[15-0]) \ll 2$

Las señales de control son las habituales.

Ciclo 3 (EX/MEM)/Estado 10: Este es un estado nuevo, en el que se lee la primera palabra en memoria y se guarda en el registro de datos de memoria.

$MDR \leftarrow M[A]$
 $Salida\ ALU \leftarrow A + 4$

Señales de control:

$EscrPC = 0$ (protegemos el **PC** contra escrituras indeseadas)
 $EscrPC\ Cond = 0$ (no es instrucción condicional)
 $IoD = 10$ (seleccionamos **A** como dirección de memoria)
 $LeerMem = 1$ (se lee la memoria)
 $EscrMem = 0$ (si se lee la memoria, no se puede escribir en ella)
 $EscrIR = 0$ (protegemos el registro **IR** contra escrituras indeseadas)
 $ALUOp = 00$ (sumar) – si quisiéramos usar el valor **10**, deberíamos haber rellenado el campo de función de la instrucción con el valor **100000**
 $SelALUB = 01$ (seleccionamos **4** como segundo operando de la ALU)
 $SelALUA = 1$ (seleccionamos **A** como primer operando de la ALU)
 $EscrReg = 0$ (protegemos el banco de registros contra escrituras indeseadas)
El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 4 (MEM/WB)/Estado 11: Este es otro estado nuevo, en el que se lee una segunda palabra en la memoria, mientras se escribe en el banco de registros.

$MDR \leftarrow M[Salida\ ALU]$
 $Reg[IR[20-16]] \leftarrow MDR$

Señales de control:

$EscrPC = 0$ (protegemos el **PC** contra escrituras indeseadas)
 $EscrPC\ Cond = 0$ (no es instrucción condicional)
 $IoD = 01$ (seleccionamos **Salida ALU** como dirección de memoria)
 $LeerMem = 1$ (se lee la memoria)
 $EscrMem = 0$ (si se lee la memoria, no se puede escribir en ella)
 $EscrIR = 0$ (protegemos el registro **IR** contra escrituras indeseadas)
 $MemaReg = 1$ (seleccionamos **MDR** como dato que hay que escribir en el banco)
 $EscrReg = 1$ (activamos la escritura en el banco de registros)
 $RegDest = 0$ (seleccionamos el registro indicado en los bits **20-16** de la instrucción como destino)
El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 5 (WB)/Estado 12: Este es otro nuevo estado, que no debemos confundir ni con el 4 ni con el 7. Se realiza una escritura en el banco de registros.

$\text{Reg}[\text{IR}[15-11]] \leftarrow \text{MDR}$

Señales de control:

$\text{EscrPC} = 0$ (protegemos el PC contra escrituras indeseadas)

$\text{EscrPC Cond} = 0$ (no es instrucción condicional)

$\text{LeerMem} = 0$ (no hay que leer la memoria)

$\text{EscrMem} = 0$ (protegemos la memoria contra escrituras indeseadas)

$\text{EscrIR} = 0$ (protegemos el registro IR contra escrituras indeseadas)

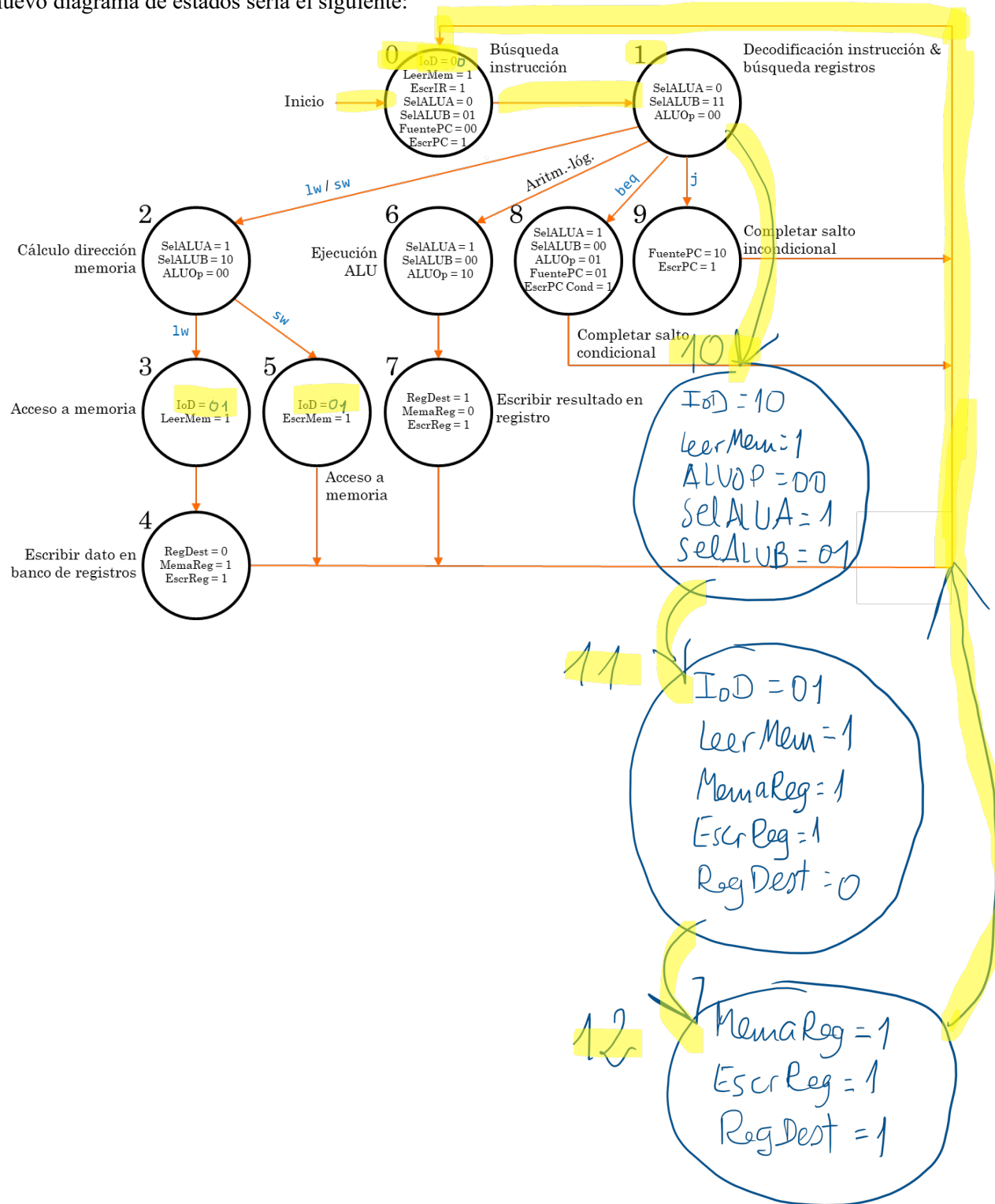
$\text{MemaReg} = 1$ (seleccionamos MDR como dato que hay que escribir en el banco)

$\text{EscrReg} = 1$ (activamos la escritura en el banco de registros)

$\text{RegDest} = 1$ (seleccionamos el registro indicado en los bits 15-11 de la instrucción como destino)

El resto de las señales son irrelevantes para este ciclo (X).

El nuevo diagrama de estados sería el siguiente:



La secuencia de estados de la nueva instrucción sería: $0 \rightarrow 1 \rightarrow 10 \rightarrow 11 \rightarrow 12 \rightarrow 0$.

La nueva tabla de verdad de las salidas de la unidad de control principal sería:

Estado actual	EscrPC	EscrPCCond	Iod1	Iod0	LeerMem	EscrMem	EscrIR	MemReg	FuentePC1	FuentePC0	ALUOp1	ALUOp0	Se1ALUB1	Se1ALUB0	Se1ALUA	EscrReg	RegDest
0000	1	0	0	0	1	0	1	X	0	0	0	0	0	1	0	0	X
0001	0	0	X	X	0	0	0	X	X	X	0	0	1	1	0	0	X
0010	0	0	X	X	0	0	0	X	X	X	0	0	1	0	1	0	X
0011	0	0	0	1	1	0	0	X	X	X	X	X	X	X	X	0	X
0100	0	0	X	X	0	0	0	1	X	X	X	X	X	X	X	1	0
0101	0	0	0	1	0	1	0	X	X	X	X	X	X	X	X	0	X
0110	0	0	X	X	0	0	0	X	X	X	1	0	0	0	1	0	X
0111	0	0	X	X	0	0	0	0	X	X	X	X	X	X	X	1	1
1000	0	1	X	X	0	0	0	X	0	1	0	1	0	0	1	0	X
1001	1	0	X	X	0	0	0	X	1	0	X	X	X	X	X	0	X
1010	0	0	1	0	1	0	0	X	X	X	0	0	0	1	1	0	X
1011	0	0	0	1	1	0	0	1	X	X	X	X	X	X	X	1	0
1100	0	0	X	X	0	0	0	1	X	X	X	X	X	X	X	1	1

Microoperaciones realizadas por la instrucción, desde el punto de vista de la organización multiciclo:

- 1: $IR \leftarrow M[PC]$
 $PC \leftarrow PC + 4$
- 2: $A \leftarrow \text{Reg}[IR[25-21]]$
 $B \leftarrow \text{Reg}[IR[20-16]]$
 $\text{Salida ALU} \leftarrow PC + \text{extSigno16a32}(IR[15-0]) \ll 2$
- 3: $MDR \leftarrow M[A]$
 $\text{Salida ALU} \leftarrow A + 4$
- 4: $MDR \leftarrow M[\text{Salida ALU}]$
 $\text{Reg}[IR[20-16]] \leftarrow MDR$
- 5: $\text{Reg}[IR[15-11]] \leftarrow MDR$

15. [Ejercicio de examen] Sea un procesador MIPS con un camino de datos multiciclo (se adjunta en una hoja aparte). Al repertorio seleccionado en clase, se desea añadir una variante de la instrucción de suma, que llamaremos **add2**. Un ejemplo de esta instrucción en ensamblador sería el siguiente:

add2 \$s0, 12(\$t0)

La instrucción del ejemplo hace lo siguiente:

- Realiza la suma de dos valores: el registro **\$s0** y la palabra almacenada en la memoria en la dirección que se calcula a partir del registro **\$t0** y del valor numérico 12.
- El resultado de la suma se almacena en el registro **\$s0**.

Se pide:

- b) Desde el punto de vista de la **arquitectura** (tal cual se vería en una tabla de resumen de instrucciones), escribir qué operación hace esta instrucción en notación de **pseudocódigo**.

- c) Justificar qué **formato binario** de instrucción sería el más adecuado para esta instrucción y descomponerla en campos. No es necesario expresarla en hexadecimal.
- d) Indicar cuáles son **todos los operandos** de la instrucción, así como los **modos de direccionamiento** de cada uno de ellos.
- e) Explicar razonada y detalladamente **todos los cambios** que son necesarios en el camino de datos para poder ejecutar esta instrucción. Los requisitos que debe cumplir la instrucción son los siguientes:
 - El hecho de añadir esta nueva instrucción debe ser retrocompatible con todas las instrucciones estándar, que deben seguir siendo ejecutables como hasta ahora.
 - El número total de ciclos para ejecutar la nueva instrucción debe ser el mínimo posible. Para ello se podrán solapar en el mismo ciclo operaciones simultáneas que no interfieran entre sí.
 - El número total de estados de la unidad de control debe ser el mínimo posible.
 - El número total de cambios en el camino de datos debe ser el mínimo posible y también estos cambios han de ser lo más pequeños posible.

Este apartado tiene tres puntos de vista interrelacionados que habrá que explicar:

- Por un lado, habrá que **explicar** las modificaciones y **dibujarlas** sobre el camino de datos, tales como añadir cables o ampliar otros existentes, añadir multiplexores o ampliar otros existentes, añadir registros o ampliar otros existentes, añadir puertas lógicas o ampliar las existentes, etc.
- Por otro, habrá que **explicar, ciclo a ciclo**, qué operaciones se realizan (desde el punto de vista de la **organización**, es decir, indicando los componentes concretos del camino de datos involucrados). Para ello, se deberá usar la notación de **pseudocódigo** con el operador de asignación flecha $\leftarrow$ para indicar todas las operaciones que intervienen en dicho ciclo. Además del pseudocódigo:
 - Si el ciclo corresponde a un estado nuevo de la unidad de control, habrá que explicar e indicar el valor de **todas las señales de control relevantes**.
 - Si el ciclo corresponde a un estado existente que ahora es modificado, habrá que explicar e indicar **solo** el valor de las señales de control **nuevas o modificadas**.
 - Si el ciclo corresponde a un estado existente que no se modifica, **no** será necesario explicar ni indicar el valor de las señales de control.
- Finalmente, habrá que ampliar el **diagrama de estados** de la unidad de control, así como la **tabla de verdad** de las salidas de la **unidad de control** para que incluyan los cambios realizados. Ambos se adjuntan con el enunciado.

Solución:

a) Esta instrucción es una mezcla entre **lw** y **add**. Esta instrucción realiza las siguientes operaciones **desde el punto de vista de la arquitectura**:

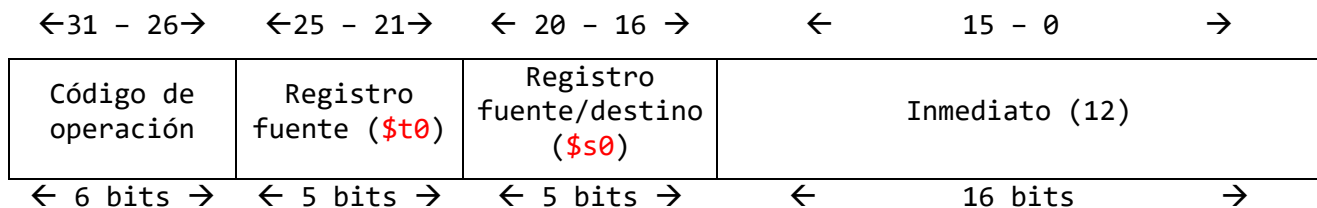
$PC \leftarrow PC + 4$
 $\$s0 \leftarrow M[\$t0 + \text{extSigno16a32}(12)] + \$s0$

Es decir, calculamos una dirección de memoria exactamente igual que **lw**, con la suma del registro base, en este caso, **\$t0**, y con el inmediato, en este caso, **12**, al cual se le ha realizado la extensión de signo de 16 a 32 bits. Se lee esa dirección de memoria y la palabra leída se suma al otro registro, en este caso, **\$s0**. El resultado se carga o almacena en el registro destino, de nuevo, **\$s0**.

Por otro lado, como es habitual en todas las instrucciones, se incrementa el contador de programa **PC** en 4 unidades para que apunte a la siguiente instrucción del programa.

b) El formato más adecuado es el I, que es el que nos permite codificar en la instrucción 2 registros y un inmediato de 16 bits.

Formato I:

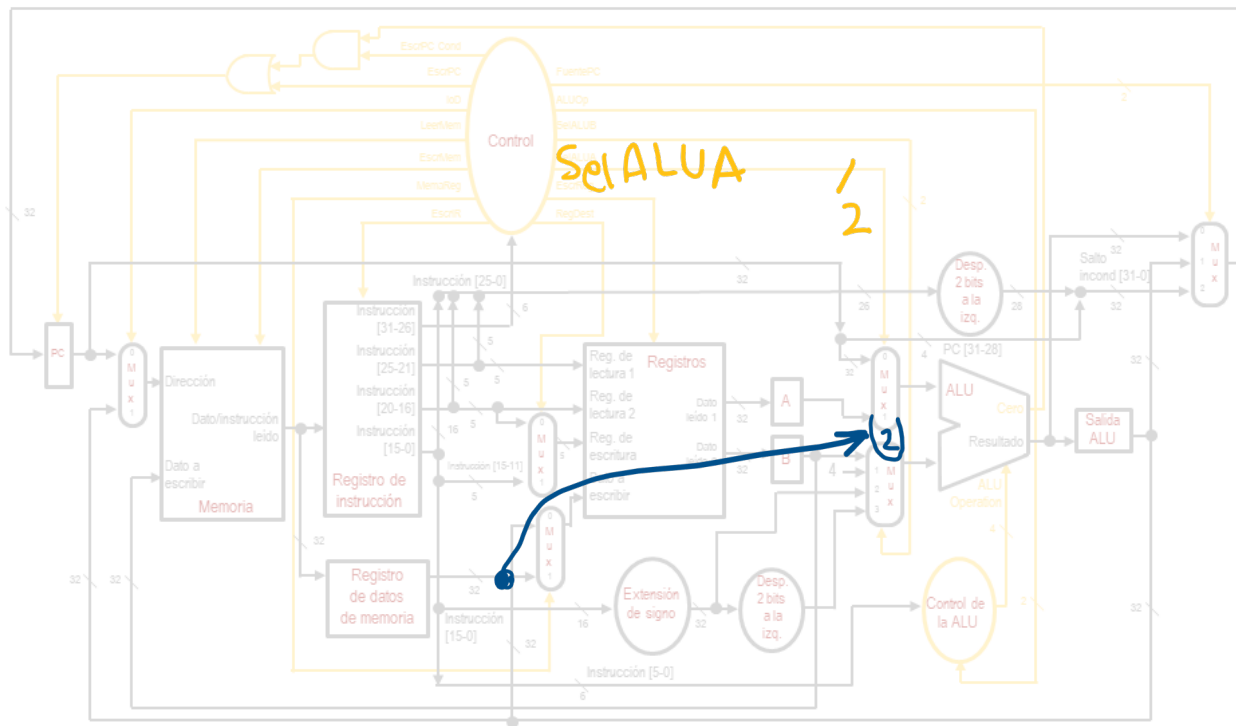


c) Esta instrucción tiene los siguientes operandos:

- El registro **\$s0**, que utiliza el direccionamiento directo a registro, ya que se codifica su número (16) con 5 bits en la instrucción y el operando de la suma es el valor del registro.
- El dato que leemos en memoria, que utiliza el direccionamiento indirecto a registro con desplazamiento. Nótese que el verdadero operando es esa palabra en memoria, no el registro **\$t0** y el inmediato 12 (solo se usan para calcular la dirección del operando). Por eso, necesitamos codificar en la instrucción un número de registro con 5 bits y un inmediato con 16 bits, pero ni el registro ni el inmediato son los valores que intervienen en la suma.
- El contador de programa **PC**, al que se le suma 4 para que apunte a la siguiente instrucción. Su direccionamiento es implícito porque no se codifica en ninguna parte de la instrucción.

d) Se proponen los siguientes cambios en el camino de datos:

- Ampliar el multiplexor que hay en la entrada 1 de la ALU para que sea de tamaño 4 a 1. La señal de control **Se1ALUA** pasaría a ser de 2 bits.
- Conectar con un cable la salida del registro de datos de memoria **MDR** con la entrada 2 del multiplexor anterior. La entrada 3 queda libre y las otras dos se dejan como están.



La ejecución ciclo a ciclo sería la siguiente:

Ciclo 1 (IF)/Estado 0: La lectura de la instrucción se realiza como en el resto de las instrucciones. Las operaciones realizadas son:

$IR \leftarrow M[PC]$ // IR contiene la palabra de operación de la instrucción
 $PC \leftarrow PC + 4$

Las señales de control son las habituales para este estado. Debemos recordar que $SelALUA$ ahora es de 2 bits y su valor es 00 .

Ciclo 2 (ID)/Estado 1: La decodificación de la instrucción se realiza como en el resto de las instrucciones. El registro A contiene el valor de $\$t0$, mientras que B contiene $\$s0$. Las operaciones realizadas son:

$A \leftarrow Reg[IR[25-21]]$
 $B \leftarrow Reg[IR[20-16]]$
 $Salida\ ALU \leftarrow PC + extSigno16a32(IR[15-0]) \ll 2$

Las señales de control son las habituales para este estado. De nuevo, $SelALUA$ vale 00 .

Ciclo 3 (EX)/Estado 2: La ejecución de la instrucción continúa como si fuera una carga normal, por tanto, a través del estado existente número 2. El registro A contiene el registro base (en este caso, $\$t0$), mientras que el inmediato 12 está codificado en los bits 15-0 del registro IR . Por tanto, las operaciones realizadas son:

$Salida\ ALU \leftarrow A + extSigno16a32(IR[15-0])$

Las señales de control son las habituales para este estado. Ahora, **Se1ALUA** vale 01.

Ciclo 4 (MEM)/Estado 3: Ahora leemos la dirección de memoria que acabamos de calcular en la ALU, exactamente igual que si fuera una carga normal. Por ello, reaprovechamos el estado 3. Las operaciones son:

$$\text{MDR} \leftarrow M[\text{Salida ALU}]$$

Las señales de control son las habituales para este estado.

Ciclo 5 (EX)/Estado 10: Aquí comienza la parte diferenciadora de esta instrucción. En **MDR** tenemos el primer operando de la suma, el valor leído en la memoria, mientras que el otro se encuentra en **B**, que contiene el valor del registro **\$s0**. Por tanto:

$$\text{Salida ALU} \leftarrow \text{MDR} + \text{B}$$

Las señales de control necesarias son las siguientes:

EscrPC = 0 (protegemos el registro **PC** contra escrituras indeseadas)

EscrPC Cond = 0 (aunque es una instrucción condicional, en este ciclo no debemos tocar **PC**)

LeerMem = 0 (protegemos la memoria contra lecturas indeseadas)

EscrMem = 0 (protegemos la memoria contra escrituras indeseadas)

EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)

ALUOp = 00 (sumar)

Se1ALUB = 00 (seleccionamos **B** como segundo operando de la **ALU**)

Se1ALUA = 10 (seleccionamos **MDR** como primer operando de la **ALU**)

EscrReg = 0 (protegemos el banco de registros contra escrituras indeseadas)

El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 6 (WB)/Estado 11: Para terminar, se almacena el resultado de la suma en el banco de registros. Nótese que es un estado diferente al número 4 porque no coincide el valor de la señal **MemaReg**. Luego:

$$\text{Reg}[\text{IR}[20-16]] \leftarrow \text{Salida ALU}$$

Las señales de control necesarias son las siguientes:

EscrPC = 0 (protegemos el registro **PC** contra escrituras indeseadas)

EscrPC Cond = 0 (aunque es una instrucción condicional, en este ciclo no debemos tocar **PC**)

LeerMem = 0 (protegemos la memoria contra lecturas indeseadas)

EscrMem = 0 (protegemos la memoria contra escrituras indeseadas)

EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)

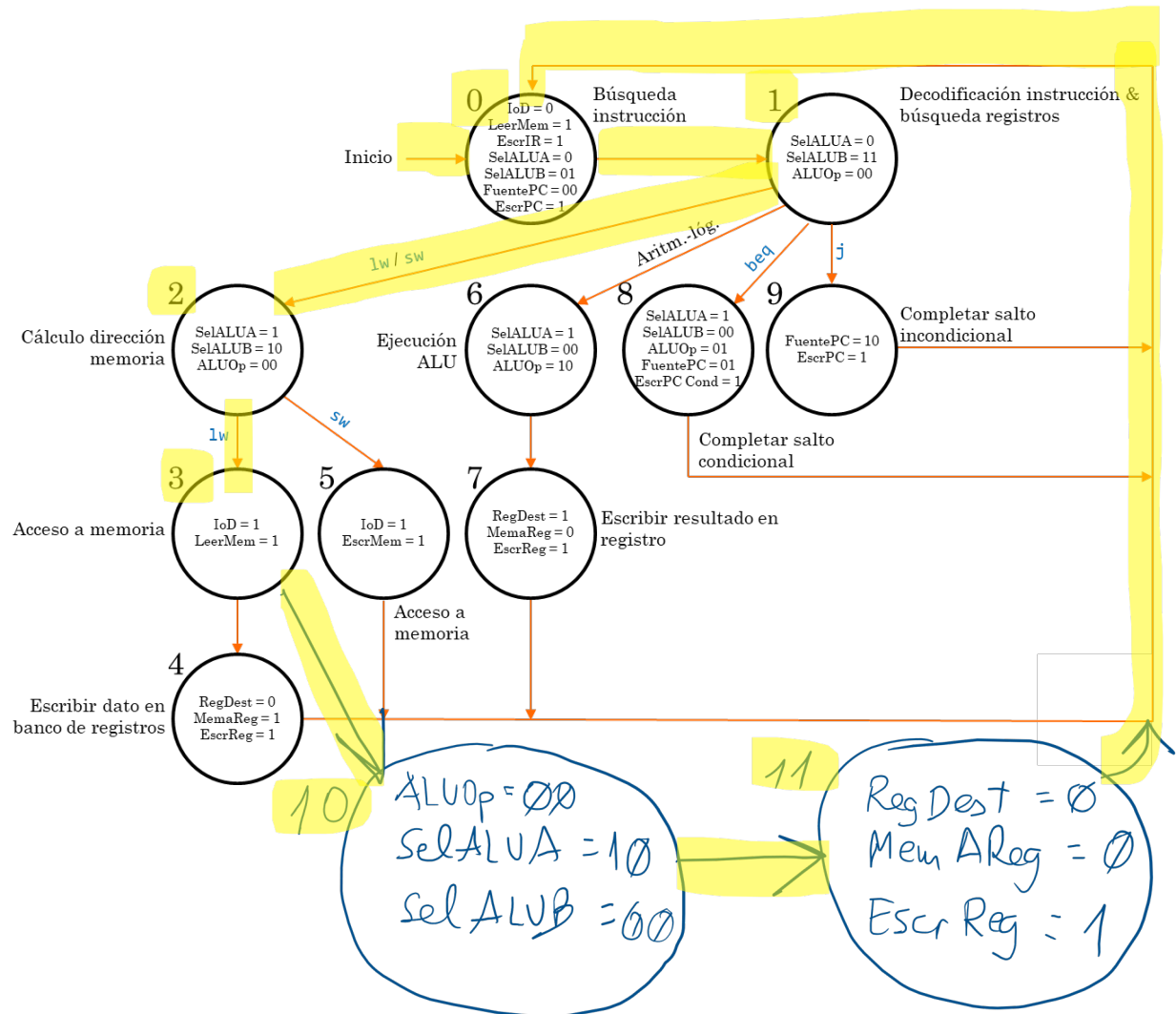
MemaReg = 0 (seleccionamos **Salida ALU** como dato a escribir en el banco de registros)

EscrReg = 1 (activamos la escritura del resultado en el banco de registros)

RegDest = 0 (seleccionamos el registro indicado en los bits 20-16 de la instrucción como destino)

El resto de las señales son irrelevantes para este ciclo (X).

El nuevo diagrama de estados es:



La secuencia de estados de la nueva instrucción sería: 0 → 1 → 2 → 3 → 10 → 11 → 0.

La nueva tabla de verdad de las salidas de la unidad de control principal sería:

Estado actual	EscrPC	EscrPCCond	IOd	LeerMem	EscrMem	EscrIR	MemReg	FuentePC1	FuentePC0	ALUp1	ALUp0	Se1ALUB1	Se1ALUB0	Se1ALUA1	Se1ALUA0	EscrReg	RegDest
0000	1	0	0	1	0	1	X	0	0	0	0	0	1	0	0	0	X
0001	0	0	X	0	0	0	X	X	X	0	0	1	1	0	0	0	X
0010	0	0	X	0	0	0	X	X	X	0	0	1	0	0	1	0	X
0011	0	0	1	1	0	0	X	X	X	X	X	X	X	X	X	0	X
0100	0	0	X	0	0	0	1	X	X	X	X	X	X	X	X	1	0
0101	0	0	1	0	1	0	X	X	X	X	X	X	X	X	X	0	X
0110	0	0	X	0	0	0	X	X	X	1	0	0	0	0	1	0	X
0111	0	0	X	0	0	0	0	X	X	X	X	X	X	X	X	1	1
1000	0	1	X	0	0	0	X	0	1	0	1	0	0	0	1	0	X
1001	1	0	X	0	0	0	X	1	0	X	X	X	X	X	X	0	X
1010	0	0	X	0	0	0	X	X	X	0	0	0	0	1	0	0	X
1011	0	0	X	0	0	0	0	X	X	X	X	X	X	X	X	1	0

Microoperaciones realizadas por la instrucción, desde el punto de vista de la organización multiciclo:

- 1: $IR \leftarrow M[PC]$
 $PC \leftarrow PC + 4$
- 2: $A \leftarrow \text{Reg}[IR[25-21]]$
 $B \leftarrow \text{Reg}[IR[20-16]]$
 $\text{Salida ALU} \leftarrow PC + \text{extSigno16a32}(IR[15-0]) \ll 2$
- 3: $\text{Salida ALU} \leftarrow A + \text{extSigno16a32}(IR[15-0])$
- 4: $MDR \leftarrow M[\text{Salida ALU}]$
- 5: $\text{Salida ALU} \leftarrow MDR + B$
- 6: $\text{Reg}[IR[20-16]] \leftarrow \text{Salida ALU}$

16. [Ejercicio de examen] Sea un procesador MIPS con un camino de datos multiciclo estándar. Se desea añadir una nueva instrucción al repertorio, denominada **lw2**, que debe realizar una carga de palabra mediante una doble indirección. Un ejemplo de esta instrucción en ensamblador sería el siguiente:

lw2 \$s0, 4((\$s1))

Esta instrucción hace lo siguiente:

- Primero, se lee la palabra contenida en la dirección de memoria a la que apunta **\$s1**.
- Después, a la palabra leída, se le suma el inmediato, en este caso, 4. El resultado es la dirección de memoria de otra palabra contenida en memoria.
- Por último, se lee dicha palabra y se almacena en el registro destino **\$s0**.

Se pide:

- a) Desde el punto de vista de la **arquitectura** (tal cual se vería en una tabla de resumen de instrucciones), escribir qué operación hace esta instrucción en notación de **pseudocódigo**.
- b) Justificar qué **formato binario** de instrucción sería el más adecuado para esta instrucción y descomponerla en campos. Habrá que indicar también **todos los operandos** de la instrucción, así como los **modos de direccionamiento** de cada uno de ellos.

- c) Escribir la instrucción que se pone en el ejemplo en **formato hexadecimal**.
- d) Explicar razonada y detalladamente **todos los cambios** que son necesarios en el camino de datos para poder ejecutar esta instrucción. Los requisitos que debe cumplir la instrucción son los siguientes:
- El hecho de añadir esta nueva instrucción debe ser retrocompatible con todas las instrucciones estándar, que deben seguir siendo ejecutables como hasta ahora.
 - El número total de ciclos para ejecutar la nueva instrucción debe ser el mínimo posible. Para ello se podrán solapar en el mismo ciclo operaciones simultáneas que no interfieran entre sí.
 - El número total de estados de la unidad de control debe ser el mínimo posible.
 - El número total de cambios en el camino de datos debe ser el mínimo posible y también estos cambios han de ser lo más pequeños posible.

Este apartado tiene tres puntos de vista interrelacionados que habrá que explicar:

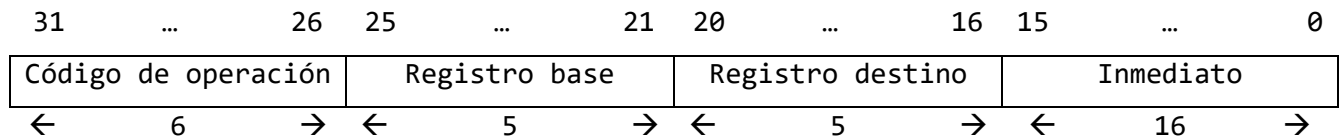
- Por un lado, habrá que **explicar** las modificaciones y **dibujarlas** sobre el camino de datos, tales como añadir cables o ampliar otros existentes, añadir multiplexores o ampliar otros existentes, añadir registros o ampliar otros existentes, añadir puertas lógicas o ampliar las existentes, etc.
- Por otro, habrá que **explicar, ciclo a ciclo**, qué operaciones se realizan (desde el punto de vista de la **organización**, es decir, indicando los componentes concretos del camino de datos involucrados). Para ello, se deberá usar la notación de **pseudocódigo** con el operador de asignación flecha $\leftarrow$ para indicar todas las operaciones que intervienen en dicho ciclo. Además del pseudocódigo:
 - Si el ciclo corresponde a un estado nuevo de la unidad de control, habrá que explicar e indicar el valor de **todas las señales de control relevantes**.
 - Si el ciclo corresponde a un estado existente que ahora es modificado, habrá que explicar e indicar **solo** el valor de las señales de control **nuevas o modificadas**.
 - Si el ciclo corresponde a un estado existente que no se modifica, **no** será necesario explicar ni indicar el valor de las señales de control.
- Finalmente, habrá que ampliar el **diagrama de estados** de la unidad de control, así como la **tabla de verdad** de las salidas de la **unidad de control** para que incluyan los cambios realizados. Ambos se adjuntan con el enunciado.

Solución:

- a) Esta instrucción realiza las siguientes operaciones **desde el punto de vista de la arquitectura**:

```
PC  $\leftarrow$  PC + 4
$S0  $\leftarrow$  M[M[$S1] + 4]
```

b) Se trata de una instrucción que necesita codificar dos registros y un inmediato. Por tanto, el formato de instrucción ideal es el I, que se divide en campos de la siguiente manera:



Los operandos de esta instrucción son tres:

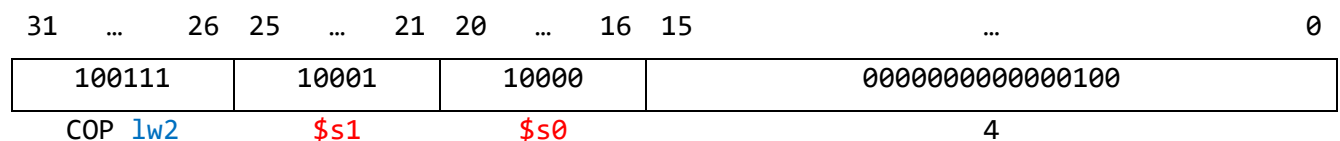
- Un operando fuente que hay que leer de la memoria, al que se hace referencia mediante un registro base (\$s1) y un inmediato (4). El modo de direccionamiento sería esta “doble indirección con desplazamiento” que introduce el enunciado como novedad.
- Un registro destino (\$s0), en el que se escribirá el dato leído, y al que se hace referencia con un modo de direccionamiento directo a registro.
- El registro PC también es operando porque se actualiza al valor PC + 4. Este operando está oculto en la instrucción, ya que esta suma se codifica en el código de operación, con lo que el modo de direccionamiento es el implícito.

c) Para poder escribir esta instrucción en hexadecimal, debemos recurrir a la hoja de resumen de instrucciones de MIPS. Por un lado, vemos en la tabla de registros que el \$s0 (registro destino) corresponde al número 16, mientras que \$s1 (registro base) es el 17. Ambos registros deben codificarse en binario puro con 5 bits (10000 y 10001, respectivamente).

El inmediato 4 se codifica en complemento a 2 con 16 bits. Al ser un número positivo, su representación coincide con la de binario puro, con lo que sería 00000000000000100.

Por último, necesitamos el código de operación. No podemos usar el mismo que el de la instrucción lw normal, ya que entonces ambas instrucciones serían indistinguibles para la unidad de control. Necesitamos un código de operación que esté sin usar. Elegimos uno al azar de la hoja de resumen, por ejemplo, 100111 (aquí cada uno que elija el que más le guste de los que no estén asignados ya).

Con estos valores, la instrucción en código máquina se codifica de la siguiente manera:



Para pasar la instrucción a hexadecimal, agrupamos los bits de 4 en 4, con lo que resulta:

100111 10001 10000 00000000000000100 = 1001 1110 0011 0000 0000 0000 0000 0100 = 0x9E300004

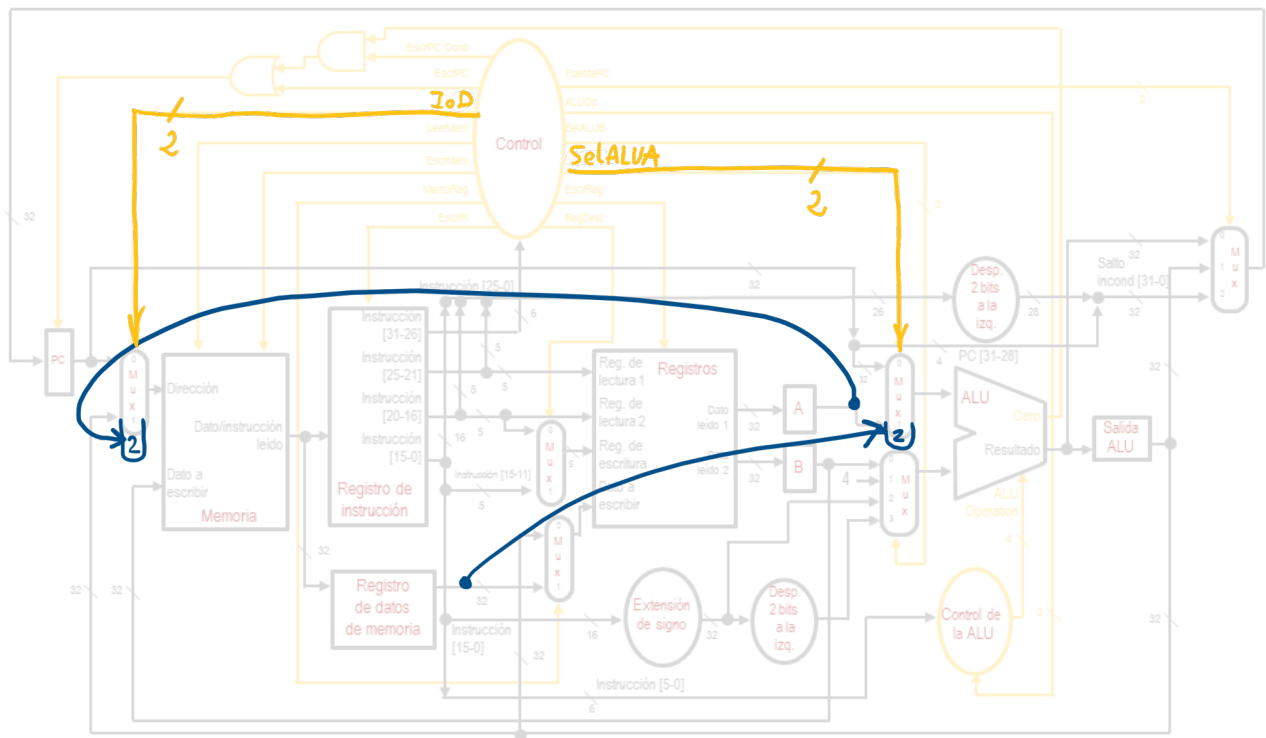
d) Estos son los cambios que proponemos en el camino de datos para poder ejecutar esta instrucción:

- Por un lado, debemos leer el registro fuente \$s1, que se almacenará en el registro auxiliar A durante el segundo ciclo de la instrucción. Este valor hay que llevarlo directamente a la entrada de dirección de la memoria, para lo cual extendemos un cable desde A hasta dicha entrada. Como hay un multiplexor de 2 a 1 controlado por la señal IoD, ampliamos dicho multiplexor para que sea de 4 a 1, de tal forma que en la

entrada 2 colocamos el cable procedente de **A**. A su vez, la señal **IoD** pasa a ser de 2 bits (**IoD1 IoD0**). La entrada 3 del multiplexor queda al aire, mientras que las entradas 0 y 1 se dejan como hasta ahora por retrocompatibilidad.

- El valor leído de la memoria se podrá almacenar en el Registro de Datos de Memoria (**MDR**). Pero a este valor le tenemos que poder sumar el inmediato en la ALU. Como el inmediato, tras sufrir extensión de signo, llega a la ALU por su segundo operando, entonces debemos añadir un nuevo cable desde **MDR** hasta el primer operando de la ALU. Allí hay un multiplexor 2 a 1 controlado por la señal **Se1ALUA**, que deberemos ampliar para que sea un multiplexor 4 a 1. A su vez, la señal **Se1ALUA** pasa a ser de 2 bits (**Se1ALUA1 Se1ALUA0**). En la entrada 2 del multiplexor conectamos el cable procedente de **MDR**. La entrada 3 queda libre y las entradas 0 y 1 se dejan como hasta ahora por retrocompatibilidad.

El camino de datos modificado quedaría de la siguiente manera:



La ejecución ciclo a ciclo de la nueva instrucción sería la siguiente:

Ciclo 1 (IF)/Estado 0: La búsqueda de la instrucción se realiza como en el resto de las instrucciones. Los únicos cambios son que ahora las señales de control **IoD** y **Se1ALUA** son de 2 bits, no de 1 bit cada una.

IR $\leftarrow$ **M[PC]**
PC $\leftarrow$ **PC** + 4

Las señales de control son las habituales para este estado.

Ciclo 2 (ID)/Estado 1: La decodificación de la instrucción se realiza como en el resto de las instrucciones.

$A \leftarrow \text{Reg}[\text{IR}[25-21]]$
 $B \leftarrow \text{Reg}[\text{IR}[20-16]]$
 $\text{Salida ALU} \leftarrow \text{PC} + \text{extSigno16a32}(\text{IR}[15-0]) \ll 2$

A contendrá el valor del registro base **\$s1**, mientras que **B** contendrá el valor del registro destino **\$s0**. El cálculo de **Salida ALU** representa el hipotético destino de un salto condicional y esta instrucción no es de ese tipo, con lo que este valor no nos sirve para nada en este caso.

Las señales de control son las habituales para este estado.

Ciclo 3 (MEM)/Estado 10: Este ciclo consiste en la lectura de la posición de memoria a la que apunta el registro base, que está almacenado en **A**. Necesitamos un estado nuevo, que llamaremos 10 (en binario 1010). La microoperación realizada es:

$\text{MDR} \leftarrow \text{M}[\text{A}]$

Señales de control:

EscrPC = 0 (protegemos el **PC** contra escrituras indeseadas)
EscrPC Cond = 0 (no es instrucción condicional)
IoD = 10 (seleccionamos **A** como dirección de memoria que hay que leer)
LeerMem = 1 (se lee la memoria)
EscrMem = 0 (si se lee la memoria, no se puede escribir en ella)
EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)
EscrReg = 0 (protegemos el banco de registros contra escrituras indeseadas)

El resto de las señales son irrelevantes para este ciclo (X).

Nótese que este es un estado diferente al 3, que es utilizado por la instrucción **lw** normal, ya que en ese caso necesitamos que la señal **IoD** valga 01.

Ciclo 4 (EX)/Estado 11: En este ciclo se realiza en la ALU la suma entre **MDR** y el inmediato 4 tras extensión de signo. El resultado se almacena en el registro **Salida ALU**. Se trata de un estado nuevo, que llamaremos 11 (en binario 1011). La microoperación realizada es:

$\text{Salida ALU} \leftarrow \text{MDR} + \text{extSigno16a32}(\text{IR}[15-0])$

Las señales de control son:

EscrPC = 0 (protegemos el **PC** contra escrituras indeseadas)
EscrPC Cond = 0 (no es instrucción condicional)
LeerMem = 0 (no hay que leer la memoria)
EscrMem = 0 (protegemos la memoria contra escrituras indeseadas)
EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)

$ALUOp = 00$ (sumar)

$Se1ALUB = 10$ (seleccionamos el inmediato con extensión de signo como segundo operando de la ALU)

$Se1ALUA = 10$ (seleccionamos MDR como primer operando de la ALU)

$EscrReg = 0$ (protegemos el banco de registros contra escrituras indeseadas)

El resto de las señales son irrelevantes para este ciclo (X).

Nótese que este es un estado diferente al 2, que es utilizado por la instrucción lw normal, ya que en ese caso necesitamos que la señal $Se1ALUA$ valga 01.

Ciclo 5 (MEM)/Estado 3: Este ciclo es el mismo que realiza la instrucción lw en el estado 3, ya que se produce la lectura de la palabra a la que apunta el registro $Salida\ ALU$. Luego:

$MDR \leftarrow M[Salida\ ALU]$

Las señales de control son las habituales para este estado.

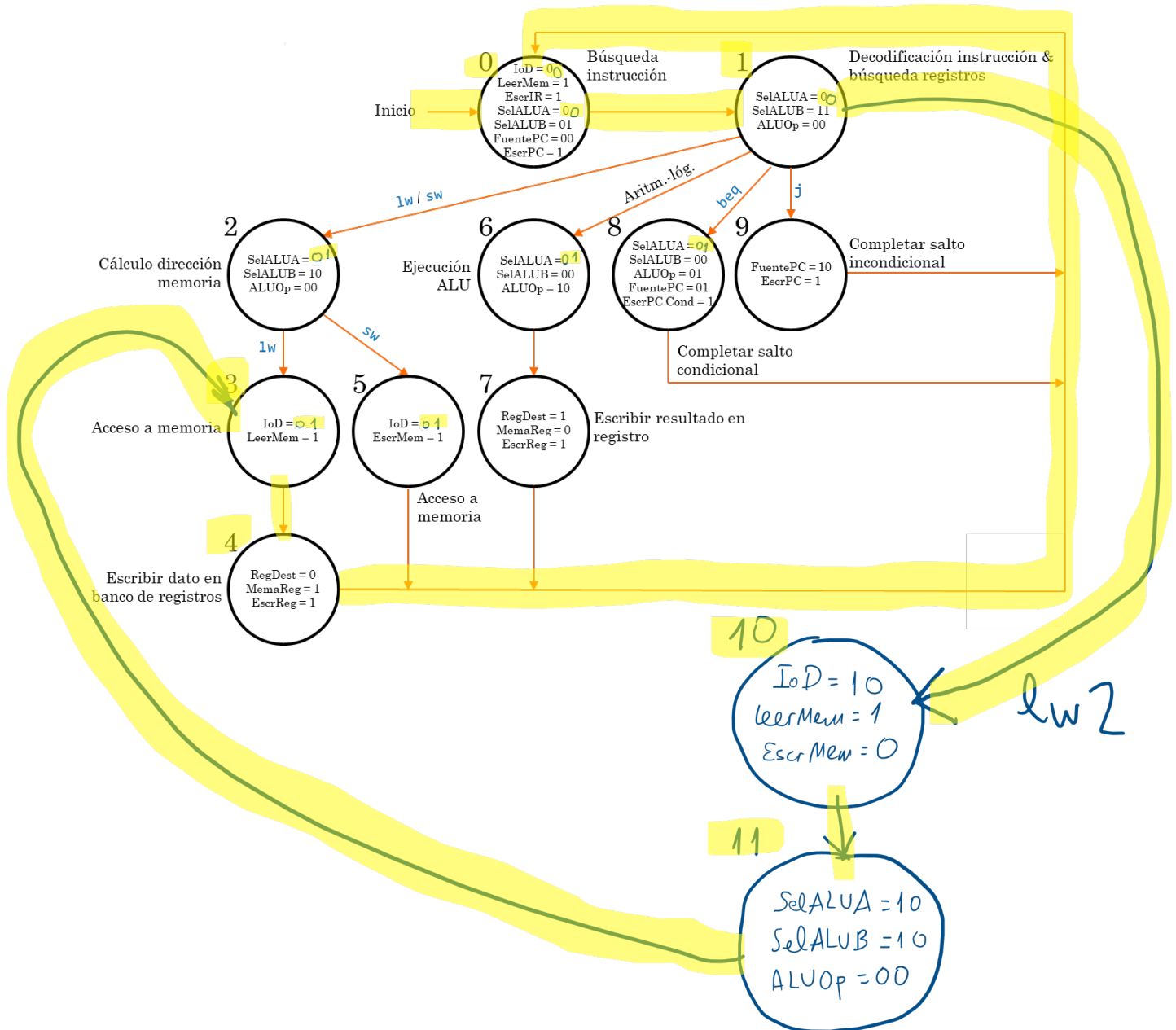
Ciclo 6 (WB)/Estado 4: En este último ciclo se produce la escritura de la palabra leída en el banco de registros en el registro destino $\$s0$, exactamente igual que en el estado 4 habitual de la instrucción lw . Luego:

$Reg[IR[20-16]] \leftarrow MDR$

Las señales de control son las habituales para este estado.

Hemos logrado ejecutar la nueva instrucción $lw2$ en 6 ciclos en vez de los 5 habituales de lw . Y solo nos ha hecho falta añadir dos nuevos estados a la unidad de control.

El nuevo diagrama de estados de la unidad de control es el siguiente:



Hemos añadido dos nuevos estados: el 10 y el 11. También hemos hecho que las señales IoD y $SelALUA$ sean de 2 bits, lo que nos ha obligado a modificar también varios estados, como el 0, el 1, el 2, el 4, el 5, 6 y el 8.

La secuencia de estados de la nueva instrucción sería: $0 \rightarrow 1 \rightarrow 10 \rightarrow 11 \rightarrow 3 \rightarrow 4 \rightarrow 0$.

Por último, la tabla de verdad de la unidad de control queda ahora así:

Estado actual	EscrPC	EscrPCCond	IoD1	IoD0	LeerMem	EscrMem	EscrIR	MemReg	FuentePC1	FuentePC0	ALUp1	ALUp0	SelALUB1	SelALUB0	SelALUA1	SelALUA0	EscrReg	RegDest
0000	1	0	0	0	1	0	1	X	0	0	0	0	0	1	0	0	0	X
0001	0	0	X	X	0	0	0	X	X	X	0	0	1	1	0	0	0	X
0010	0	0	X	X	0	0	0	X	X	X	0	0	1	0	0	1	0	X
0011	0	0	0	1	1	0	0	X	X	X	X	X	X	X	X	X	0	X
0100	0	0	X	X	0	0	0	1	X	X	X	X	X	X	X	X	1	0
0101	0	0	0	1	0	1	0	X	X	X	X	X	X	X	X	X	0	X
0110	0	0	X	X	0	0	0	X	X	X	1	0	0	0	0	1	0	X
0111	0	0	X	X	0	0	0	0	X	X	X	X	X	X	X	X	1	1
1000	0	1	X	X	0	0	0	X	0	1	0	1	0	0	0	1	0	X
1001	1	0	X	X	0	0	0	X	1	0	X	X	X	X	X	X	0	X
1010	0	0	1	0	1	0	0	X	X	X	X	X	X	X	X	X	0	X
1011	0	0	X	X	0	0	0	X	X	X	0	0	1	0	1	0	0	X

La antigua señal **IoD** ahora corresponde a **IoD0**. De igual forma, la antigua señal **SelALUA** es ahora **SelALUA0**. Hemos añadido las filas correspondientes a los nuevos estados 10 (**1010**) y 11 (**1011**), así como las columnas correspondientes a las nuevas señales **IoD1** y **SelALUA1**.

Para los dos nuevos estados, en la medida de lo posible, las señales **EscrPC**, **EscrPC Cond**, **LeerMem**, **EscrMem**, **EscrIR** y **EscrReg** se han mantenido a 0 para evitar que se modifique la memoria, el registro de instrucción o el banco de registros o que se produzca un salto de instrucción indeseado. La excepción es **LeerMem** en el estado 10, donde vale 1 para realizar la lectura del operando en la memoria. El resto de las señales de control se han marcado a X al no intervenir en la instrucción y ser su valor irrelevante.

Microoperaciones realizadas por la instrucción, **desde el punto de vista de la organización multiciclo**:

```

1: IR ← M[PC]
   PC ← PC + 4
2: A ← Reg[IR[25-21]]
   B ← Reg[IR[20-16]]
   Salida ALU ← PC + extSigno16a32(IR[15-0])<<2
3: MDR ← M[A]
4: Salida ALU ← MDR + extSigno16a32(IR[15-0])
5: MDR ← M[Salida ALU]
6: Reg[IR[20-16]] ← MDR

```

17. [Ejercicio de examen] Sea un procesador MIPS con un camino de datos multiciclo (se adjunta en una hoja aparte). Al repertorio seleccionado en clase, se desea añadir una variante de la instrucción de suma, que llamaremos **add2consec** (suma de 2 datos consecutivos). Un ejemplo de esta instrucción en ensamblador sería el siguiente:

add2consec \$s1, (\$s0)

La instrucción del ejemplo hace lo siguiente:

- El registro **\$s0** apunta a una palabra que se encuentra en la memoria. Se lee esta palabra en memoria.
- Se lee también en memoria la palabra en la dirección inmediatamente consecutiva (**\$s0+4**).

- Se suman ambas palabras y el resultado de la suma se almacena en el registro **\$s1**. El valor del registro **\$s0** permanece constante en todo el proceso.

Se pide:

- a) Desde el punto de vista de la arquitectura (tal cual se vería en una tabla de resumen de instrucciones), escribir qué operación hace esta instrucción en notación de **pseudocódigo**.
- b) Justificar razonadamente qué **formato binario de instrucción** sería el más adecuado para esta instrucción y descomponerla en campos. No es necesario expresarla en hexadecimal.
- c) Indicar cuáles son todos los **operandos** de la instrucción, así como los **modos de direccionamiento** de cada uno de ellos.
- d) Explicar **razonada y detalladamente** todos los cambios que son necesarios en el camino de datos para poder ejecutar esta instrucción. Los requisitos que debe cumplir la instrucción son los siguientes:
 - El hecho de añadir esta nueva instrucción debe ser retrocompatible con todas las instrucciones estándar, que deben seguir siendo ejecutables como hasta ahora.
 - El número total de ciclos para ejecutar la nueva instrucción debe ser el mínimo posible. Para ello se podrán solapar en el mismo ciclo operaciones simultáneas que no interfieran entre sí.
 - El número total de estados de la unidad de control debe ser el mínimo posible.
 - El número total de cambios en el camino de datos debe ser el mínimo posible y también estos cambios han de ser lo más pequeños posible.
 - La prioridad de los criterios de minimización es la siguiente: número total de ciclos (más prioritario), número total de estados, número y tamaño de los cambios (menos prioritario).

Este apartado tiene tres puntos de vista interrelacionados que habrá que explicar:

- Por un lado, habrá que **explicar** las modificaciones y **dibujarlas** sobre el camino de datos, tales como añadir cables o ampliar otros existentes, añadir multiplexores o ampliar otros existentes, añadir registros o ampliar otros existentes, añadir puertas lógicas o ampliar las existentes, etc.
- Por otro, habrá que **explicar, ciclo a ciclo**, qué operaciones se realizan (desde el punto de vista de la organización, es decir, indicando los componentes concretos del camino de datos multiciclo involucrados). Para ello, se deberá usar la **notación de pseudocódigo** con el operador de asignación flecha **←** para indicar **todas las operaciones** que intervienen en dicho ciclo.

Además del pseudocódigo:

- Si el ciclo corresponde a un estado nuevo de la unidad de control, habrá que explicar e indicar el valor de **todas las señales de control relevantes**.
- Si el ciclo corresponde a un estado existente que ahora es modificado, habrá que explicar e indicar **solo** el valor de las señales de control **nuevas o modificadas**.

- Si el ciclo corresponde a un estado existente que no se modifica, **no** será necesario explicar ni indicar el valor de las señales de control.
- Finalmente, habrá que ampliar el **diagrama de estados** de la unidad de control, así como la **tabla de verdad** de las salidas de la **unidad de control** para que incluyan los cambios realizados. Ambos se adjuntan en la hoja del diagrama del camino de datos.

Solución:

a) Esta instrucción realiza las siguientes operaciones **desde el punto de vista de la arquitectura**:

$PC \leftarrow PC + 4$
 $\$s1 \leftarrow M[\$s0] + M[\$s0+4]$

b) En cuanto al formato de instrucción, solo necesitamos codificar 2 registros, con lo que podríamos elegir entre el I o el R, ya que en ese aspecto son equivalentes, pero, por razones que veremos inmediatamente, elegimos el formato R.

Formato R:

← 31 – 26 →	← 25 – 21 →	← 20 – 16 →	← 15 – 11 →	← 10 – 6 →	← 5 – 0 →
Código de operación	Registro Base (\$s0)	XXXXXX	Registro Destino (\$s1)	XXXXXX	100000
← 6 bits →	← 5 bits →	← 5 bits →	← 5 bits →	← 5 bits →	← 6 bits →

Nótese que el registro destino (\$s1) estará codificado en los bits 15-11, como es habitual en el formato R. La elección de este formato nos permitirá reaprovechar un estado preexistente (el 7), ya que el último ciclo de la instrucción (escritura en el banco) coincide con el último ciclo de la instrucción **add**.

En el formato R, el registro fuente \$s0 podría estar codificado en los bits 25-21 o 20-16, ya que en principio son posiciones equivalentes. Sin embargo, la elección de las posiciones 20-16 complicaría la suma $\$s0 + 4$, ya obligaría a introducir más cambios en el camino de datos. Por eso, escogemos los bits 25-21.

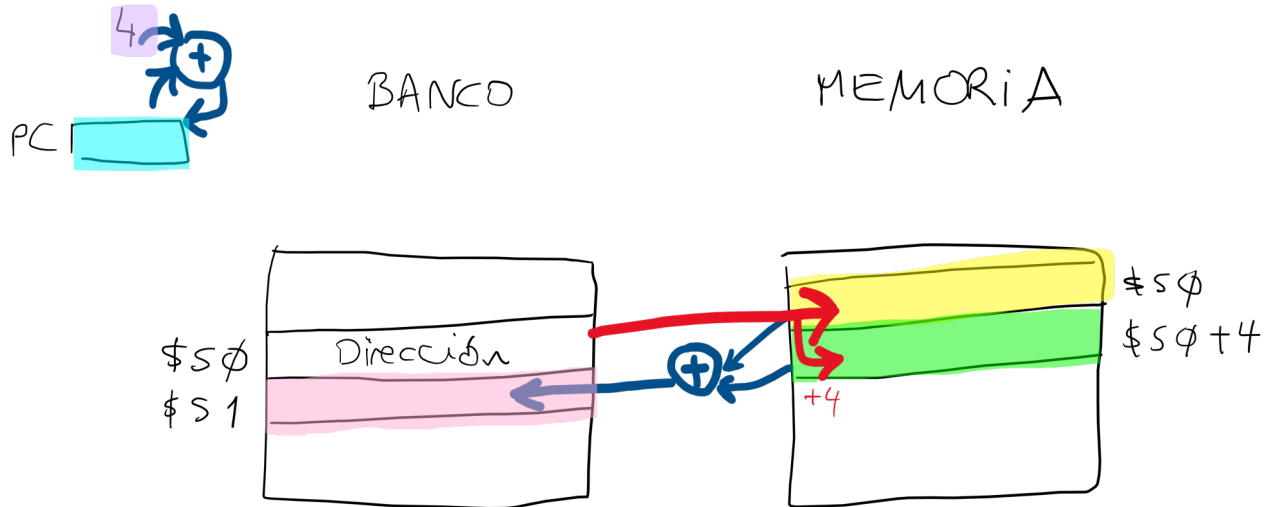
Por último, para poder reaprovechar un estado existente (el 6), codificaremos el campo de función **100000** en los 6 bits menos significativos de la instrucción, que corresponden a la operación de sumar en la ALU.

c) Viendo el pseudocódigo desde el punto de vista de la arquitectura, se ven de manera evidente los operandos de la instrucción:

$PC \leftarrow PC + 4$
 $\$s1 \leftarrow M[\$s0] + M[\$s0+4]$

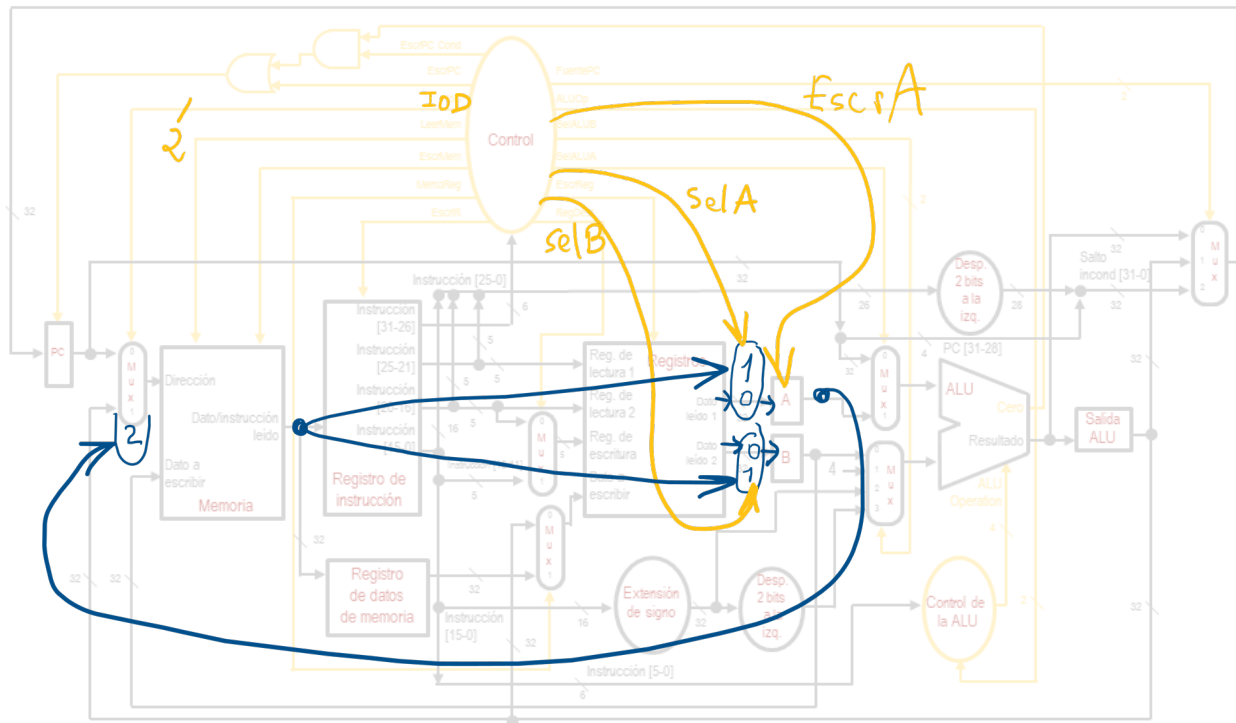
- El **valor en memoria al que apunta el registro \$s0**, al que accedemos con un direccionamiento indirecto a registro sin desplazamiento. Este será el primer sumando de la suma. Nótese que el registro \$s0 NO es el operando (es solo la dirección del primer sumando, no el sumando en sí).

- Un **segundo valor en memoria**, que es el que se encuentra en la dirección inmediatamente posterior a la del dato anterior. Accedemos a él con un direccionamiento indirecto a registro con desplazamiento. Este es el segundo sumando de la suma. El valor del desplazamiento (4) es implícito, ya que no se codifica en ninguna parte en la instrucción. Este 4 NO es un inmediato, ya que en el formato R no hay inmediatos.
- El registro **\$s1**, que es el que almacenará el resultado de la suma. Tiene un direccionamiento directo a registro.
- El contador de programa **PC**, que se ve incrementado en 4 unidades, como es habitual. Tiene un direccionamiento implícito. El 4 que se le suma también es operando implícito, aunque normalmente no lo tenemos en cuenta.



d) Se proponen los siguientes cambios en el camino de datos:

- Por un lado, ampliamos el multiplexor controlado por la señal **IoD** y llevamos un cable desde la salida del registro **A** hasta la entrada 2 (la 3 sería equivalente). Esto nos permitirá llevar el valor del registro **\$s0** a la entrada de direcciones de la memoria.
- Por otro lado, hemos añadido dos multiplexores entre las salidas del banco de registros y los registros auxiliares **A** y **B**. Las señales de control se llaman **SelA** y **SelB**, respectivamente. En la entrada 0, hemos conectado la salida que proviene del banco de registros, como era hasta ahora. En la entrada 1, hemos conectado con un cable nuevo la salida del registro **MDR**.
- Por último, el registro **A** tiene ahora una señal de habilitación de escritura **EscrA**. Solo cuando esta señal valga 1, se producirá una escritura en dicho registro. Si vale 0, el valor almacenado permanece constante.



La ejecución ciclo a ciclo sería la siguiente:

Ciclo 1 (IF)/Estado 0: La lectura de la instrucción se realiza como en el resto de las instrucciones. Las operaciones realizadas son:

$IR \leftarrow M[PC]$ // IR contiene la palabra de operación de la instrucción
 $PC \leftarrow PC + 4$

Las señales de control son las habituales (ver tabla). Debemos recordar que **IoD** ahora es de 2 bits y su valor es 00.

Ciclo 2 (ID)/Estado 1: La decodificación de la instrucción se realiza como en el resto de las instrucciones. El registro **A** contiene el valor de **\$s0**, mientras que el valor de **B** es irrelevante. Las operaciones realizadas son:

$A \leftarrow \text{Reg}[IR[25-21]]$
 $B \leftarrow \text{Reg}[IR[20-16]]$
 $\text{Salida ALU} \leftarrow PC + \text{extSigno16a32}(IR[15-0]) \ll 2$

Las señales de control son las habituales (ver tabla). Las señales **SelA** y **SelB** valen 0 cada una para escoger los valores leídos en el banco de registros. La señal **EscrA** vale 1 para permitir la escritura en **A**.

Ciclo 3 (EX/MEM)/Estado 10: En este ciclo, leemos el primer sumando de la operación, que se encuentra en memoria, y lo volvemos a guardar en **A**. Aprovechamos también para incrementar la dirección para apuntar al segundo valor, de forma que **Salida ALU** contenga ahora **\$s0 + 4**.

$A \leftarrow M[A]$
 $Salida\ ALU \leftarrow A + 4$

Señales de control:

$EscrPC = 0$ (al final del ciclo se actualiza el PC)
 $EscrPC\ Cond = 0$ (no es instrucción condicional)
 $IoD = 10$ (seleccionamos A como dirección de memoria)
 $LeerMem = 1$ (se lee la memoria)
 $EscrMem = 0$ (si se lee la memoria, no se puede escribir en ella)
 $EscrIR = 0$ (protegemos el registro IR contra escrituras indeseadas)
 $SelA = 1$ (seleccionamos el valor leído en memoria para escribirlo en A)
 $EscrA = 1$ (habilitamos la escritura en A)
 $ALUOp = 00$ (suma en la ALU)
 $SelALUA = 1$ (elegimos A como primer sumando)
 $SelALUB = 01$ (elegimos 4 como segundo sumando)
 $EscrReg = 0$ (protegemos el banco de registros contra escrituras indeseadas)

El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 4 (MEM)/Estado 11: En este ciclo, leemos el segundo sumando de la operación, también en memoria, y lo guardamos en B.

$B \leftarrow M[Salida\ ALU]$

Señales de control:

$EscrPC = 0$ (al final del ciclo se actualiza el PC)
 $EscrPC\ Cond = 0$ (no es instrucción condicional)
 $IoD = 01$ (seleccionamos Salida ALU como dirección de memoria)
 $LeerMem = 1$ (se lee la memoria)
 $EscrMem = 0$ (si se lee la memoria, no se puede escribir en ella)
 $EscrIR = 0$ (protegemos el registro IR contra escrituras indeseadas)
 $EscrA = 0$ (deshabilitamos la escritura en A para proteger el valor que contiene)
 $SelB = 1$ (seleccionamos el valor leído en memoria para escribirlo en B)
 $EscrReg = 0$ (protegemos el banco de registros contra escrituras indeseadas)

El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 5 (EX)/Estado 6: Este ciclo es idéntico al de ejecución de la instrucción **add**. En los registros **A** y **B** tenemos cargados los dos sumandos y la ALU recibe la operación de sumar gracias al campo de función de la instrucción.

$$\text{Salida ALU} \leftarrow A + B$$

Las señales son las habituales para este estado.

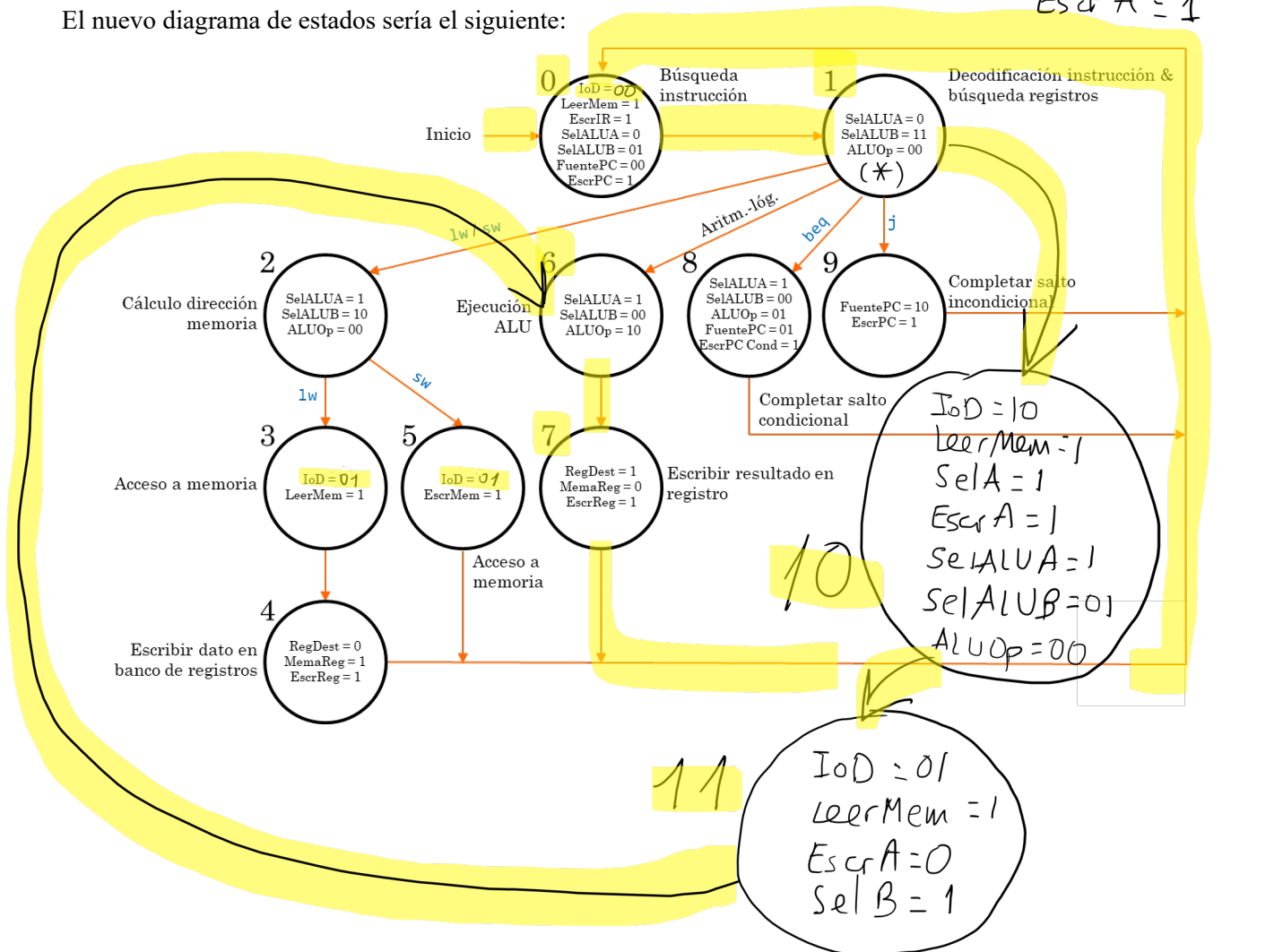
Nótese que, si hubiéramos elegido el formato I, entonces este sería un estado nuevo y no podría ser el 6, ya que la señal **ALUOp = 10** solo funciona en el formato R (la ALU necesita el campo de función para saber que tiene que sumar). En ese hipotético estado nuevo, pondríamos **ALUOp = 00**.

Ciclo 6 (WB)/Estado 7: En este último ciclo se guarda el resultado de la suma en el banco de registros, también como si fuera una instrucción **add**.

$$\text{Reg}[\text{IR}[15-11]] \leftarrow \text{Salida ALU}$$

Las señales son las habituales para este estado.

El nuevo diagrama de estados sería el siguiente:



La secuencia de estados de la nueva instrucción sería: $0 \rightarrow 1 \rightarrow 10 \rightarrow 11 \rightarrow 6 \rightarrow 7 \rightarrow 0$.

La nueva tabla de verdad de las salidas de la unidad de control principal sería:

Estado actual	EsCrPC	EsCrPCCond	Iod1	Iod0	LeerMem	EsCrMem	EsCrIR	MemReg	FuentePC1	FuentePC0	ALUp1	ALUp0	SeLALUB1	SeLALUB0	SeLALUA	EsCrReg	RegDest	SeLA	EsCrA	SeLB
0000	1	0	0	0	1	0	1	X	0	0	0	0	0	1	0	0	X	X	X	X
0001	0	0	X	X	0	0	0	X	X	X	0	0	1	1	0	0	X	0	1	0
0010	0	0	X	X	0	0	0	X	X	X	0	0	1	0	1	0	X	X	X	X
0011	0	0	0	1	1	0	0	X	X	X	X	X	X	X	X	0	X	X	X	X
0100	0	0	X	X	0	0	0	1	X	X	X	X	X	X	X	1	0	X	X	X
0101	0	0	0	1	0	1	0	X	X	X	X	X	X	X	X	0	X	X	X	X
0110	0	0	X	X	0	0	0	X	X	X	1	0	0	0	1	0	X	X	X	X
0111	0	0	X	X	0	0	0	0	X	X	X	X	X	X	X	1	1	X	X	X
1000	0	1	X	X	0	0	0	X	0	1	0	1	0	0	1	0	X	X	X	X
1001	1	0	X	X	0	0	0	X	1	0	X	X	X	X	X	0	X	X	X	X
1010	0	0	1	0	1	0	0	X	X	X	0	0	0	1	1	0	X	1	1	X
1011	0	0	0	1	1	0	0	X	X	X	X	X	X	X	X	0	X	X	0	1

Microoperaciones realizadas por la instrucción, desde el punto de vista de la organización multiciclo:

- 1: $IR \leftarrow M[PC]$
 $PC \leftarrow PC + 4$
- 2: $A \leftarrow \text{Reg}[IR[25-21]]$
 $B \leftarrow \text{Reg}[IR[20-16]]$
 $\text{Salida ALU} \leftarrow PC + \text{extSigno16a32}(IR[15-0]) \ll 2$
- 3: $A \leftarrow M[A]$
 $\text{Salida ALU} \leftarrow A + 4$
- 4: $B \leftarrow M[\text{Salida ALU}]$
- 5: $\text{Salida ALU} \leftarrow A + B$
- 6: $\text{Reg}[IR[15-11]] \leftarrow \text{Salida ALU}$

18. [Ejercicio de examen] Se desea modificar el camino de datos multiciclo del procesador MIPS para añadir un nuevo modo de direccionamiento para los operandos de las instrucciones aritmético-lógicas, en particular el llamado “direccionamiento indirecto a registro”. Consiste en que un operando de la instrucción está ubicado en una determinada posición de la memoria y, para indicar cuál es dicha dirección de memoria, se utiliza un registro del banco. La notación estándar en ensamblador es escribir el registro entre paréntesis. Supongamos el caso extremo en el que los tres operandos utilizan dicho direccionamiento, es decir:

`add ($s0), ($s1), ($s2)`

Esta instrucción lee dos datos en memoria. Uno está ubicado en la dirección de memoria que coincide con el valor del registro `$s1`. El otro está en la dirección de memoria que coincide con `$s2`. Ambas palabras de memoria se suman en la ALU y el resultado de la operación se ha de escribir en la dirección de memoria que coincide con el valor del registro `$s0`.

Se pide:

- a) Desde el punto de vista de la **arquitectura** (tal cual se vería en una tabla de resumen de instrucciones), escribir qué operación hace esta instrucción en notación de **pseudocódigo**.
- b) Justificar qué **formato binario** de instrucción sería el más adecuado para esta instrucción y descomponerla en campos. Habrá que indicar también **todos los operandos** de la instrucción, así como los **modos de direccionamiento** de cada uno de ellos.
- c) Sabiendo que la instrucción estándar `add $s0, $s1, $s2` tiene código de operación `000000`, ¿podría usarse también este código de operación para la nueva instrucción? Razonar la respuesta.
- d) Explicar razonada y detalladamente **todos los cambios** que son necesarios en el camino de datos para poder ejecutar esta instrucción. Los requisitos que debe cumplir la instrucción son los siguientes:
 - El hecho de añadir esta nueva instrucción debe ser retrocompatible con todas las instrucciones estándar, que deben seguir siendo ejecutables como hasta ahora.
 - El número total de ciclos para ejecutar la nueva instrucción debe ser el mínimo posible. Para ello se podrán solapar en el mismo ciclo operaciones simultáneas que no interfieran entre sí.
 - El número total de estados de la unidad de control debe ser el mínimo posible.
 - El número total de cambios en el camino de datos debe ser el mínimo posible y también estos cambios han de ser lo más pequeños posible.

Este apartado tiene tres puntos de vista interrelacionados que habrá que explicar:

- Por un lado, habrá que **explicar** las modificaciones y **dibujarlas** sobre el camino de datos, tales como añadir cables o ampliar otros existentes, añadir multiplexores o ampliar otros existentes, añadir registros o ampliar otros existentes, añadir puertas lógicas o ampliar las existentes, etc.
- Por otro, habrá que **explicar, ciclo a ciclo**, qué operaciones se realizan (desde el punto de vista de la **organización**, es decir, indicando los componentes concretos del camino de datos involucrados). Para ello, se deberá usar la notación de **pseudocódigo** con el operador de asignación flecha $\leftarrow$ para indicar todas las operaciones que intervienen en dicho ciclo. Además del pseudocódigo:
 - Si el ciclo corresponde a un estado nuevo de la unidad de control, habrá que explicar e indicar el valor de **todas las señales de control relevantes**.
 - Si el ciclo corresponde a un estado existente que ahora es modificado, habrá que explicar e indicar **solo** el valor de las señales de control **nuevas o modificadas**.
 - Si el ciclo corresponde a un estado existente que no se modifica, **no** será necesario explicar ni indicar el valor de las señales de control.
- Finalmente, habrá que ampliar el **diagrama de estados** de la unidad de control, así como la **tabla de verdad** de las salidas de la **unidad de control** para que incluyan los cambios realizados. Ambos se adjuntan con el enunciado.

Solución:

a) Esta instrucción realiza las siguientes operaciones **desde el punto de vista de la arquitectura**:

$$PC \leftarrow PC + 4$$

$$M[\$s0] \leftarrow M[\$s1] + M[\$s2]$$

b) Este ejercicio, como todos los de este tema, puede tener varias soluciones válidas. En concreto, la instrucción tiene formato R ya que contiene 3 registros.

31	...	26	25	...	21	20	...	16	15	...	11	10	...	6	5	...	0
Código de operación			RF1			RF2			RD			Shamt			Campo de función		
←	6	→	←	5	→	←	5	→	←	5	→	←	5	→	←	6	→

En este caso, el campo de desplazamiento (shamt) estaría sin usar y contendría el valor 00000. Por otro lado, el registro fuente RF1 sería \$s1 (el 17 del banco de registros, es decir, el 10001 en binario puro), el registro fuente RF2 sería \$s2 (el 18 o 10010), mientras que el registro destino RD sería \$s0 (el 16 o 10000). Del valor del código de operación y el campo de función hablaremos en el siguiente apartado.

Esta instrucción tiene estos cuatro operandos:

- Dos datos fuente que hay que leer en memoria, cuya dirección se especifica mediante los registros \$s1 y \$s2. El direccionamiento sería el nuevo que introduce este ejercicio: indirecto a registro.
- Una posición de memoria donde vamos a escribir el resultado de la suma, cuya dirección se especifica mediante el registro \$s0. El direccionamiento también sería indirecto a registro.
- El registro PC, que se actualiza sumándole 4 para que apunte a la siguiente instrucción (modo de direccionamiento implícito).

c) Los 6 bits del código de operación son compartidos por todas las instrucciones aritmético-lógicas de formato R (000000). Para distinguirlas, se colocan distintos códigos en los 6 bits menos significativos (campo de función). Esto da lugar a dos alternativas para la solución de este ejercicio:

- Si usáramos también el código de operación 000000 para esa nueva instrucción, necesitaríamos un código de función diferente al de la suma habitual. Para la instrucción add con 3 registros normales (es decir, con direccionamiento directo a registro), el campo de función vale 100000. La nueva instrucción con 3 direccionamientos indirectos a registros necesitaría un campo de función diferente para poder distinguirse de la suma normal. Como este campo es de 6 bits, habría 64 combinaciones y previsiblemente alguna estaría sin usar y podríamos elegirla de la hoja resumen de instrucciones MIPS.

Por tanto, a priori esta podría ser una posibilidad. Sin embargo, esta elección también induce cambios inesperados que hacen que la solución de este ejercicio sea mucho más complicada. Y es que la nueva instrucción sigue una secuencia de ciclos muy diferente a la del resto de instrucciones aritmético-lógicas, como veremos después. A ojos de la unidad de control principal, sin embargo, ambas secuencias de ejecución son idénticas, ya que le proporcionamos solo el código de operación 000000, con lo que espera realizar 4 etapas de ejecución. Para arreglar esto, los 6 bits del campo de función también habrían de proporcionarse a la unidad de control principal para así distinguir cuándo hacemos una suma normal de una suma de las nuevas. Pero esto va en contra del proceso de diseño del procesador que hemos seguido en todo el tema, en el que hemos separado la unidad de control en dos partes independientes: la que

identifica la instrucción y solo recibe el código de operación y la que identifica la operación aritmético-lógica y recibe el campo de función y la señal **ALUOp**.

En resumen, seleccionar el mismo código de operación que usan el resto de las instrucciones aritmético-lógicas nos obliga a rediseñar TODA la unidad de control. Esta opción queda descartada.

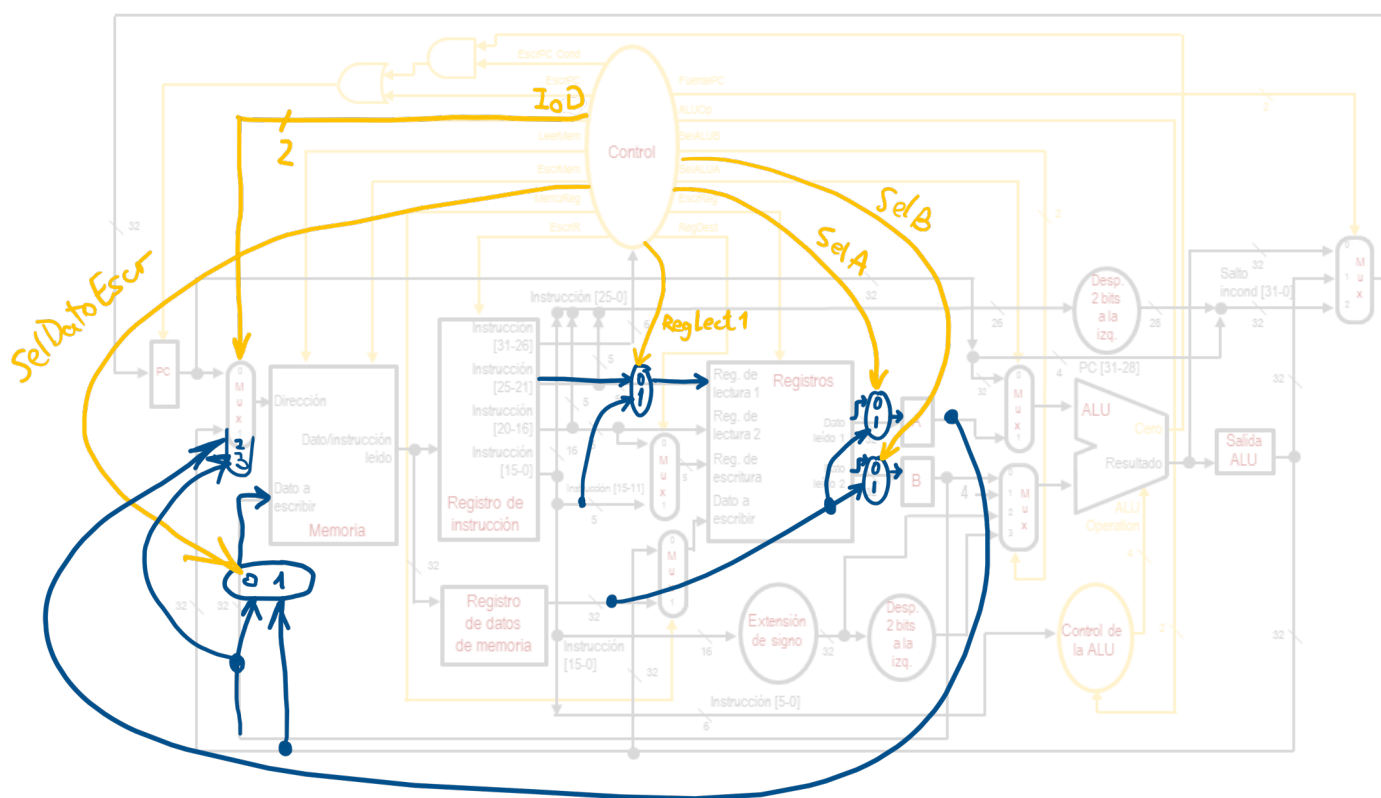
- La otra posibilidad es hacer que la nueva instrucción tenga un código de operación propio y, por tanto, distinto de **000000**. Consultando la hoja resumen de instrucciones MIPS es fácil encontrar uno que esté sin usar para asignarlo a esta instrucción. Sin embargo, el campo de función valdrá **100000**, igual que en la instrucción **add** normal.

Esta solución permite que la unidad de control pase del estado 1 (ciclo ID) a un nuevo estado 10, específico para la nueva instrucción, simplemente conociendo el código de operación, sin necesidad de consultar el código de función. De esta forma, los cambios en la unidad de control son mínimos. Es por ello por lo que continuaremos este ejercicio suponiendo que esta es la opción escogida.

d) Las modificaciones que se proponen son las siguientes:

- Ampliar el multiplexor que elige la dirección de acceso a memoria para que sea de 4 entradas. La señal de control **IoD** pasa a ser de 2 bits (**IoD1 IoD0**). Las entradas del multiplexor serán ahora:
 - 0: **PC** (como hasta ahora)
 - 1: **Salida ALU** (como hasta ahora)
 - 2: Registro **A**
 - 3: Registro **B**
- Añadir un multiplexor 2 a 1 en la entrada de Dato a escribir de la memoria. Necesitamos una nueva señal de control que llamaremos **SelDatoEscr**. Las entradas del multiplexor serían:
 - 0: Registro **B**
 - 1: **Salida ALU**
- En el banco de registros, añadimos un nuevo multiplexor 2 a 1 en la entrada correspondiente al Registro de Lectura 1. Para ello usaremos una nueva señal de control que llamaremos **RegLect1**. Las entradas del multiplexor son:
 - 0: **IR**[25-21]
 - 1: **IR**[15-11]
- Añadir un nuevo multiplexor 2 a 1 en la entrada del registro A. La nueva señal de control será **SelA**. Las entradas del multiplexor serán:
 - 0: Dato leído 1 del banco
 - 1: Registro **MDR**
- Añadir un nuevo multiplexor 2 a 1 en la entrada del registro B. La nueva señal de control será **SelB**. Las entradas del multiplexor serán:
 - 0: Dato leído 2 del banco
 - 1: Registro **MDR**

Los cambios propuestos son para cumplir los requisitos del problema. En particular, hay que leer los registros **A** y **B** y llevar sus valores (por turnos) a la entrada de dirección de memoria. Por eso hay que ampliar el multiplexor correspondiente de 2 a 4 entradas.



La ejecución ciclo a ciclo de la nueva instrucción sería la siguiente:

Ciclo 2 (ID)/Estado 1: La decodificación de la instrucción se realiza como en el resto de las instrucciones, salvo por el hecho de que ahora existen las señales **RegLect1**, **Se1A** y **Se1B**.

$A \leftarrow \text{Reg}[\text{IR}[25-21]]$
 $B \leftarrow \text{Reg}[\text{IR}[20-16]]$
 $\text{Salida ALU} \leftarrow \text{PC} + \text{extSigno16a32}(\text{IR}[15-0]) \ll 2$

El valor de **Salida ALU** calculados en este ciclo es irrelevante para esta instrucción, pero aun así se calcula por retrocompatibilidad con el resto de las instrucciones.

Las señales de control son las habituales para este estado.

Ciclo 3 (MEM)/Estado 10: Llevamos el valor del registro **A** (que contiene **\$s1**) a la entrada de dirección de memoria y activamos la lectura.

$\text{MDR} \leftarrow M[A]$

Señales de control:

EscrPC = 0 (protegemos el **PC** contra escrituras indeseadas)
EscrPC Cond = 0 (no es instrucción condicional)
IoD = 10 (seleccionamos **A** como dirección de memoria)
LeerMem = 1 (se lee la memoria)
EscrMem = 0 (si se lee la memoria, no se puede escribir en ella)
EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)
EscrReg = 0 (protegemos el banco de registros contra escrituras indeseadas)

El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 4 (MEM)/Estado 11: Llevamos el valor del registro **B** (que contiene **\$s2**) a la entrada de dirección de memoria y activamos la lectura. Mientras tanto, llevamos el valor de **MDR** (que contiene $M[A]$) al registro **A**.

$A \leftarrow \text{MDR}$
 $\text{MDR} \leftarrow M[B]$

Señales de control:

EscrPC = 0 (protegemos el **PC** contra escrituras indeseadas)
EscrPC Cond = 0 (no es instrucción condicional)
IoD = 11 (seleccionamos **B** como dirección de memoria)
LeerMem = 1 (se lee la memoria)
EscrMem = 0 (si se lee la memoria, no se puede escribir en ella)
EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)
EscrReg = 0 (protegemos el banco de registros contra escrituras indeseadas)
Se1A = 1 (seleccionamos el registro **MDR** para guardarlo en **A**)

El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 5 (EX)/Estado 12: Llevamos el valor de **MDR** (que contiene $M[B]$) al registro **B**.

$B \leftarrow MDR$

Señales de control:

EscrPC = 0 (protegemos el **PC** contra escrituras indeseadas)

EscrPC Cond = 0 (no es instrucción condicional)

LeerMem = 0 (no hay que leer la memoria)

EscrMem = 0 (protegemos la memoria contra escrituras indeseadas)

EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)

EscrReg = 0 (protegemos el banco de registros contra escrituras indeseadas)

Se1B = 1 (seleccionamos el registro **MDR** para guardarlo en **B**)

El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 6 (EX)/Estado 13: En este ciclo realizamos la suma de los valores contenidos en los registros **A** y **B**. Nótese que hemos colocado el código de la suma 100000 en el campo de función y la señal **ALUOp** vale 10, con lo que la unidad de control de la ALU genera la señal **ALU Operation** correspondiente a la suma. Por otra parte, podemos aprovechar el tiempo para leer el registro **\$s0**, cuyo índice está en **IR[15-11]**, y almacenar su valor en **A**. Se recuerda que el enunciado nos permite, en el mismo ciclo, leer los registros **A** y **B** como operandos de la ALU y luego escribir un nuevo valor en **A**, como ocurre en este caso.

$Salida\ ALU \leftarrow A + B$

$A \leftarrow Reg[IR[15-11]]$

Señales de control:

EscrPC = 0 (protegemos el **PC** contra escrituras indeseadas)

EscrPC Cond = 0 (no es instrucción condicional)

LeerMem = 0 (no hay que leer la memoria)

EscrMem = 0 (protegemos la memoria contra escrituras indeseadas)

EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)

ALUOp = 10 (sumar, ya que el campo de función vale 100000)

Se1ALUB = 00 (seleccionamos **B** como segundo operando de la ALU)

Se1ALUA = 1 (seleccionamos **A** como segundo operando de la ALU)

EscrReg = 0 (protegemos el banco de registros contra escrituras indeseadas)

RegLect = 1 (seleccionamos **IR[15-11]** como primer registro que hay que leer en el banco)

Se1A = 0 (seleccionamos el registro que nos proporciona el banco para guardarlo en **A**)

El resto de las señales son irrelevantes para este ciclo (X).

Ciclo 7 (MEM)/Estado 14: En este último ciclo, llevamos el valor del registro **A** (que contiene **\$s0**) a la entrada de dirección de memoria y activamos la escritura. El valor que toca escribir es precisamente el almacenado en **Salida ALU**.

$$M[A] \leftarrow \text{Salida ALU}$$

Señales de control:

EscrPC = 0 (protegemos el **PC** contra escrituras indeseadas)

EscrPC Cond = 0 (no es instrucción condicional)

IoD = 10 (seleccionamos **A** como dirección de memoria)

LeerMem = 0 (si se escribe en memoria, no puede leerse)

EscrMem = 1 (se escribe en memoria)

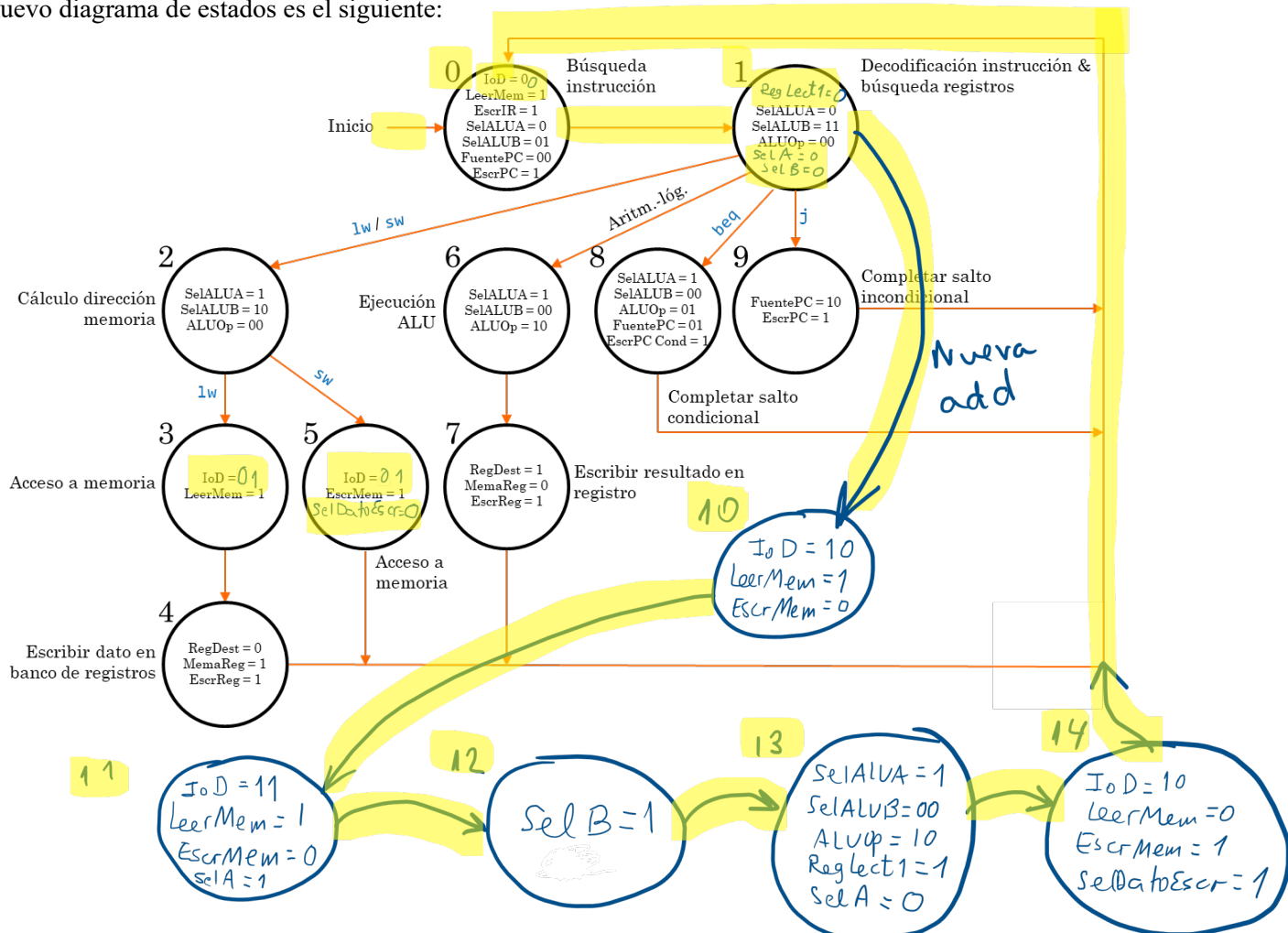
EscrIR = 0 (protegemos el registro **IR** contra escrituras indeseadas)

EscrReg = 0 (protegemos el banco de registros contra escrituras indeseadas)

SelDatoEscr = 1 (seleccionamos **Salida ALU** como el dato que vamos a escribir en memoria)

El resto de las señales son irrelevantes para este ciclo (X).

El nuevo diagrama de estados es el siguiente:



Hemos añadido cinco nuevos estados: del 10 al 14. También hemos hecho que la señal **IoD** sea de 2 bits, lo que nos ha obligado a modificar también varios estados, como el 0, el 3 y el 5. También hemos modificado el estado 5 al haber añadido la nueva señal **SelDatoEscr**, así como el estado 1 al haber añadido las señales **SelA**, **SelB** y **RegLect1**.

La secuencia de estados de la nueva instrucción sería: 0 → 1 → 10 → 11 → 12 → 13 → 14 → 0.

La nueva tabla de verdad de las salidas de la unidad de control principal sería:

Estado actual	EscrPC	EscrPCCond	IoD1	IoD0	LeerMem	EscrMem	EscrIR	MemReg	FuentePC1	FuentePC0	ALUp1	ALUp0	SelALUB1	SelALUB0	SelALUA	EscrReg	RegDest	RegLect1	SelA	SelB	SelDatoEscr
0000	1	0	0	0	1	0	1	X	0	0	0	0	0	1	0	0	X	X	X	X	X
0001	0	0	X	X	0	0	0	X	X	X	0	0	1	1	0	0	X	0	0	0	X
0010	0	0	X	X	0	0	0	X	X	X	0	0	1	0	1	0	X	X	X	X	X
0011	0	0	0	1	1	0	0	X	X	X	X	X	X	X	X	0	X	X	X	X	X
0100	0	0	X	X	0	0	0	1	X	X	X	X	X	X	X	1	0	X	X	X	X
0101	0	0	0	1	0	1	0	X	X	X	X	X	X	X	X	0	X	X	X	X	0
0110	0	0	X	X	0	0	0	X	X	X	1	0	0	0	1	0	X	X	X	X	X
0111	0	0	X	X	0	0	0	0	X	X	X	X	X	X	X	1	1	X	X	X	X
1000	0	1	X	X	0	0	0	X	0	1	0	1	0	0	1	0	X	X	X	X	X
1001	1	0	X	X	0	0	0	X	1	0	X	X	X	X	X	0	X	X	X	X	X
1010	0	0	1	0	1	0	0	X	X	X	X	X	X	X	X	0	X	X	X	X	X
1011	0	0	1	1	1	0	0	X	X	X	X	X	X	X	X	0	X	X	1	0	X
1100	0	0	X	X	0	0	0	X	X	X	X	X	X	X	X	0	X	X	0	1	X
1101	0	0	X	X	0	0	0	X	X	X	1	0	0	0	1	0	X	1	0	X	X
1110	0	0	1	0	0	1	0	X	X	X	X	X	X	X	X	0	X	X	X	X	1

La antigua señal **IoD** ahora corresponde a **IoD0**. Hemos añadido las filas correspondiente a los estados 10 (1010), 11 (1011), 12 (1100), 13 (1110) y 14 (1110), así como las columnas correspondientes a las nuevas señales **IoD1**, **RegLect1**, **SelA**, **SelB** y **SelDatoEscr**.

Para los nuevos estados, las señales **EscrPC**, **EscrPC Cond**, **LeerMem**, **EscrMem**, **EscrIR** y **EscrReg** se han mantenido a 0 para evitar que se modifique la memoria, el registro de instrucción o el banco de registros, salvo que en los estados 10 y 11 se activa **LeerMem** y en el estado 14 se activa **EscrMem**. El resto de las señales de control se han marcado a X al no intervenir en la instrucción y ser su valor irrelevante.

Microoperaciones realizadas por la instrucción, desde el punto de vista de la organización multiciclo:

```

1: IR ← M[PC]
   PC ← PC + 4
2: A ← Reg[IR[25-21]]
   B ← Reg[IR[20-16]]
   Salida ALU ← PC + extSigno16a32(IR[15-0])<<2
3: MDR ← M[A]
4: A ← MDR
   MDR ← M[B]
5: B ← MDR
6: Salida ALU ← A + B
   A ← Reg[IR[15-11]]
7: M[A] ← Salida ALU

```